

Конспект лекций по курсу «Разработка Java-приложений»

Ввод-вывод

Функции ввода-вывода (input/output, или I/O) информации, реализованные в составе стандартного пакета java.io, определены в терминах потоков данных (streams). **Потоки данных** – это упорядоченные последовательности данных, которым соответствует определённый источник (для потоков ввода) или получатель (для потоков вывода). Большинство типов потоков данных *(например, тех, которые реализуют файловые операции)* поддерживает методы определённых базовых интерфейсов и абстрактных классов с некоторыми дополнениями. *Мы вначале рассмотрим именно базовые интерфейсы и абстрактные классы, а затем посмотрим примеры практического воплощения абстрактных моделей в конкретных типах потоков данных.*

Пакет java.io охватывает определения типов двух основных разновидностей – символьных потоков (character streams) и байтовых потоков (byte streams). Под **символьными потоками** понимают последовательности 16-битовых символов Unicode, а каждому байту, как всегда, отвечает порция данных длиной 8 бит. Ввод-вывод может быть либо текстовым (text-based), либо бинарным (binary). Функции текстового ввода-вывода способны обращаться с потоками символов, поддающихся восприятию человеком (например, с исходными текстами программ), а средства обработки бинарных данных имеют дело с информацией, хранящейся в двоичном виде (например, с битовыми представлениями графических изображений). Символьные потоки используются в операциях текстового ввода-вывода, а байтовые – при работе с бинарными данными. *Потоки, ориентированные на поддержку байтовых данных, вообще говоря, не способны корректно переносить символьную информацию, и некоторые аспекты обработки последовательностей символов в контексте байтовых потоков просто лишены смысла, хотя последние могут с успехом применяться для представления символов на основе устаревших 7-ми и 8-мибитовых протоколов.* **Байтовые потоки** принято называть **потоками ввода** (input streams) и **потоками вывода** (output streams), а **символьные** – **потоками чтения** (readers) и **потоками записи** (writers). Почти для каждого потока ввода существует соответствующий поток вывода, а для большинства потоков ввода и вывода определены символьные потоки чтения и записи, обладающие схожими функциональными чертами, и наоборот.

Классы и интерфейсы из состава пакета java.io можно условно разделить на пять обширных групп:

- типы общего назначения, служащие для построения различных типов байтовых и символьных потоков: потоки ввода и вывода, чтения и записи, а также классы, позволяющие выполнять взаимные преобразования между ними;
- классы, определяющие разновидности потоков: фильтрованные потоки (filtered streams), буферизованные потоки (buffered streams),

канальные потоки (pipedReaders), а также специальные подвиды таких потоков, такие как поток чтения строк с сохранением нумерации (line number reader) и лексический анализатор потока чтения (stream tokenizer);

- специализированные потоковые классы и интерфейсы для ввода и вывода значений простых типов и строк;
- классы и интерфейсы для обработки файловых данных в стиле, не зависящем от особенностей платформы;
- классы и интерфейсы, поддерживающие механизм сериализации (serialization) объектов, который позволяет преобразовать объект в байтовый поток и при необходимости восстановить его из данных потока.

Объекты класса IOException используются многими методами ввода-вывода в качестве сигнала о возникновении исключительной ситуации. Некоторые классы, производные от IOException, служат для представления конкретных проблем ввода-вывода, но в большинстве случаев используются объекты IOException со строкой, содержащей описание ошибки.

Байтовые потоки

Существует ряд особенностей, присущих всем байтовым потокам данных. Например, все они поддерживают механизм открытия и закрытия. Поток открывается при создании объекта класса и, оставаясь открытым, позволяет осуществлять операции чтения и записи данных. Поток закрывается при вызове метода close объекта. Закрытие потока служит цели высвобождения системных ресурсов (таких как дескрипторы открытых файлов), которые используются потоком; объект потока должен быть утилизирован сразу по окончании работы с ним. Если поток явно не закрывается, он продолжает существовать и расходовать системные ресурсы. Потоки следует закрывать сразу после завершения их использования.

Общими для всех байтовых потоков данных являются также политика синхронизации и характеристики поведения при попытках одновременного доступа к ним со стороны нескольких потоков вычислений.

Прежде чем рассматривать детали конкретных типов байтового ввода-вывода, целесообразно изучить особенности соответствующих базовых абстрактных классов: InputStream и OutputStream.

InputStream

В составе абстрактного класса InputStream объявлены методы, обеспечивающие выполнение функций ввода байтовых данных из определённого источника. InputStream является базовым классом по отношению к потокам ввода, определённых в пакете java.io, и обладает методами, перечисленными ниже.

- `public abstract int read() throws IOException`
Вводит один байт данных и возвращает его в виде целого числа из диапазона от 0 до 255 (но не от -128 до 127); иными словами, байтовое значение трактуется как целое без знака. Если байтов, готовых для ввода, не существует ввиду достижения конца потока,

возвращается значение -1. Метод выполняет блокировку до тех пор, пока имеются доступные для ввода байты, не достигнут конец потока либо не выброшено исключение. Метод возвращает именно `int`, а не `byte`, поскольку необходимо обеспечить возможность получения как всех корректных значений типа `byte`, так и флага, свидетельствующего о достижении конца потока, и поэтому в качестве типа возвращаемого значения вместо `byte` используется более широкий тип `int`.

- `public int read(byte[] buf, int offset, int count) throws IOException`
Вводит байты данных и сохраняет их в части массива `buf` типа `byte`. Максимальное количество байтов, подлежащих вводу, определяется значением `count`. Байты заносятся в элементы массива, начиная с `buf[offset]` и до `buf[offset+count-1]`, если ранее не будет достигнут конец потока, содержимое всех остальных элементов массива `buf` остаётся неизменным. Метод возвращает количество фактически введенных байтов. Если ввиду достижения конца потока не введен ни один байт, возвращается -1. Если значение `count` равно нулю, ввод не производится и возвращается 0. Метод выполняет блокировку до тех пор, пока имеются доступные для ввода байты, не достигнут конец потока либо не выброшено исключение. Если первый байт не может быть введен по причине, не связанной с достижением конца потока (например, из-за того, что поток уже закрыт), выбрасывается исключение типа `IOException`. Как только успешно введен хотя бы один байт, любая ошибка, возникающая при попытке ввода последующих данных, не приводит к выбрасыванию исключения, а трактуется как событие получения признака конца потока – метод завершает выполнение нормальным образом и возвращает число байтов, введенных до момента возникновения ошибки.
- `public int read(byte[] buf) throws IOException`
Метод аналогичен предыдущему при условии `read(buf, 0, buf.length)`.
- `public long skip(long count) throws IOException`
Пропускает, самое большее, `count` байтов либо часть последовательности байтов до конца потока. Возвращает количество фактически пропущенных байтов. Если значение `count` отрицательно, операция не выполняется.
- `public int available() throws IOException`
Возвращает количество байтов, которые могут быть введены или пропущены, не вызывая блокировки. Реализация метода, предлагаемая по умолчанию, возвращает значение 0.
- `public void close() throws IOException`
Закрывает ранее открытый поток ввода. Метод используется для высвобождения ресурсов, связанных с потоком (таких как

дескрипторы открытых файлов). Любые попытки обращения к закрытому потоку приводят к выбрасыванию исключения типа `IOException`, но повторное закрытие потока эффектов не вызывает. В реализации метода, предлагаемой по умолчанию, никакие действия не предусмотрены.

При реализации `InputStream` требуется, чтобы производный класс обеспечил практическое воплощение варианта метода `read`, осуществляющего ввод единственного байта, поскольку остальные версии `read` основываются на первой. Во многих случаях, однако, производительность операций может быть улучшена за счёт переопределения и других методов. Исходные версии методов `available` и `close` нуждаются в переопределении в соответствии с потребностями конкретных типов потоков.

Следующая программа (пример 1) демонстрирует приёмы использования потоков ввода для подсчёта общего количества байтов в указанном файле либо в стандартном потоке `System.in`, если имя файла не задано.

Пример 1.

```
import java.io.*;
```

```
class CountBytes {
    public static void main(String[] args) throws IOException {
        InputStream in;
        if (args.length == 0)
            in = System.in;
        else
            in = new FileInputStream(args[0]);
        int total = 0;
        while (in.read() != -1)
            total++;
        System.out.println(total + " байтов");
    }
}
```

Программа принимает имя файла в виде параметра командной строки. Переменная `in` представляет байтовый поток ввода. Если имя файла не задано, используется стандартный поток ввода, `System.in` в противном случае создаётся объект типа `FileInputStream`, расширяющего класс `InputStream`. В цикле `while` подсчитывается общее количество байтов в файле, а затем на экран выводится результат. Так выглядит итог работы программы, которой в качестве параметра передано имя файла с её же исходным текстом:

«350 байтов»

Можно было бы попытаться вычислить значение `total`, воспользовавшись методом `available`, но для многих типов потоков результат, полученный таким образом, окажется, вообще говоря, неверным. Метод `available` возвращает количество байтов, которые можно ввести без блокировки. Если в качестве источника данных используется файл, `available`,

вероятно, вернёт как можно было предвидеть, число байтов, образующих содержимое файла в целом. Если поток `System.in` связан, скажем, с клавиатурой, результатом может оказаться даже нуль: когда входных данных, ожидающих обработки, нет, очередной вызов `read` вызовет блокировку.

OutputStream

Абстрактный класс `OutputStream` во многом аналогичен классу `InputStream`; он предлагает модель вывода байтовых данных в объект - получатель. Методы класса `OutputStream` перечислены ниже:

- `public abstract void write(int b) throws IOException`
Выводит значение `b` в виде байта. В качестве параметра передаётся именно `int`, поскольку значение часто является результатом выполнения арифметических операций над аргументами `byte`. Выражение, операнды которого относятся к типу `byte`, обладает типом `int`, поэтому объявление параметра в виде `int` означает возможность избежать необходимости использования операций явного преобразования результатов вычисления выражения к типу `byte`. Следует заметить, однако, что выводится только 8 младших битов результат типа `int`. Метод выполняет блокировку до тех пор, пока байт не выведен.
- `public void write(byte[] buf, int offset, int count) throws IOException`
Выводит `count` байтов массива `buf`, начиная с элемента `buf[offset]`. Метод выполняет блокировку до тех пор, пока байты не выведены.
- `public void write(byte[] buf) throws IOException`
Метод аналогичен предыдущему при условии `write(buf, 0, buf.length)`.
- `public void flush() throws IOException`
Осуществляет сброс (`flush`) потока. Если поток сохраняет в промежуточном буфере некоторое число байтов, переданных методами `write` различных версий, `flush` провоцирует выполнение операции непосредственного вывода данных в объект-получатель. Затем, если получателем является другой поток, тот в свою очередь также сбрасывается. Таким образом, единственный вызов `flush` приводит к сбросу всех буферов в цепочке взаимосвязанных потоков. Если поток не относится к категории буферизированных, реализация метода, предлагаемая по умолчанию, не предусматривает выполнения каких бы то ни было действий.
- `public void close() throws IOException`
Закрывает ранее открытый поток вывода. Метод используется для высвобождения ресурсов, связанных с потоком (таких как дескрипторы открытых файлов). Любые попытки обращения к закрытому потоку приводят к выбрасыванию исключения типа `IOException`, но повторное закрытие потока эффектов не вызывает. В реализации метода, предлагаемой по умолчанию, никакие действия не предусмотрены.

При реализации `OutputStream` требуется, чтобы производный класс обеспечил практическое воплощение варианта метода `write`, осуществляющего вывод единственного байта, поскольку остальные версии `write` основываются на первой. Во многих случаях, однако, производительность операций может быть улучшена за счёт переопределения и других методов. Исходные версии методов `flush` и `close` нуждаются в переопределении в соответствии с потребностями конкретных типов потоков - в частности, перед закрытием буферизованного потока разумно выполнять его сброс.

Символьные потоки

Абстрактными классами, обеспечивающими чтение и запись символьных потоков, являются `Reader` и `Writer`. В каждом из них предусмотрены методы, схожие с «собратьями» из состава классов, представляющих байтовые потоки (соответственно, `InputStream` и `OutputStream`). Например, класс `OutputStream` обладает методами для вывода данных из массивов типа `byte`, а `Writer` предлагает аналогичные методы для записи значений из массивов типа `char`. Классы символьных потоков появились в составе Java после реализации байтовых потоков и имели целью обеспечение полной поддержки таблиц символов Unicode. Как и в случае байтовых потоков, по окончании использования символьного потока его следует закрывать, чтобы высвободить используемые им системные ресурсы.

Reader

Абстрактный класс `Reader` обеспечивает поддержку символьного потока чтения аналогично тому, как это делает `InputStream`, реализующий модель байтового потока ввода, и методы `Reader` схожи с теми, которые объявлены в составе `InputStream`.

- `public int read() throws IOException`
Считывает один символ данных и возвращает его в виде целого числа из диапазона от 0 до 65535. Если символов, готовых для чтения, не существует ввиду достижения конца потока, возвращается значение -1. Метод выполняет блокировку до тех пор, пока имеются доступные для чтения символы, не достигнут конец потока либо не выброшено исключение.
- `public abstract int read(char[] buf, int offset, int count) throws IOException`
Считывает символы и сохраняет их в части массива `buf` типа `char`. Максимальное количество символов, подлежащих чтению, определяется значением `count`. Символы заносятся в элементы массива, начиная с `buf[offset]` и вплоть до `buf[offset+count-1]`, если ранее не будет достигнут конец потока, - содержимое всех остальных элементов массива `buf` остаётся неизменным. Метод возвращает количество фактически считанных символов. Если ввиду достижения конца потока не считан ни один символ, возвращается -1. Если значение `count` равно нулю, чтение не производится и возвращается 0. Метод выполняет блокировку до

тех пор, пока имеются доступные для чтения символы, не достигнут конец потока либо не выброшено исключение. Если первый символ не может быть считан по причине, не связанной с достижением конца потока (например, из-за того, что поток уже закрыт), выбрасывается исключение типа `IOException`. Как только успешно прочитан хотя бы один символ, любая ошибка, возникающая при попытке чтения последующих данных, не приводит к выбрасыванию исключения, а трактуется как событие получения признака конца потока, метод завершает выполнение нормальным образом и возвращает число символов, считанных до момента возникновения ошибки.

- `public int read(char[] buf) throws IOException`
Метод аналогичен предыдущему при условии `read(buf, 0, buf.length)`.
- `public long skip(long count) throws IOException`
Пропускает, самое большее, `count` символов либо часть последовательности символов до конца потока. Возвращает количество фактически пропущенных символов. Значение `count` не должно быть отрицательным.
- `public boolean ready() throws IOException`
Возвращает `true`, если поток готов для чтения данных, т.е. в нём существует хотя бы один доступный символ. Заметим, что результат, равный `false`, отнюдь не свидетельствует о том, что очередной вызов `read` приведёт к возникновению блокировки, поскольку в промежутке между обращениями к `ready` и `read` в поток может поступить очередная порция данных.
- `public abstract void close() throws IOException`
Закрывает ранее открытый поток чтения. Метод используется для высвобождения ресурсов, связанных с потоком (таких как дескрипторы открытых файлов). Любые попытки обращения к закрытому потоку приводят к выбрасыванию исключения типа `IOException`, но повторное закрытие потока эффектов не вызывает.

При реализации `Reader` требуется, чтобы производный класс обеспечил практическое воплощение варианта метода `read`, осуществляющего чтение данных в массив символов, и версии метода `close`. Во многих случаях, однако, производительность операций может быть улучшена за счёт переопределения и других методов.

Между классами `Reader` и `InputStream` существует ряд различий. В `Reader` базовым абстрактным методом чтения является вариант метода `read`, осуществляющий чтение в массив типа `char`, а остальные версии основываются на базовой. В классе `InputStream` в качестве основного объявлен метод `read`, выполняющий ввод одного байта. Классы, производные от `Reader`, обязаны предоставить реализацию абстрактного метода `close`: в отличие от наследования пустой реализации; многим классам потоков, как правило, необходимо «знать», был ли закрыт поток или нет, так что метод `close` обычно подлежит переопределению. Наконец, в классе `InputStream` предусмотрен

метод `available`, позволяющий выяснить, какой объём данных готов для ввода, а класс `Reader` предоставляет метод `ready`, свидетельствующий о том, существуют ли такие данные или нет.

В качестве примера приведём программу, выполняющую подсчёт общего количества символов и числа знаков пробела в символьном потоке чтения (пример 2).

Пример 2.

```
import java.io.*;
```

```
class CountSpace {
    public static void main(String[] args) throws IOException {
        Reader in;
        if (args.length == 0)
            in = new InputStreamReader(System.in);
        else
            in = new FileReader(args[0]);
        int ch;
        int total;
        int spaces = 0;
        for (total = 0; (ch = in.read()) != -1; total++) {
            if (Character.isWhitespace((char) ch))
                spaces++;
        }
        System.out.println(total + " символов, " + spaces + " пробелов ");
    }
}
```

Программа принимает имя файла в виде параметра командной строки. Переменная `in` представляет символьный поток чтения. Если имя файла не задано, используется стандартный поток ввода, `System.in`, после «вложения» его в объект типа `InputStreamReader`, выполняющий преобразование байтового потока ввода в символьный поток чтения; в противном случае создаётся объект типа `FileReader`, расширяющий класс `Reader`. В цикле `for` подсчитывается общее количество символов в файле; кроме того, с помощью метода `isWhitespace` класса `Character` выявляются символы пробела и вычисляется их количество, а затем на экран выводится результат. Так выглядит итог программы, которой в качестве параметра передано имя файла с её же исходным текстом:

477 символов , 130 пробелов

Writer

Абстрактный класс `Writer` обеспечивает поддержку символьного потока записи аналогично тому, как это делает `OutputStream`, реализующий модель байтового потока вывода, и многие методы `Writer` схожи с теми, которые объявлены в составе `OutputStream`; помимо этого, в классе `Writer` предусмотрены некоторые другие полезные версии метода `write`.

- `public void write(int ch) throws IOException`
Записывает `ch` в виде символа. Символ передаётся как значение `int`, но записывается только 16 битов последнего. Метод выполняет блокировку до тех пор, пока символ не записан.
- `public abstract void write(char[] buf, int offset, int count) throws IOException`
Записывает `count` символов массива `buf`, начиная с элемента `buf[offset]`. Метод выполняет блокировку до тех пор, пока символы не записаны.
- `public void write(char[] buf) throws IOException`
Метод аналогичен предыдущему при условии `write(buf, 0, buf.length)`.
- `public void write(String str, int offset, int count) throws IOException`
Записывает `count` символов строки `str`, начиная с символа `str.charAt(offset)`.
- `public void write(String str) throws IOException`
Метод аналогичен предыдущему при условии `write(str, 0, str.length())`.
- `public abstract void flush() throws IOException`
Осуществляет сброс (`flush`) потока. Если поток сохраняет в промежуточном буфере некоторое число символов, переданных методами `write` различных версий, `flush` провоцирует выполнение операции непосредственной записи данных в объект-получатель. Затем, если получателем является другой поток, тот в свою очередь также сбрасывается. Таким образом, единственный вызов `flush` приводит к сбросу всех буферов в цепочке взаимосвязанных потоков. Если поток не относится к категории буферизированных, никакие действия не выполняются.
- `public abstract void close() throws IOException`
Закрывает ранее открытый поток записи, выполняя при необходимости его сброс. Метод используется для высвобождения ресурсов, связанных с потоком (таких как дескрипторы открытых файлов). Любые попытки обращения к закрытому потоку приводят к выбрасыванию исключения типа `IOException`, но повторное закрытие потока эффектов не вызывает.

Классы, производные от `Writer`, обязаны обеспечить реализацию варианта метода `write`, связанного с записью символов из части массива, а также методов `close` и `flush`. Все остальные методы `Writer` основаны на трёх методах, названных выше. Это отличает `Writer` от `OutputStream`, в котором в качестве базового метода вывода предусмотрен вариант `write`, осуществляющий вывод одного байта, а для методов `close` и `flush` предложены реализации по умолчанию. Как и в случае класса `Reader`, производительность операций может быть улучшена за счёт переопределения в производных классах и других методов.

Символьные и стандартные потоки

Стандартные потоки `System.in`, `System.out` и `System.err` существовали в Java ещё до появления в языке символьных потоков, поэтому **стандартные потоки относятся к категории байтовых**, хотя в соответствии с элементарной логикой им следовало быть символьными. Такая ситуация приводит к возникновению ряда аномалий. Например, не представляется возможным непосредственно заменить `System.in` потоком типа `LineNumberReader`, обеспечивающим чтение строк с сохранением их нумерации. Объект `LineNumberReader` может быть создан при посредничестве объекта `InputStreamReader`, получающего в качестве параметра `System.in` и выполняющего преобразование байтового потока ввода в символьный поток чтения. `System.in`, на самом деле, есть `InputStream`, так что его нельзя непосредственно заменить объектом `LineNumberReader`, относящимся к типу `Reader`, а не `InputStream`.

`System.in`, на самом деле, есть `InputStream`, `System.out` и `System.err` есть объекты класса `PrintStream`. Устаревший и не рекомендованный для дальнейшего использования класс `PrintStream` был заменён аналогичным по назначению классом символьного потока `PrintWriter`. Создавать объекты `PrintStream` непосредственно не следует.

InputStreamReader и OutputStreamWriter

Эти классы обеспечивают возможность преобразования байтовых потоков ввода в символьные потоки чтения и символьных потоков записи в байтовые потоки вывода соответственно, с учётом заданной кодировки символов или кодировки, принятой по умолчанию в конкретной локальной системе. Объекту `InputStreamReader` в качестве источника передаётся байтовый поток ввода, и `InputStreamReader` обеспечивает чтение соответствующих символов Unicode. Объекту `OutputStreamWriter` в качестве получателя передаётся байтовый поток вывода, и `OutputStreamWriter` сохраняет в нём байтовые представления символов Unicode.

Описания конструкторов классов приведены ниже:

- `public InputStreamReader(InputStream in)`
Создаёт объект `InputStreamReader` для ввода данных из указанного потока `InputStream` с учётом кодировки символов, принятой по умолчанию.
- `public InputStreamReader(InputStream in, String encoding) throws UnsupportedOperationException`
Создаёт объект `InputStreamReader` для ввода данных из указанного потока `InputStream` с учётом заданной кодировки символов `encoding`. Если кодировка `encoding` не поддерживается, выбрасывается исключение типа `UnsupportedEncodingException`.
- `public OutputStreamWriter(OutputStream out)`
Создаёт объект `OutputStreamWriter` для вывода данных в указанный поток `OutputStream` с учётом кодировки символов, принятой по умолчанию.

- `public OutputStreamWriter (OutputStream out, String encoding)`
throws `UnsupportedEncodingException`
Создаёт объект `OutputStreamWriter` для вывода данных в указанный поток `OutputStream` с учётом заданной кодировки символов `encoding`. Если кодировка `encoding` не поддерживается, выбрасывается исключение типа `UnsupportedEncodingException`.

Методы `read` класса `InputStreamReader` обеспечивают ввод байтов из заданного потока `InputStream` и преобразование их в символы с использованием соответствующей кодировки. Аналогично, методы `write` класса `OutputStreamWriter` получают переданные символы, преобразуют их в байты, используя соответствующую кодировку, и выводят в заданный поток `OutputStream`.

В обоих классах при закрытии потока-преобразователя также закрывается и связанный с ним байтовый поток. *Такое поведение не всегда желательно (например, при преобразовании данных из стандартных потоков), поэтому вопрос, когда именно следует закрывать поток `InputStreamReader` или `OutputStreamWriter` заслуживает особого внимания.*

В обоих классах поддерживается метод `getEncoding`, возвращающий строку с каноническим наименованием кодировки, используемой потоком, либо значение `null`, если поток уже закрыт.

Классы `FileReader` и `FileWriter` являются расширенными версиями классов `InputStreamReader` и `OutputStreamWriter` соответственно и обеспечивают возможности обработки файловых данных с поддержкой `Unicode` и локальных стандартов кодировки символов.

Классов `ReaderInputStream` и `WriterOutputStream`, которые могли бы обеспечить трансляцию символьных потоков в байтовые и наоборот, не существует.

Краткий обзор классов потоков

В пакете `java.io` определён целый ряд типов потоков. Типы потоков обычно представлены парами классов, отвечающими за ввод (чтение) и вывод (запись) данных, и большинство из них реализовано как в байтовом, так и в символьном вариантах. Некоторые из потоков обладают свойствами, определяющими поведение остальных потоков.

Потоки Filter

Классы семейства фильтрованных потоков (filtered streams) `Filter` – `FilterInputStream`, `FilterOutputStream`, `FilterReader` и `FilterWriter` – позволяют объединять потоки в цепочки для получения составных потоков, обладающих расширенным набором функций. Каждый фильтрованный поток данных привязывается к другому потоку, которому передаёт полномочия по фактическому выполнению операций ввода или вывода.

Байтовые потоки `Filter` обладают дополнительными конструкторами, получающими в качестве параметров ссылки на объекты потоков данных соответствующего типа `InputStream` или `OutputStream`, которые обеспечивают ввод или вывод. Аналогично, в составе символьных потоков `Filter`

предусмотрены конструкторы, которым передаются ссылки на объекты потоков Reader или Writer, выполняющих чтение или запись данных.

Позволяется связывать в единую цепочку любое количество байтовых (символьных) Filter-потоков ввода (чтения). Источником исходных данных может быть поток, не относящийся к семейству потоков Filter. Байтовый поток ввода нетрудно преобразовать в символьный поток чтения с помощью объекта класса InputStreamReader.

Подобным образом могут соединяться в цепочки и Filter-потоки вывода (записи), так что данные, выводимые (записываемые) в один поток, будут отфильтрованы и переданы следующему потоку. Все потоки вывода (записи) в цепочке, начиная с первого и заканчивая предпоследним, должны относиться к классу Filter, но последним может быть объект любого потокового класса вывода (записи). Для преобразования символьного потока записи в байтовый поток вывода можно воспользоваться объектом класса OutputStreamWriter.

Не все классы семейства Filter в действительности осуществляют изменение данных. Некоторые просто привносят дополнительные черты поведения, а другие предлагают новые интерфейсы взаимодействия с потоками (например, классы типа Print).

Потоки Buffered

Классы семейства буферизованных потоков (buffered streams) Buffered – BufferedInputStream, BufferedOutputStream, BufferedReader и BufferedWriter – осуществляют буферизацию данных, позволяющую избежать необходимости обращения к источнику (получателю) данных при выполнении каждой отдельной операции read или write. Эти классы часто используются в сочетании с потоками семейства File: доступ к данным на диске выполняется намного медленнее, чем к информации в буфере памяти, и средства буферизации помогают снизить потребность в обращениях к диску.

В каждом из потоков Buffered поддерживаются два конструктора: один в качестве параметра принимает ссылку на объект «внутреннего» потока и значение объёма буфера, а другому передаётся только ссылка на объект потока – размер буфера предлагается по умолчанию.

Когда метод read потока Buffered, вызываемый для ввода (чтения) данных, обнаруживает, что буфер потока пуст, он вызывает одноимённый метод потока источника, заполняет буфер настолько большой порцией данных, насколько это возможно (выполняя блокировку только в том случае, если необходимо дождаться появления данных), и возвращает запрошенные данные из буфера. При очередных обращениях к методу read объекта Buffered данные будут извлекаться из буфера, пока его содержимое не исчерпается, и в этот момент объекту вновь придётся вызвать read потока-источника. Процесс повторяется до тех пор, пока не иссякнет источник данных.

Buffered – потоки вывода (записи) ведут себя аналогичным образом. Когда write заполняет буфер данных, вызывается одноимённый метод потока-получателя, освобождающий буфер. Такой механизм буферизации помогает превратить последовательность запросов на вывод (запись) небольших порций

данных, адресуемых объекту `Buffered`, в единственный вызов метода `write` потока-получателя.

Вот как можно построить байтовый буферизованный поток вывода данных в файл:

```
new BufferedOutputStream(new FileOutputStream(path));
```

Мы создаём объект класса `FileOutputStream`, передавая его конструктору аргумент строку, задающую путь к файлу, и на основе объекта `FileOutputStream` строим новый экземпляр класса `BufferedOutputStream`. Подобная схема позволяет осуществлять буферизацию байтовых данных, выводимых в файл.

Символьные потоки `Buffered` способны обращаться и со строками текста. Метод `newLine` класса `BufferedWriter` записывает в поток признак завершения строки. В каждой системе содержимое признака завершения строки, которое не обязательно должно ограничиваться единственным символом, задаётся по-разному; для этого используется одно из системных свойств типа `String`, `line.separator`. Обращаться к методу `newLine` для разграничения строк в текстовом файле следует, в частности, тогда, когда файл может быть открыт пользователем для визуального просмотра.

Метод `readLine` класса `BufferedReader` возвращает строку текста, считанную из потока, в виде объекта `String`. Метод способен распознавать разделители строк любого вида: перевод строки (`\n`), возврат каретки (`\r`) либо символ возврата каретки, сопровождаемый признаком перевода строки (`\r\n`). При этом предполагается, что вы не пользовались средством `line.separator` для определения иного признака завершения строки. В противном случае корректность распознавания методом `readLine` строк, разграниченных посредством `newLine`, не гарантируется. В строку, возвращаемую методом `readLine`, разделитель не включается. Если в процессе считывания признак конца потока встречается раньше признака завершения строки, `readLine` возвращает значение `null`.

Потоки Piped

Канальные потоки (pipedReaders), определяемые классами семейства `Piped` - `PipedInputStream`, `PipedOutputStream`, `PipedReader` и `PipedWriter` – используются в виде пар ввода-вывода (записи-чтения). Данные, переданные в поток вывода (записи), служат источником для потока ввода (чтения). С каналом связан внутренний буфер, ёмкость которого определяется при реализации класса, что позволяет поддерживать разные уровни производительности процессов вывода и ввода; однако средств динамического управления размером буфера не существует.

Каналы реализуют механизм обмена данными между различными потоками вычислений. Единственный безопасный способ обращения с потоками данных `Piped` связан с использованием двух потоков вычислений: один из них осуществляет вывод данных, а другой – их ввод. Когда буфер канала полностью заполняется, попытка вывода данных в него приводит к блокированию соответствующего потока вычислений. Если операции вывода и ввода выполняются одним потоком вычислений, тот будет блокирован

постоянно. Поток вычислений, осуществляющий ввод, блокируется, если буфер канала пуст.

Чтобы избежать опасности «вечного» блокирования одного потока вычислений, когда его «собрат» на другом конце канала прекращает работу, каждый канал отслеживает подлинность и работоспособность потоков вычислений, обращавшихся к нему с целью вывода и ввода данных последними. Канал, прежде чем заблокировать текущий поток вычислений, проверяет, «жив» ли поток на другом конце канала. Если обнаруживается, что работа противоположного потока вычислений прекращена, текущий поток генерирует исключение типа `IOException`.

Потоки `Piped` должны быть связаны друг с другом. Это можно сделать, передав при создании потоков `Piped` конструктору потока `PipedReader` в качестве аргумента ссылку на поток `PipedWriter`. Порядок построения объектов пары `Piped` несущественен: наоборот, можно передать конструктору `PipedWriter` ссылку на объект `PipedReader`.

Потоки `Piped` должны быть соединены, но сделать это можно позже, после их создания, поскольку в составе классов `Piped` предлагаются и конструкторы без аргументов. Для соединения канальных потоков применяется метод `connect`. Методу `PipedReader.connect` в качестве параметра передаётся объект `PipedWriter`, а методу `PipedWriter.connect` – `PipedReader`. Как и при использовании конструкторов, не важно, присоединяется ли объект `x` к объекту `y` или наоборот, объект `y` к `x`, поскольку результат в обоих случаях одинаков. При попытках использования потоков `Piped` до их соединения или связывания ранее соединённых потоков выбрасывается исключение типа `IOException`.

Байтовые потоки `ByteArray`

Массивы байтов, размещённые в оперативной памяти, могут выступать в роли источника или получателя данных при работе с потоками семейства `ByteArray`.

Объект класса `ByteArrayInputStream` использует массив типа `byte` в качестве источника данных. В составе класса предусмотрены два конструктора:

- `public ByteArrayInputStream(byte[] buf, int offset, int count)`
Создаёт объект потока `ByteArrayInputStream` на основе части заданного массива байтов `buf`, начиная с элемента `buf[offset]` и заканчивая `buf[offset+count-1]` либо последним элементом массива, в зависимости от того, какой из двух будет достигнут раньше. Массив используется непосредственно (он не копируется), поэтому в процессе ввода данные изменять не следует.
- `public ByteArrayInputStream(byte[] buf)`
Конструктор аналогичен предыдущему при условии `ByteArrayInputStream(buf, 0, buf.length)`

Ввод данных из потока `ByteArrayInputStream` никогда не блокируется.

В составе класса `ByteArrayOutputStream` предусмотрены средства динамического наращивания объёма массива типа `byte`, получающего выводимые данные. Перечислим конструкторы и методы класса.

- `public ByteArrayOutputStream()`
Создаёт объект потока `ByteArrayOutputStream`, предусматривающий вывод данных в массив типа `byte` с размером, предусмотренным по умолчанию.
- `public ByteArrayOutputStream(int size)`
Создаёт объект потока `ByteArrayOutputStream`, предусматривающий вывод данных в массив типа `byte` заданного размера `size`.
- `public byte[] toByteArray()`
Возвращает копию массива байтов, ранее выведенных потоком
- `public int size()`
Возвращает значение количества байтов, ранее выведенных потоком.
- `public void reset()`
Очищает буфер – массив потока, позволяя использовать его заново.
- `public String toString()`
Возвращает текущее содержимое буфера массива в виде объекта типа `String`, преобразуя байты в символы в соответствии с кодировкой символов, принятой по умолчанию.
- `public String toString(String enc) throws UnsupportedOperationException`
Возвращает текущее содержимое буфера – массива в виде объекта типа `String`, преобразуя байты в символы в соответствии с кодировкой символов, заданной строкой `enc`. Если указанная кодировка не поддерживается, выбрасывается исключение типа `UnsupportedEncodingException`.
- `public void writeTo(OutputStream out) throws IOException`
Выводит содержимое массива – буфера в заданный поток `out`.

Если `ByteArrayOutputStream` используется в качестве получателя данных, выводимых из «внешних» фильтрованных потоков, прежде чем обратиться к методу `toByteArray`, следует выполнить операцию сброса (`flush`) фильтрованных потоков.

Символьные потоки CharArray

Символьные потоки семейства `CharArray` (`CharArrayReader` и `CharArrayWriter`) аналогичны по назначению потокам `ByteArray` – они позволяют в качестве источника или получателя данных использовать массивы типа `char`, размещённые в памяти. *Конструкторы класса `CharArrayReader`, принимающие в качестве параметров массивы типа `char`, описаны ниже:*

- `public CharArrayReader(char[] buf, int offset, int length)`
Создаёт объект потока `CharArrayReader` на основе части заданного массива символов `buf`, начиная с элемента `buf[offset]` и заканчивая `buf[offset+length-1]` либо последним элементом массива, в зависимости от того, какой из двух будет достигнут раньше. Массив используется непосредственно (он не копируется), поэтому в процессе ввода данные изменять не следует.
- `public CharArrayReader (char[] buf)`
Конструктор аналогичен предыдущему при условии `CharArrayReader (buf, 0, buf.length)`

Ввод данных из потока `CharArrayReader` никогда не блокируется.

В составе класса `CharArrayWriter` предусмотрены средства динамического наращивания объёма массива типа `char`, получающего выводимые данные. Перечислим конструкторы и методы класса.

- `public CharArrayWriter ()`
Создаёт объект потока `CharArrayWriter`, предусматривающий вывод данных в массив типа `char` с размером, предусмотренным по умолчанию.
- `public CharArrayWriter (int size)`
Создаёт объект потока `CharArrayWriter`, предусматривающий вывод данных в массив типа `char` заданного размера `size`.
- `public char[] toCharArray()`
Возвращает копию массива символов, ранее выведенных потоком
- `public int size()`
Возвращает значение количества символов, ранее выведенных потоком.
- `public void reset()`
Очищает буфер – массив потока, позволяя использовать его заново.
- `public String toString()`
Возвращает текущее содержимое буфера массива в виде объекта типа `String`.
- `public void writeTo(OutputStream out) throws IOException`
Выводит содержимое массива – буфера в заданный поток `out`.

Если `CharArrayWriter` используется в качестве получателя данных, выводимых из «внешних» фильтрованных потоков, прежде чем обратиться к методу `toCharArray`, следует выполнить операцию сброса (`flush`) фильтрованных потоков.

Символьные потоки String

Объект класса `StringReader` позволяет считывать символы из строки `String` и никогда не блокируется. Единственный конструктор класса в качестве параметра принимает ссылку на объект `String`, содержимое которого подлежит чтению.

Объект класса `StringWriter` позволяет записывать символы в буфер, который может интерпретироваться как объект типа `String` или `StringBuffer`. Перечислим конструкторы и методы класса:

- `public StringWriter ()`
Создает объект потока `StringWriter`, предусматривающий вывод данных в буфер с размером, предусмотренным по умолчанию.
- `public StringWriter (int size)`
Создает объект потока `StringWriter`, предусматривающий вывод данных в буфер заданного размера `size`. Удачный выбор размера буфера во многих случаях способен привести к существенному росту производительности программы.
- `public StringBuffer getBuffer()`
Возвращает буфер данных, фактически используемый потоком, в виде объекта `StringBuffer`. Поскольку возвращается именно рабочий буфер, а не его копия, в процессе вывода данных изменять его напрямую не следует.
- `public String toString()`
Возвращает текущее содержимое буфера потока в виде объекта `String`.

Потоки Print

Классы семейства `Print` – `PrintStream` и `PrintWriter` - содержат объявление ряда методов, которые упрощают задачу вывода (записи) в поток значений простых типов и объектов в удобочитаемом текстовом формате. В потоковых классах `Print` реализованы перегруженные версии методов `print` и `println` для вывода следующих значений типов – `char`, `char[]`, `int`, `long`, `float`, `double`, `Object`, `String`, `boolean`. Эти методы гораздо удобнее для применения, нежели обычные методы потокового вывода (записи) `write`. Метод `println` добавляет в конец порции выводимых данных, переданной в виде параметра, признак завершения строки; инструкция `println` без параметров просто завершает текущую строку. Содержимое признака завершения строки определяется значением системного свойства `line.separator`, которое не обязательно должно ограничиваться единственным символом (например, `\n`).

Каждый из потоков `Print` действует как поток `Filter` и поэтому в процессе вывода данные могут быть подвержены дополнительной фильтрации.

Класс `PrintStream` работает с байтовыми потоками, а `PrintWriter` – с символьными. Поскольку чаще возникает потребность в записи символов, в обычных ситуациях следует пользоваться средствами класса `PrintWriter`. Исторически сложилось так, что стандартные потоки `System.out` и `System.err`, воспринимающие байты как символы из набора `Latin-1`, относятся к классу `PrintStream`, собственно говоря, это единственные потоки `PrintStream`, которые уместно применять на практике. Мы рассмотрим только класс `PrintWriter`, так как `PrintStream` реализует почти аналогичный интерфейс. В составе класса `PrintWriter` имеется четыре конструктора, которые описаны ниже:

- `public PrintWriter(Writer out, boolean autoflush)`
Создаёт объект класса `PrintWriter`, осуществляющий запись данных в заданный поток `out`. Если значение флага `autoflush` равно `true`, при обращении к `println` вызывается метод `flush`. В противном случае `println` не получает преимуществ перед другими методами, и сброс потока с помощью `flush` не производится. После создания потока свойство, связанное с автоматическим вызовом `flush`, изменено быть не может.
- `public PrintWriter(Writer out)`
Конструктор аналогичен предыдущему при условии `public PrintWriter(Writer out, false)`.
- `public PrintWriter(OutputStream out, boolean autoflush)`
Конструктор аналогичен первому при условии `PrintWriter(new OutputStreamWriter(out), autoflush)`.
- `public PrintWriter(OutputStream out)`
Конструктор аналогичен предыдущему при условии `PrintWriter(new OutputStreamWriter(out), false)`.

Одна из важных характеристик поведения потоков `Print` связана с тем, что ни один из реализованных в них методов вывода (записи) не выбрасывает исключения типа `IOException`. Если при передаче данных «внутреннему» потоку возникает ошибка, методы завершают выполнение нормальным образом. Проверить наличие ошибки можно с помощью вызова метода `checkError`, возвращающего результат типа `boolean`; метод осуществляет сброс (`flush`) потока и проверяет его состояние. Если произошла ошибка, средств очистки состояния потока не существует. Если любая из операций, выполняемых «внутренним» потоком, вызвала появление исключения `InterruptedIOException`, состояние ошибки не регистрируется, но вместо этого текущий поток вычислений заново получает уведомление о прерывании посредством `Thread.currentThread().interrupt()`.

Класс `StreamTokenizer`

Задачи лексического анализа потока данных относятся к числу традиционных и в составе пакета `java.io` представлен класс `StreamTokenizer`, позволяющий решать некоторые из них. Поток разбивается на лексемы с помощью объекта `StreamTokenizer`, конструктор которого принимает в качестве параметра объект типа `Reader`, выполняющий функцию источника данных; объект `StreamTokenizer` действует в соответствии с заданными параметрами сканирования данных. На каждой итерации цикла сканирования вызывается метод `nextToken`, который возвращает очередную считанную из потока лексему и информацию о её типе, присваивая эти данные полям объекта `StreamTokenizer`.

Класс `StreamTokenizer` ориентирован преимущественно на анализ текстов, представляющих код, написанный на каком-либо языке программирования; класс нельзя отнести к числу инструментов лексического анализа общего назначения.

Когда метод `nextToken` распознаёт лексему, он возвращает её тип в виде значения и присваивает последнее полю `ttype`. Существует четыре типа лексем, воспринимаемых методом `nextToken`:

- `TT_WORD`; обнаружено слово; оно сохраняется в поле `sval` типа `String`;
- `TT_NUMBER`; обнаружено число; оно сохраняется в поле `nval` типа `double`; распознаются только десятичные числа с плавающей запятой (с десятичной точкой или без таковой) в нормальной нотации (анализатор не распознает ни `3.4e79` как число с плавающей запятой, ни `0xffff` как шестнадцатеричное число);
- `TT_EOL`; обнаружен признак завершения строки;
- `TT_EOF`; достигнут конец файла.

Подразумевается, что текст, подлежащий анализу, состоит из байтов, значение которых относятся к диапазону от `\u0000` до `\u00FF`, - корректность распознавания символов Unicode, не принадлежащих указанному интервалу, не гарантируется. Поток ввода включает специальные и обычные символы. К категории специальных относятся те символы, которые анализатор трактует специальным образом, а именно: символы пробела, символы, обозначающие числа и слова, и т.д. Любой другой символ воспринимается как обычный. Когда очередным символом потока является обычный символ, в качестве его типа возвращается значение этого символа.

Рассмотрим код метода (пример 3), выполняющего суммирование числовых значений, которые вводятся из символьного потока.

Пример 3.

```
static double sumStream(Reader in) throws IOException {  
    StreamTokenizer nums = new StreamTokenizer(in);  
    double result = 0.0;  
    while (nums.nextToken() != StreamTokenizer.TT_EOF) {  
        if (nums.ttype == StreamTokenizer.TT_NUMBER)  
            result += nums.nval;  
    }  
    return result;  
}
```

Вначале на основе потока `in` типа `Reader` создаётся объект типа `StreamTokenizer`, а затем на каждом шаге цикла считывается очередная лексема; если лексема представляет собой число, его значение прибавляется к содержимому переменной `result`. По достижении конца потока значение `result` возвращается в код-инициатор.

Байтовые потоки *Data*

Технологии обработки символьной информации находят самое широкое применение, но достаточно часто возникают и задачи трансляции бинарных данных конкретных типов. В интерфейсах `DataInput` и `DataOutput` определены методы потокового ввода-вывода данных простых типов, а классы `DataInputStream` и `DataOutputStream` обеспечивают реализацию интерфейсов,

предлагаемую по умолчанию. *Вначале мы рассмотрим интерфейсы, а затем их реализации.*

DataInput и DataOutput

Интерфейсы, предусматривающие потоковый ввод и вывод бинарных данных, обладают почти одинаковой структурой. Ниже перечислены методы семейств read и write, обеспечивающих ввод и вывод данных каждого из простых типов:

readBoolean (writeBoolean), readChar (writeChar), readByte (writeByte), readShort (writeShort), readInt (writeInt), readLong (writeLong), readFloat (writeFloat), readDouble (writeDouble), readUTF (writeUTF).

readUTF (writeUTF) – чтение (запись) строк (String) в формате UTF (Unicode Transformation Format) – формат преобразования Unicode. Символы Unicode передаются в соответствии со спецификацией Unicode-1-1-UTF-8, определяющий компактный бинарный формат, предназначенный для кодирования 16-битовых символов Unicode посредством 8-битовых байтов.

Методы интерфейса DataInput обычно реагируют на событие достижения конца потока, выбрасывая исключение типа EOFException, производного от IOException.

Помимо перечисленных выше методов в составе интерфейсов DataInput и DataOutput есть ряд других методов. *Например, в интерфейсе DataInput*

public abstract void readFully (byte[] buf, int offset, int count) throws IOException

Осуществляет ввод данных из потока в часть байтового массива buf. Наибольшее количество байтов, подлежащих считыванию, задаётся параметром count. Байты сохраняются в элементах массива, начиная с buf[offset] и заканчивая buf[offset+count-1]. Если содержимое count равно нулю, ввод данных не производится. Метод выполняет блокировку до тех пор, пока имеются доступные для ввода байты, не достигнут конец потока (в этом случае генерируется исключение типа EOFException) либо не выброшено исключение, связанное с ошибкой ввода-вывода.

В интерфейсе DataOutput есть метод

public abstract void writeChars (String s) throws IOException
Выводит в поток содержимое строки s в виде последовательности значений char. Каждый символ сохраняется в виде пары байтов, и старший байт выводится первым.

Классы потоков Data

Для каждого интерфейса Data существует определённый поток Data. Кроме того, имеется класс RandomAccessFile, который реализует одновременно оба интерфейса Data – ввода и вывода. Каждый класс Data является расширением соответствующего класса Filter, так что потоки Data могут использоваться в целях фильтрации данных других потоков. В составе каждого класса Data есть конструкторы, в качестве параметра принимающие ссылки на другой подходящий поток ввода или вывода. Например, фильтрацию выводимых в файл данных можно осуществить, если передать конструктору DataOutputStream ссылку на объект FileOutputStream. Для

последующего ввода данных надлежит использовать поток `DataInputStream`, построенный на основе объекта `FileInputStream`. Примеры реализации соответствующих методов приведены в примере 4.

Пример 4.

```
public static void writeData(double[] data, String file) throws IOException {  
    OutputStream fout = new FileOutputStream(file);  
    DataOutputStream out = new DataOutputStream(fout);  
    out.writeInt(data.length);  
    for (int i = 0; i < data.length; i++)  
        out.writeDouble(data[i]);  
    out.close();  
}
```

```
public static double[] readData(String file) throws IOException {  
    InputStream fin = new FileInputStream(file);  
    DataInputStream in = new DataInputStream(fin);  
    double[] data = new double[in.readInt()];  
    for (int i = 0; i < data.length; i++)  
        data[i] = in.readDouble();  
    in.close();  
    return data;  
}
```

В теле метода `writeData` вначале открывается файл, в который записывается длина массива, подлежащего сохранению. Далее в цикле осуществляется вывод элементов массива. Содержимое файла может быть считано с помощью метода `readData`.

Класс *File*

Класс `File` реализует ряд полезных средств для работы с файлами и директориями: создание, проверка атрибутов, удаление, переименование.

- *exists*; возвращает `true`, если файл существует в файловой системе
- *canWrite*; возвращает `true`, если файл существует и его содержимое может быть изменено.

В составе класса `File` также объявлены методы, которые позволяют создавать временные файлы, используемые, например, для хранения промежуточных результатов и удаляемые при завершении работы программы.

Потоки *File*

Потоковые классы семейства `File` – `FileInputStream`, `FileOutputStream`, `FileReader` и `FileWriter` – позволяют трактовать файл как поток, предназначенный для ввода(чтения) или вывода(записи) данных. В составе каждого из типов предусмотрены три конструктора, принимающие в качестве параметра одно из следующих значений:

- строку `String`, задающую имя файла;
- объект класса `File`, указывающий на файл;
- объект `FileDescriptor`.

Если при использовании байтового или символьного потоков ввода (чтения), обнаруживается, что файл не существует, выбрасывается исключение типа `FileNotFoundException`. При доступе к файлу во всех случаях проверяется наличие соответствующих полномочий: если необходимые права на обращение к файлу отсутствуют, генерируется исключение типа `SecurityException`.

Два первых конструктора при построении байтового или символьного потоков вывода(записи) предусматривают создание файла, если такого не существует, при наличии файла его содержимое усекается. Процессом усечения данных можно управлять, используя соответствующие варианты конструкторов с двумя параметрами: строкой `String`, задающей имя файла, и флагом типа `boolean`, который, будучи равным `true`, устанавливает, что в процессе записи каждая новая порция данных должна передаваться в поток, не уничтожая ранее сохранённого содержимого. Если в качестве значения флага задаётся `false`, вместо прежних данных будут записаны новые. Если файл не существует, он будет создан (значение флага в этом случае во внимание не принимается).

Объект `FileDescriptor` служит для представления сущности, описывающий открытый файл и определяемой особенностями применяемой операционной системы. Объект дескриптора открытого файла может быть получен вызовом метода `getFD` объекта байтового потока `File`, но его нельзя определить для символьных потоков `File`. Объект `FileDescriptor` может быть применен для создания нового потока `File` на основе того же файла, который используется другим потоком, информация об имени и местоположении файла в данном случае не нужна. *Следует избегать опасности непредусмотренного взаимодействия между потоками, выполняющими различные функции над данными одного и того же файла: невозможно предсказать, что случится, например, если два потока вычислений примутся одновременно записывать данные в один файл посредством двух различных объектов `FileOutputStream`.*

Сериализация объектов

Одной из существенных возможностей многих реальных приложений является их способность преобразовывать объекты в байтовые потоки, которые могут передаваться в пределах сети - скажем, с целью использования в рамках технологии удалённого вызова метода, - сохраняться на диске в виде файлов, а затем при необходимости восстанавливаться в форме «живых» объектов. Процесс преобразования содержимого объекта в поток байтов принято называть **сериализацией объекта** (`object serialization`), а обратное преобразование — восстановление объекта из данных потока — **десериализацией** (`deserialization`). Потоки `Object` — `ObjectOutputStream` и `ObjectInputStream` - позволяют осуществлять сериализацию и десериализацию объектов.

`ObjectInputStream` и `ObjectOutputStream`, помимо данных стандартных классов (простых типов, строк и их массивов) позволяют вводить и выводить **графы объектов** (`object graphs`). Под термином граф объекта имеется в виду,

что когда содержимое объекта выводится в поток `ObjectOutputStream` средствами методов `writeObject`, в потоки сохраняются наборы байтов, представляющие и текущий объект, и все другие объекты, на который тот ссылается. Поскольку данные об объекте, подвергшимся сериализации, представляются в форме байтов, в семействе потоков `Object` отсутствуют символьные разновидности `Reader` и `Writer`.

Результатом ввода (десериализации) последовательности байтов, представляющих ранее сериализованный объект, из потока `ObjectInputStream` с помощью методов `readObject` служит граф объекта, равнозначный исходному.

В примере 5 представлены примеры сериализации и десериализации объекта.

Пример 5.

// Сериализация

```
LinkedList ls = new LinkedList();  
FileOutputStream fileOut = new FileOutputStream("list");  
ObjectOutputStream out = new ObjectOutputStream(fileOut);  
out.writeObject(ls);
```

// Десериализация

```
FileInputStream fileIn = new FileInputStream("list");  
ObjectInputStream in = new ObjectInputStream(fileIn);  
LinkedList ls = (LinkedList) in.readObject();
```

Подготовка классов к сериализации

Необходимым условием возможности осуществления сериализации объектов класса средствами потока `ObjectOutputStream` является реализация классом интерфейса-маркера `Serializable`. Поддержка классом интерфейса `Serializable` свидетельствует о том, что объекты класса готовы к сериализации.

По умолчанию процесс сериализации заключается в сериализации каждого поля объекта, которое не обозначено как `transient` или `static`. Данные простых типов и строки сохраняются с помощью того же механизма, какой поддерживается потоками `DataOutputStream`, а сериализация объектов выполняется средствами метода `writeObject`. Схема сериализации, предусмотренная по умолчанию, предполагает, что все поля-объекты, подлежащие сериализации, должны указывать на типы, в свою очередь поддерживающие возможность сериализации. Кроме того, требуется, чтобы класс, базовый по отношению к рассматриваемому, либо обладал конструктором без параметров (чтобы он мог быть вызван в процессе десериализации), либо сам в свою очередь обеспечивал реализацию интерфейса `Serializable`. В большинстве ситуаций для обеспечения возможности сериализации объектов класса вполне достаточно тех мер, которые предусмотрены схемой сериализации, предлагаемой по умолчанию, и проблема исчерпывается простым упоминанием интерфейса `Serializable` в объявлении класса.

Схема десериализации, предлагаемая по умолчанию, предусматривает считывание из потока байтовых данных, сохранённых в процессе сериализации. Статические поля класса остаются в неприкосновенности – в ходе загрузки класса будут выполнены все обычные процедуры инициализации, и статические поля получают требуемые исходные значения. Каждому полю `transient` в восстановленном объекте присваивается значение по умолчанию, соответствующее типу этого поля.

Иногда встречаются классы, которые сами по себе допускают возможность сериализации, но их отдельные элементы – нет. Например, контейнерный класс как таковой, позволяющий сериализацию, может содержать ссылки на объекты, которые сериализации не поддаются. Любая попытка осуществления сериализации объекта, не поддерживающего эту возможность, приводит к выбрасыванию исключения типа `NotSerializableException`.

Порядок сериализации и десериализации

Каждый класс несёт ответственность за обеспечение надлежащей сериализации состояния собственных объектов, т.е. содержимого их полей. Сериализация и десериализация объектов выполняется в нисходящем порядке по древовидной иерархии типов – от первого из классов, поддерживающих интерфейс `Serializable`, до классов более частных типов. Указанный порядок редко играет весомую роль при сериализации, но приобретает существенное значение в процессе десериализации. Рассмотрим следующую иерархию типов, которой подчиняется класс `HTTPInput`.

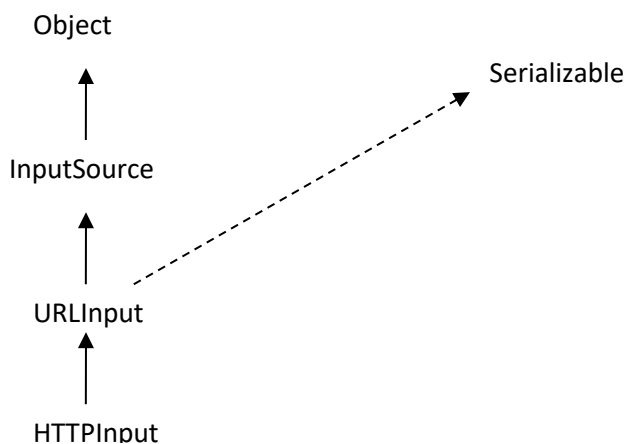


Рисунок 1

При десериализации объекта `HTTPInput` поток `ObjectInputStream` вначале выделяет память для нового объекта, а затем находит в иерархии типов ближайший из классов, поддерживающий интерфейс `Serializable`, - в нашем случае – `URLInput`. Далее поток вызывает конструктор без параметров для класса, базового по отношению к найденному (ближайшего из числа тех, которые не поддерживают интерфейс `Serializable`) – в данном случае, `InputSource`. Если должно быть сохранено иное состояние части объекта, относящейся к базовому классу (`InputSource`), ответственность за сериализацию и десериализацию этого состояния возлагается на `URLInput`.

Если же базовый класс, не обеспечивающий сериализацию, обладает собственным значимым состоянием, то почти наверняка придётся отступить от схемы сериализации, предлагаемой по умолчанию, и осуществить дополнительную настройку ближайшего из классов, поддерживающих механизм сериализации. Если такой класс непосредственно наследует класс Object, процедура настройки окажется простой, поскольку Object не обладает каким-либо особым состоянием, подлежащим сохранению и последующему восстановлению.

Как только ближайший из классов, поддерживающих механизм сериализации, завершает восстановление данных того подмножества полей, которые относятся к базовому классу, он приступает к воссозданию своего собственного состояния, считывая данные из потока. Затем поток ObjectInputStream осуществляет просмотр иерархии объектов в нисходящем порядке и выполняет десериализацию каждого объекта посредством вызовов readObject. По достижении нижнего уровня дерева иерархии процесс десериализации исходного объекта полностью завершается.

В ходе десериализации объекта могут быть найдены ссылки на другие ранее сериализованные объекты. Эти объекты подвергаются десериализации в том порядке, в котором они обнаруживаются. Если бы объект URLInput, например, обладал ссылкой на объект HashMap, адресуемая хеш-таблица вместе со всем её содержимым была бы подвергнута десериализации раньше, чем оставшаяся часть объекта URLInput как такового.

Прежде чем будут выполнены любые из рассмотренных операций десериализации, соответствующие классы должны быть загружены. В процессе загрузки выполняется поиск класса с требуемым именем и проверка версии этого класса. Если подлинность и соответствие версии класса установлены, тот должен быть загружен. Если же класс не найден либо не может быть загружен по иным причинам, метод readObject выбрасывает исключение типа ClassNotFoundException.

Настройка механизма сериализации

Схема сериализации/десериализации, предлагаемая по умолчанию, вполне работоспособна во многих ситуациях, но, увы, не во всех. Для некоторых классов обычные средства десериализации могут оказаться неадекватными или неэффективными. *Наглядным примером способен служить класс HashMap. Следуя обычной схеме сериализации, можно было бы сохранить в потоке все структуры данных хеш-таблицы, включая и значения хеш-кодов её элементов. Такой подход окажется и неверным, и неэффективным. Неправильность подхода обусловлена тем, что значения хеш-кодов восстановленных элементов могут отличаться от исходных, - в частности, если используется реализация hashCode, предусмотренная по умолчанию. Неэффективность обычной схемы сериализации в применении к хеш-таблицам объясняется тем, что многие из элементов хеш-таблиц, как правило, бывают пустыми и выполнять их сериализацию совершенно нецелесообразно. Более уместно сохранять в потоке только те элементы,*

ссылки на которые существуют, нежели осуществлять сериализацию всех данных таблицы.

Руководствуясь подобными соображениями, создатели Java предусмотрели возможность включать в состав класса специальные private – версии методов writeObject и readObject. Методы активизируются потоками ObjectOutputStream и ObjectInputStream соответственно в тот момент, когда необходимо осуществить сериализацию и десериализацию объекта. Подобные переопределённые варианты методов writeObject и readObject вызываются только в контексте объектов соответствующих классов, и методы несут ответственность исключительно за состояние объекта класса как такового, в том числе и в части, унаследованной от базового класса, не поддерживающего механизм сериализации. Если в составе класса реализованы собственные варианты методов writeObject и readObject, они не должны обращаться к одноимённым методам базового класса.

```
private void writeObject(ObjectOutputStream out) throws IOException
private void readObject(ObjectInputStream in) throws IOException,
ClassNotFoundException
```

В объявлении метода writeObject обозначена вероятность выбрасывания им исключения типа IOException, так как оно может быть сгенерировано любым из «низкоуровневых» методов write, которые будут здесь вызываться, и если подобное произойдёт, процесс сериализации должен быть завершён аварийным образом. В предложении throws объявления метода readObject также следует упомянуть исключение IOException – оно может быть выброшено вызываемыми методами read, а в такой ситуации процесс десериализации следует остановить. Помимо того, метод readObject способен генерировать и исключение типа ClassNotFoundException, так как в общем случае при десериализации полей текущего объекта может потребоваться загрузка и других классов.

Существует ограничение, которое следует принять во внимание, приступая к настройке механизма сериализации: нельзя присваивать значение полю final в теле метода readObject, поскольку поля final могут быть проинициализированы только в блоках инициализации или конструкторах. Механизм сериализации, предусмотренный по умолчанию, однако, способен «обходить» это ограничение – он вполне нормально работает и с теми классами, в составе которых имеются поля final.

Контроль версий объектов

Реализация классов со временем изменяется. Если подобное происходит в промежутке времени между сериализацией и десериализацией объекта этого класса, поток ObjectInputStream способен обнаружить факт внесения изменений. При сохранении объекта вместе с ним записывается уникальный идентификатор номера версии – 64-битовое значение типа long. По умолчанию идентификатор создаётся в виде хеш-кода, построенного на основе информации об именах класса, его членов и базовых интерфейсов. Изменение таких данных служит сигналом о возможной несовместимости версий класса.

При вводе данных об объекте из потока `ObjectInputStream` считывается также и идентификатор номера версии. Затем предпринимается попытка загрузки соответствующего класса. Если требуемый класс не найден, либо идентификатор загруженного класса не совпадает с тем, который считан из потока, метод `readObject` выбрасывает исключение типа `InvalidClassException`. Если успешно загружены все нужные классы и выявлено совпадение всех идентификаторов, объект, как принято считать, может быть подвергнут десериализации.

Таким образом, получается, что любое, даже самое незначительное, изменение в структуре класса способно привести к несовместимости версий. Но во многих случаях модификация классов носит отнюдь не радикальный характер. Существует возможность сохранить совместимость класса, снабжённого дополнительными несущественными нововведениями, с данными ранее сериализованных объектов. Для этого следует принудительно объявить в составе класса специальное поле, содержащее значение идентификатора номера версии, например:

```
private static final long serialVersionUID = ...;
```

Конкретное значение идентификатора рассчитывается средствами используемой среды разработки Java. Ничто не мешает использовать в качестве идентификатора и любое произвольное значение, если оно фиксируется в самой первой версии класса, но это не самое лучшее решение, так как вы почти наверняка не сможете столь же тщательно сгенерировать идентификатор, как это делают стандартные инструменты и обеспечить его уникальность, чтобы предотвратить возможные конфликты с хеш-кодами других классов.

Теперь, когда поток `ObjectInputStream` находит описание требуемого класса и сопоставляет его идентификатор с тем, который сохранён вместе с сериализованными данными, проблема несовместимости не возникнет, даже если реализация класса на самом деле подверглась изменению.

Многопоточное программирование

Последовательность операций, выполняемых программой по одной в каждый момент времени, называют **поток**ом (или **нитью** - **thread**). До сих пор во всех рассматриваемых примерах подразумевалось, что в один момент времени исполняется лишь одно выражение, или действие. Однако, начиная с самых первых версий, виртуальные машины Java поддерживают многопоточность, т.е. поддержку нескольких потоков исполнения одновременно.

Проблемы однопоточного подхода

- Монопольный захват задачей процессорного времени.
- Смешение логически несвязанных фрагментов кода.
- Попытка их разделения приводит к возникновению в программе новых систем.

Реализацию многопоточной архитектуры проще всего представить себе для системы, в которой есть несколько центральных вычислительных

процессоров. В этом случае для каждого из них можно выделить задачу, которую он будет выполнять. В результате несколько задач будут обслуживаться одновременно.

Однако возникает вопрос – каким же тогда образом обеспечивается многопоточность в системах с одним центральным процессором, который в принципе выполняет лишь одно вычисление в один момент времени? В таких системах применяется процедура **квантования времени (time-slicing)**. Время разделяется на небольшие интервалы (кванты времени). Во время одного кванта обрабатывается один поток команд. Перед началом каждого интервала принимается решение, какой именно поток выполнения будет обрабатываться на протяжении этого кванта времени. За счет частого переключения между задачами эмулируется многопоточная архитектура, т.е. создается иллюзия одновременности выполнения.

На самом деле, как правило, и для многопроцессорных систем применяется процедура квантования времени. Дело в том, что даже в мощных серверах приложений процессоров не так много (*редко бывает больше десяти*), а потоков исполнения запускается, как правило, гораздо больше. Т.о. в многопроцессорной системе поток не занимает монополю один процессор.

Особенности многопоточности

- Потоки выполняются условно независимо.
- Потоки могут взаимодействовать друг с другом.
- Простота выделения подзадач.
- Более гибкое управление выполнением задач. *Предположим, пользователь системы, не поддерживающей многопоточность, решил скачать большой файл из сети или произвести сложное вычисление, что занимает, скажем, 2 часа. Запустив задачу на выполнение, он может внезапно обнаружить, что ему нужен не этот, а какой-нибудь другой файл (или вычисление с другими начальными параметрами). Однако если приложение занимается только работой с сетью (вычислениями) и не реагирует на действия пользователя (не обрабатываются данные с устройств ввода, таких как клавиатура или мышь), то он не сможет быстро исправить свою ошибку. Получается, что процессор выполняет большее количество вычислений, но при этом приносит гораздо меньше пользы своему пользователю.*
- Более медленное выполнение? Действительно, для переключения между задачами на каждом интервале требуется дополнительное время, а ведь переключения происходят довольно часто. Если бы процессор, не отвлекаясь, выполнял задачи последовательно, одну за другой, он закончил бы их заметно быстрее. Стало быть, преимущества заключаются не в этом. Первый тип приложений, который выигрывает от поддержки многопоточности, предназначен для задач, где действительно требуется выполнять несколько действий одновременно. Например - активные игры, или подобные приложения. Необходимо одновременно опрашивать клавиатуру и

другие устройства ввода, чтобы реагировать на действия пользователя. В то же время одновременно необходимо рассчитывать и перерисовывать изменяющееся состояние игрового поля. Понятно, что в случае отсутствия поддержки многопоточности для реализации подобных приложений потребовалось бы реализовывать квантования времени вручную. Условно говоря - одну секунду проверять состояние клавиатуры, а следующую - пересчитывать и перерисовывать игровое поле. Если сравнить две реализации *time-slicing*, одну - на низком уровне, выполненную средствами, как правило, операционной системы, другую - выполняемую вручную, на языке высокого уровня, мало подходящего для таких задач, то становится понятным преимущество многопоточности. Она обеспечивает наиболее эффективную реализацию процедуры квантования времени, существенно облегчая и укорачивая процесс разработки приложения. Код переключения между задачами на Java выглядел бы гораздо более громоздко, чем независимое описание действий для каждого потока.

- Выигрыш в скорости выполнения при разделении задач по используемым ресурсам. Это преимущество проистекает из того, что компьютер состоит не только из одного или нескольких процессоров. Вычислительное устройство - лишь одно из ресурсов, необходимых для выполнения задач. Всегда есть оперативная память, дисковая подсистема, сетевые подключения, периферия (принтер и т.п.) и другие. Предположим, пользователю требуется распечатать большой документ и скачать большой файл из сети. Очевидно, что обе задачи требуют совсем небольшого участия процессора, а основные необходимые ресурсы, которые будут использоваться, у них разные - сетевое подключение и принтер. Значит, если выполнять задачи одновременно, то замедление от организации квантования времени будет незначительным, процессор легко справится с обслуживанием обеих задач. В то же время, если каждая задача по отдельности занимала, скажем, 2 часа, то вполне вероятно, что и при одновременном исполнении потребуется не более тех же 2 часов, а сделано при этом будет гораздо больше.
- Недетерминизм при выполнении.

Использование класса *Thread*

Поток выполнения в Java представляется экземпляром класса *Thread*. Для того, чтобы написать свой поток исполнения необходимо наследоваться от этого класса и переопределить метод *run()*. Стандартная реализация метода *run* не предполагает выполнения каких бы то ни было действий.

Пример 6.

//Объявление:

```
public class <Имя класса> extends Thread {  
    public void run() {  
        // Действия, выполняемые потоком  
    }  
}
```

```

}
//Запуск:
<Имя класса> t = new <Имя класса>();
t.start();

```

Метод run() содержит действия, которые должны исполняться в новом потоке исполнения. Чтобы запустить его, необходимо создать экземпляр класса-наследника, и вызвать унаследованный метод start(), который сообщает виртуальной машине, что необходимо запустить новый поток исполнения и начать в нем исполнять метод run().

Вызов start для каждого потока может быть осуществлен только один раз – повторное обращение приводит к выбрасыванию исключения типа *IllegalThreadStateException*.

Потоку разрешено присвоить имя – либо с помощью аргумента String, переданного конструктору, либо посредством вызова метода setName. Текущее значение имени потока можно получить с помощью метода getName. Имена потоков служат только для удобства программиста (исполняющей системой они не используются), но поток должен обладать именем, и если оно не задано, исполняющая система генерирует имена в соответствии с неким простым правилом, например, thread_1, thread_2 и т.д.

Ссылку на объект Thread текущего выполняемого потока можно получить с помощью статического метода Thread.currentThread. Во время работы программы текущий поток существует всегда, даже если вы не создавали потоки явно – метод main активизируется с помощью потока, создаваемого исполняющей системой.

Использование интерфейса Runnable

Описанный подход обладает одним недостатком. Поскольку в Java отсутствует множественное наследование, требование наследоваться от Thread может привести к конфликту. *Если еще раз посмотреть на приведенный выше пример, то станет понятно, что наследование производилось только с целью переопределения метода run().* Поэтому предлагается более простой способ создать свой поток исполнения. Достаточно реализовать интерфейс Runnable, в котором объявлен только один метод - уже знакомый void run(). Запишем пример, приведенный выше, с помощью этого интерфейса:

Пример 7.

```

//Объявление:
public class <Имя класса> implements Runnable {
    public void run() {
        // Действия, выполняемые потоком
    }
}
//Запуск:
Runnable r = new <Имя класса>();
Thread t = new Thread(r);
t.start();

```

Подчеркнем, что `Runnable` не является полной заменой классу `Thread`, поскольку создание и запуск самого потока исполнения возможно только через метод `Thread.start()`.

Управление потоками

- `void start()`
Запускает выполнение потока
- `void stop()`
Прекращает выполнение потока
- `void suspend()`
Приостанавливает выполнение потока
- `void resume()`
Возобновляет выполнение потока
- `static void sleep(long millis)`
Останавливает выполнение потока как минимум на **millis** миллисекунд
- `static void yield()`
Приостанавливает выполнение потока, предоставляет возможность выполнять другие потоки
- `void join()`
Ожидает завершения указанного потока

Группы потоков (ThreadGroup)

Каждый поток находится в группе. Группа указывается при создании потока. Если группа не была указана, то поток помещается в ту же группу, где находится поток, породивший его. Группы потоков образуют дерево, корнем служит начальная группа. Поток не имеет доступа к информации о родительской группе. Изменение параметров и состояния группы влияет на все входящие в нее потоки.

Пример 8.

//Создание группы:

//Без явного указания родительской группы

```
ThreadGroup group1 = new ThreadGroup("Group1");
```

//С явным указанием родительской группы

```
ThreadGroup group2 = new ThreadGroup(group1, "Group2");
```

//Создание потока:

//Без явного указания группы

```
MyThread t = new MyThread("Thread1");
```

//С явным указанием группы

```
MyThread t = new MyThread(group2, "Thread2");
```

Операции в группе потоков

- `int activeCount()`
Возвращает приблизительное количество действующих (активных) потоков группы, включая и те потоки, которые принадлежат вложенным группам. Количество нельзя считать точным, т.к. в момент выполнения метода оно может измениться – одни потоки «умирают», другие – создаются.

- *int enumerate(Thread[] list)*
Заполняет массив list ссылками на объекты активных потоков группы и возвращает количество сохраненных ссылок. Также учитываются потоки, относящиеся ко всем вложенным группам.
- *int activeGroupCount()*
Возвращает количество групп, включая вложенные.
- *int enumerate(ThreadGroup[] list)*
Заполняет массив list ссылками на объекты вложенных групп потоков.
- *void interrupt()*
Прерывает выполнение всех потоков в группе

Приоритеты потоков

Приоритет – количественный показатель важности потока. Недетерминированно воздействует на системную политику упорядочивания потоков. Базовый алгоритм не должен зависеть от схемы расстановки приоритетов потоков. В Java потокам можно назначать приоритеты. Для этого в классе Thread существуют методы `getPriority()` и `setPriority()`, а также объявлены три

Константы

static int MAX_PRIORITY

static int MIN_PRIORITY

static int NORM_PRIORITY

Их значения описывают максимальное, минимальное и нормальное (по умолчанию) значения приоритета.

Методы потока

final int getPriority() – возвращает ранее заданное значение приоритета выполнения для текущего потока.

final void setPriority(int newPriority) – устанавливает приоритет выполнения для текущего потока.

Методы группы потоков

final int getMaxPriority() – возвращает ранее заданное значение верхней границы приоритетов выполнения для текущей группы потоков.

final void setMaxPriority(int maxPri) – устанавливает верхнюю границу приоритетов выполнения для текущей группы потоков.

Демон-потоки (Daemons)

Демон-потоки позволяют описывать фоновые процессы, которые нужны только для обслуживания основных потоков выполнения и не могут существовать без них. Уничтожаются виртуальной машиной, если в группе не осталось не-демон потоков.

- *void setDaemon(boolean on)* - устанавливает вид потока. Вызывается до запуска потока.
- *boolean isDaemon()* - возвращает вид потока:
true – демон, false – обычный.

Демон-группы потоков

Демон-группа автоматически уничтожается при останове последнего ее потока или уничтожении последней подгруппы потоков.

- `void setDaemon(boolean on)` - устанавливает вид группы.
- `boolean isDaemon()` - возвращает вид группы:
true – демон, false – обычная.

Совместное использование ресурсов

При многопоточной архитектуре приложения возможны ситуации, когда несколько потоков будут одновременно работать с одними и теми же данными, используя их значения и присваивая новые. В таком случае результат работы программы становится невозможным предопределить, глядя только на исходный код. *Финальные значения переменных будут зависеть от случайных факторов, исходя из того, какой поток какое действие успел сделать первым или последним.* Отсюда следует возможность некорректной работы алгоритма, возникновения исключительных ситуаций.

Решением проблемы является применение **механизма блокировки объекта**:

- Только один поток в один момент времени может установить блокировку на некоторый объект.
- Попытка блокировки уже заблокированного объекта приводит к останову потока до момента разблокирования этого объекта.
- Наличие блокировки не запрещает всех остальных действий с объектом.

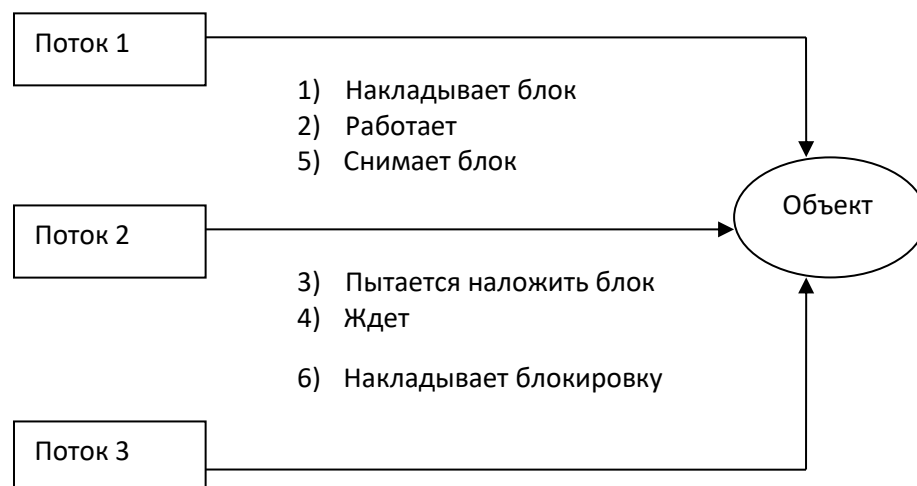


Рисунок 2

В Java-программе для того, чтобы воспользоваться механизмом блокировок, существует ключевое слово `synchronized`. Оно может быть применено в двух вариантах – для объявления `synchronized`-блока и как модификатор метода. В обоих случаях действие его примерно одинаковое.

Пример 9.

//Синхронизированный блок:

//Блокируется указанный объект

`synchronized (ref < Ссылка на объект >)`

`{`

<Тело блока синхронизации>

`}`

```
//Синхронизированный метод:
//Блокируется объект-владелец метода
public synchronized void <Имя метода>() {
    <Тело метода>
}
```

Прежде чем начать выполнять действия, описанные в этом блоке, поток обязан установить блокировку на объект, на который ссылается переменная `ref` (поэтому она не может быть `null`). Если другой поток уже установил блокировку на этот объект, то выполнение первого потока приостанавливается до тех пор, пока не удастся выполнить блокировку. После этого блок выполняется. В случае успешного либо не успешного завершения исполнения, производится снятие блокировки, чтобы освободить объект для других потоков.

`Synchronized`-методы работают аналогичным образом. Прежде чем начать выполнять их, поток пытается заблокировать объект, у которого вызывается метод. После выполнения блокировка снимается.

Характерные ошибки:

- Отсутствие синхронизации.
- Необоснованная длительная блокировка объектов.
- Взаимная блокировка (*deadlock*). Если один поток заблокировал один ресурс, и пытается заблокировать второй, а другой поток заблокировал второй и пытается заблокировать первый, то такие потоки уже никогда не выйдут из состояния ожидания. В Java нет никаких средств распознавания или предотвращения ситуаций *deadlock*. Также нет способа перед вызовом синхронизированного метода узнать, заблокирован ли уже объект другим потоком. Программист сам должен строить работу программы таким образом, чтобы неразрешимые блокировки не возникали.
- Возникновение монопольных потоков.
- Нерациональное назначение приоритетов.

volatile

Рассмотрим пример 10.

Пример 10.

```
curentValue = 5;
for(;;){
    display.showValue(currentValue);
    Thread.sleep(1000);
}
```

Если метод `showValue` сам по себе не обладает возможностью изменения значения `curentValue`, компилятор может предположить, что внутри цикла `for` это значение можно трактовать как неизменное, и использовать одну и ту же константу 5 на каждой итерации цикла при вызове `showValue`. Но если содержимое поля `curentValue` в ходе выполнения цикла подвержено обновлению посредством других потоков, предположение компилятора окажется неверным.

Преодолеть проблему может соответствующим способом оформленный синхронизированный код. Однако существует альтернативный вариант решения проблемы – объявление поля можно снабдить модификатором `volatile`. Этот признак свидетельствует о том, что значение поля может быть изменено в любой непредсказуемый момент. Если объявить `currentValue` как `volatile`, компилятор будет вынужден, выполняя каждую итерацию цикла, заново перечитывать значение переменной. Метод, выполняющий считывание содержимого переменной, обозначенной признаком `volatile`, гарантировано возвращает наиболее свежую версию сохраненного в ней значения.

Специальные методы класса Object

Наконец, переходим к рассмотрению трех методов класса `Object`, которое завершает описание механизмов поддержки многопоточности в Java.

Каждый объект в Java имеет так называемый `wait-set`, набор потоков исполнения. Любой поток может вызвать метод **`wait()`** любого объекта и таким образом попасть в его `wait-set`. При этом выполнение такого потока приостанавливается до тех пор, пока другой поток не вызовет у точно этого же объекта метод **`notifyAll()`**, который пробуждает все потоки из `wait-set`. Метод **`notify()`** пробуждает один, случайно выбранный поток из этого набора. Однако применение этих методов связано с некоторыми особенностями:

Особенности использования методов класса Object:

- Метод может быть вызван потоком у объекта только после установления блокировки на этот объект. То есть, либо внутри `synchronized`-блока с ссылкой на этот объект в качестве аргумента, либо обращение к методам должны быть в синхронизированных методах класса самого объекта.
- Потоки, прежде чем приостановить выполнение после вызова метода `wait()`, снимают все блокировки.
- После вызова освобождающего метода потоки пытаются восстановить ранее снятые блокировки.

Нерекомендуемые действия над потоками

- `Thread.suspend()`, `Thread.resume()` - увеличивает количество взаимных блокировок.
- `Thread.stop()` - использование приводит к возникновению поврежденных объектов.

Прерывание потока

- `public void interrupt()` - изменяет статус потока на прерванный.
- `public static boolean interrupted()` - возвращает и очищает статус потока (прерван или нет).
- `public boolean isInterrupted()` - возвращает статус потока (прерван или нет).

В том случае, если в текущий момент поток выполняет методы `wait()`, `sleep()`, `join()`, а его прерывают вызовом метода `interrupt()`, метод прерывает

свое выполнение с выбросом исключения `InterruptedException`. Поток не сообщается, что его прервали!

Пример кода объекта-посредника:

Пример 11.

```
public class Keeper {  
    private Object data;  
    private boolean newed = false;  
  
    synchronized public void putData(Object obj)  
throws InterruptedException {  
        while(newed)  
            wait();  
        data = obj;  
        newed = true;  
        notifyAll();  
    }  
    synchronized public Object getData() {  
        try {  
            while(!newed)  
                wait();  
            newed = false;  
            notifyAll();  
            return data;  
        } catch(InterruptedException e) {  
            Thread.currentThread().interrupt();  
        }  
    }  
}
```

Пакет `java.util.concurrent`

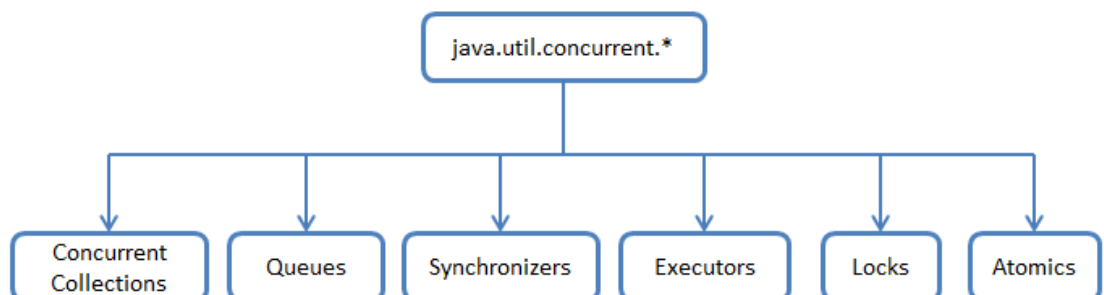


Рисунок 3

- **Concurrent Collections** — набор коллекций, более эффективно работающие в многопоточной среде нежели стандартные универсальные коллекции из `java.util` пакета, где вместо базовой обертки `Collections.synchronizedList()` с блокированием доступа ко всей

коллекции используются блокировки по сегментам данных или же оптимизируется работа для параллельного чтения данных

- **Queues** — неблокирующие (направлены на скорость и работу без блокирования потоков) и блокирующие (используются, когда нужно «притормозить» потоки чтения или записи, если не выполнены какие-либо условия, например, очередь пуста или переполнена, или же нет свободного читателя) очереди с поддержкой многопоточности
- **Synchronizers** — вспомогательные утилиты для синхронизации потоков, которые представляют собой мощное средство в параллельных вычислениях.
- **Executors** — содержит в себе отличные фреймворки для создания пулов потоков, планирования работы асинхронных задач с получением результатов
- **Locks** — представляет собой альтернативные и более гибкие механизмы синхронизации потоков по сравнению с базовыми **synchronized()**, **wait()**, **notify()**, **notifyAll()**
- **Atomics** — классы с поддержкой атомарных операций над примитивами и ссылками

Пакет **java.util.concurrent.locks**

В Java SE 5.0 появился класс **java.util.concurrent.locks.ReentrantLock**, конструирующий реентерабельную блокировку, которая может быть использована для защиты критичной части кода.

Пример 12.

```
private Lock myLock = new ReentrantLock(); // ReentrantLock реализует
интерфейс Lock
myLock.lock();
try {
    // критичный раздел
}
finally {
    myLock.unlock(); // гарантирует разблокировку даже в случае
возникновения исключения
}
```

Такая конструкция гарантирует, что только один поток в единицу времени сможет войти в критичный раздел кода. Как только один поток блокирует объект блокировки, никакой другой поток не сможет выполнить оператор **lock**. Когда другие потоки попытаются это сделать, они будут деактивированы до тех пор, пока первый поток не снимет блокировку с этого объекта. Важно, чтобы операция **unlock** помещалась в конструкцию **finally**. Если код критичного раздела сгенерирует исключение, блокировка должна быть снята. В противном случае другие потоки будут заблокированы навсегда.

Блокировка называется **реентерабельной**, потому что поток может повторно захватывать блокировку, которой он уже владеет. Объект блокировки имеет счетчик блокировок, отслеживающий вложенные вызовы

метода `lock()`. Поток должен вызвать `unlock()` для каждого вызова `lock()`, чтобы, в конце концов, снять блокировку.

Поток блокируется навечно, когда он вызывает метод `lock()` для захвата блокировки, которой владеет другой поток. Поэтому следует быть осторожным, пытаясь захватить блокировку. Метод `tryLock()` пытается захватить блокировку и возвращает `true`, если ему это удастся. Иначе возвращается `false` и тогда можно переключить поток на что-нибудь другое.

Пример 13.

```
if (myLock.tryLock())
    // теперь поток владеет блокировкой
try {
    // ...
}
finally { myLock.unlock(); }
else
    // делать что-то другое
```

Пример 14.

```
public boolean impendingBow(Friend bower) {
    Boolean myLock = false;
    Boolean yourLock = false;
    try {
        myLock = lock.tryLock();
        yourLock = bower.lock.tryLock();
    } finally {
        if (!(myLock && yourLock)) {
            if (myLock) {
                lock.unlock();
            }
            if (yourLock) {
                bower.lock.unlock();
            }
        }
    }
    return myLock && yourLock;
}
```

Пакет **java.util.concurrent.locks** определяет два класса блокировок – **ReentrantLock** и **ReentrantReadWriteLock**. Последний удобен, когда есть много потоков, читающих структуру данных, и поменьше потоков – модифицирующих её. В такой ситуации имеет смысл разрешить разделенный доступ читателям. Конечно, поток-писатель должен по-прежнему иметь исключительный доступ. Ниже описаны шаги, необходимые для использования блокировок чтения/записи.

Пример 15

1. Сконструировать объект **ReentrantReadWriteLock**:

```
private ReentrantReadWriteLock rwl = new
    ReentrantReadWriteLock();
```

2. Извлечь блокировки чтения и записи:

```
private Lock readLock = rwl.readLock(); // получает блокировку
чтения, которая может быть захвачена многими читателями, но
исключая всех писателей.
private Lock writeLock = rwl.writeLock(); // получает блокировку
записи, исключающую доступ всех читателей и писателей.
```
3. Использовать блокировку чтения для всех читателей:

```
public double getTotalBalance(){
    readLock.lock();
    try {...}
    finally {readLock.unlock();}
}
```
4. Использовать блокировку записи для всех, кто изменяет данные:

```
public void transfer(...){
    writeLock.lock();
    try {...}
    finally {writeLock.unlock();}
}
```

Пример 16.

```
public class Dictionary {
    private final ReentrantReadWriteLock readWriteLock = new
ReentrantReadWriteLock();
    private final Lock read = readWriteLock.readLock();
    private final Lock write = readWriteLock.writeLock();
    private HashMap<String, String> dictionary = new HashMap<String,
String>();
    public void set(String key, String value) {
        write.lock();
        try { dictionary.put(key, value); } finally { write.unlock(); }
    }
    public String get(String key) {
        read.lock();
        try{ return dictionary.get(key); } finally { read.unlock(); }
    }
    public String[] getKeys() {
        read.lock();
        try {
            String keys[] = new String[dictionary.size()];
            return dictionary.keySet().toArray(keys);
        } finally { read.unlock(); }
    }
}
```

Интерфейсы Callable и Future

Callable подобен Runnable, но с возвратом значения. Интерфейс Callable является параметризованным типом с единственным общедоступным методом call().

Пример 17.

```
public interface Callable <V> {  
    V call() throws Exception;  
}  
public interface Future <V> {  
    V get();  
    void cancel(boolean mayInterrupt);  
    boolean isCancelled();  
    boolean isDone();  
}
```

Параметр представляет собой тип возвращаемого значения.

Future хранит результат вычисления. Можно запустить вычисление, предоставив кому-либо объект Future и забыть о нём. Владелец объекта Future может получить результат, когда он будет готов. Интерфейс Future имеет следующие методы:

Вызов метода get() устанавливает блокировку до тех пор, пока не завершится вычисление или истечения указанного времени.

Можно прервать вычисление, вызвав метод cancel(). Если вычисление не стартовало, оно отменяется и уже не будет запущено. Если вычисление уже идёт, оно прерывается в случае равенства true параметра mayInterrupt.

isCancelled() возвращает true, если задача была отменена до её завершения.

isDone() возвращает false, если вычисление продолжается, и true – если оно завершено или отменено.

Интерфейсы Executor, ExecutorService, ScheduledExecutorService. Пул потоков. Executors.

Пакет java.util.concurrent содержит три интерфейса исполнителей:

Executor – простой интерфейс, поддерживающий запуск новой задачи.

ExecutorService, расширяющий Executor, добавляет возможности по управлению жизненным циклом отдельных задач и самого исполнителя.

ScheduledExecutorService, расширяющий ExecutorService, поддерживает периодическое выполнение задач или их запуск в будущем. Это обобщение java.util.Timer, позволяющее создавать пул потоков.

Обычно переменные-ссылки на объекты исполнителей объявляются одного из этих трех интерфейсных типов без указания конкретного имени класса исполнителя.

Интерфейс **Executor** предоставляет единственный метод, execute(), разработанный для замены общего подхода создания потоков. Если r является Runnable объектом и e является Executor объектом, вы можете заменить

```
(new Thread(r)).start();
```

на

e.execute(r);

Однако определение execute() менее специфично. В Java 1.4 создается новый поток и запускается. В зависимости от реализации Executor, execute() может сделать то же самое, но более вероятно использует существующий рабочий поток для запуска r или размещения r в очереди ожидания доступного рабочего потока.

Реализации исполнителей в java.util.concurrent спроектированы так, чтобы в полной мере использовать более продвинутые ExecutorService и ScheduledExecutorService интерфейсы, хотя они также работают с базовым интерфейсом Executor.

Создание нового потока – довольно дорогостоящая (в плане времени и ресурсов) операция, поскольку включает взаимодействие с операционной системой. Если ваша программа создает большое количество кратковременных потоков, то имеет смысл использовать *пул потоков*. Пул потоков содержит множество простаивающих потоков, готовых к запуску. Вы помещаете Runnable в пул, и один из потоков вызывает его метод run(). Когда метод run() завершается, поток не уничтожается и остается в пуле в готовности обслужить новый запрос.

Другая причина использования пула потоков – необходимость ограничить количество конкурирующих потоков. Создание огромного числа потоков может пагубно отобразиться на производительности. Если у вас есть алгоритм, создающий большое количество потоков, то вам следует установить «фиксированный» пул потоков, который ограничит общее количество конкурирующих потоков. Класс Executors имеет следующие статические методы для конструирования пулов потоков:

newCachedThreadPool() – создаёт пул потоков, выполняющий каждую задачу немедленно, используя существующий простаивающий поток, если он доступен, либо создавая новый в противном случае.

newFixedThreadPool() – создаёт пул потоков фиксированного размера. Если количество задач превышает количество простаивающих потоков, то лишние ставятся в очередь, чтобы быть запущенными, когда завершатся текущие выполняющиеся задачи.

newSingleThreadExecutor() – создает пул размером в 1 поток. Единственный поток выполняет все отправленные ему задачи – одну за другой. Перечисленные три метода возвращают объект класса ThreadPoolExecutor, реализующий интерфейс ExecutorService.

Можно послать Runnable или Callable объекту ExecutorService одним из следующих методов:

Future <?> submit (Runnable task)

Future <T> submit (Runnable task, T result)

Future <T> submit (Callable <T> task)

Пул запустит установленную таким образом задачу при первой возможности. Когда вызывается submit(), вы получаете объект Future, который можно использовать для опроса состояния задачи. Первый метод submit() вернет Future<?>, этот объект можно использовать для вызова

isDone(), cancel() или isCancelled(). Однако метод get() по завершении вернёт просто null. Вторая версия submit() также принимает Runnable, а метод get() объекта Future вернет по завершении заданный объект result. Третья версия принимает Callable, а возвращенный Future получает результат вычисления по его готовности.

Завершив работу с пулом потоков, вызывайте shutdown(). Этот метод инициирует последовательность выключения пула. После этого объект пула не принимает новых задач. Когда все задачи завершены, потоки в пуле уничтожаются. В качестве альтернативы можно вызвать shutdownNow(). В этом случае пул отменяет все задачи, которые ещё не стартовали, и пытается прервать работающие потоки. Итак, чтобы использовать пул потоков, надо:

1. Вызвать статический метод newCachedThreadPool() или newFixedThreadPool() класса Executors.
2. Вызвать submit() для установки объектов Runnable или Callable.
3. Если хотите иметь возможность прерывать задачу, или если устанавливаете объекты Callable, обратитесь к возвращенным объектам Future.
4. Вызвать shutdown(), когда больше не собираетесь запускать новых задач.

newScheduledThreadPool(int threads) – возвращает пул потоков, использующих заданное их количество для планирования запуска задач.

newSingleThreadScheduledExecutor() – возвращает исполнитель, планирующий запуск задач последовательно в одном потоке.

Эти методы возвращают объекты, реализующие интерфейс ScheduledExecutorService.

Пример 18.

```
class NetworkService implements Runnable {
    private final ServerSocket serverSocket;
    private final ExecutorService pool;
    public NetworkService(int port, int poolSize) throws IOException
    {
        serverSocket = new ServerSocket(port);
        pool = Executors.newFixedThreadPool(poolSize);
    }
    public void run() {
        try {
            for (;;) {
                pool.execute(new Handler(serverSocket.accept()));
            }
        } catch (IOException ex) { pool.shutdown(); }
    }
}
```

```

}
class Handler implements Runnable {
    private final Socket socket;
    Handler(Socket socket) { this.socket = socket; }
    public void run() { ... }
}

```

Основы создания сетевых приложений

В течение последних нескольких десятилетий размеры и количество сетей значительно выросли. В 80-х годах имелось множество типов сетей. И практически каждая из них была построена на своем типе оборудования и программного обеспечения, зачастую не совместимых между собой. Это приводило к значительным трудностям при попытке соединить несколько различных типов сетей (например, различный тип адресации делал эти попытки практически безнадежными). Эта проблема была рассмотрена Всемирной Организацией по Стандартам (International Organization for Standardization, ISO), и в результате в 1984 г. была разработана модель OSI - модель взаимодействия открытых систем (Open Systems Interconnected). Эта модель состоит из семи уровней:

- **Физический уровень.** Этот уровень описывает среду передачи данных. Стандартизируются физические устройства, отвечающие за передачу электрических сигналов (разъемы, кабеля и т.д.) и правила формирования этих сигналов. Физический уровень также отвечает за преобразование сигналов между различными средами передачи данных. Например, при необходимости соединить сегмент сети, построенной на оптоволокне и витой паре применяют т.н. конверторы (в данном случае они преобразуют световой импульс в электрический).
- **Уровень соединения (канальный уровень).** На физическом уровне пересылаются просто набор сигналов - битов. При этом не проверяется, что несколько компьютеров могут в одну среду передачи данных одновременно передавать информацию в виде битов. Поэтому одной из задач канального уровня является проверка доступности среды передачи. Также этот уровень отвечает за доставку между источником и адресатом в пределах сети с одной топологией.
- **Сетевой уровень.** Одним из ограничений канального уровня является использование “плоской” модели адресации. Протокол, который поддерживается сетевым уровнем, использует иерархическую структуру для уникальной идентификации компьютеров. Для примера представим себе телефонную сеть. Она также имеет иерархическую адресацию. Например, в номере +7-095-101-12-34 первая цифра обозначает код страны, далее идет код области/города(095), а затем указывается сам телефон (101-12-34). Последний номер также является составным. 101 - это код станции, куда подключен телефон, а 12-34 определяет местоположение телефона. Благодаря такой иерархической структуре мы можем определить расположение

требуемого абонента с наименьшими затратами. Иерархическая адресация для сети также должна позволять передавать данные между разрозненными и удаленными сетями.

- **Транспортный уровень.** Рассмотрим TCP/IP протокол транспортного уровня модели OSI. TCP/IP имеет два протокола - TCP и UDP. **Transmission Control Protocol TCP** – основанный на соединениях протокол, обеспечивающий надежную передачу данных между двумя компьютерами с сохранением порядка данных. Используется в HTTP, FTP. **User Datagram Protocol UDP** – не основанный на соединениях протокол, реализующий пересылку независимых пакетов данных, называемых дейтаграммами, от одного компьютера к другому без гарантии их доставки).
- **Сеансовый уровень.** Обеспечивает установку, контроль и окончание сессии между приложениями. Уровень сессий координирует приложения, когда они взаимодействуют между двумя хостами.
- **Уровень представления.** Уровень представлений отвечает за представление данных в форме, понятной получателю. Например, для представления данных могут быть использованы такие кодировки, как Extended Binary Coded Decimal Interchange Code (EBCDIC) или American Standard Code for Information Interchange (ASCII). Если компьютеры, между которыми установлено сетевое соединение, используют различные протоколы, то presentation layer обеспечивает их корректное взаимодействие. Уровень представлений также отвечает за шифрацию и сжатие данных.
- **Прикладной уровень.** Уровень приложений определяет, какие ресурсы существуют для связи между хостами.

Каждый уровень взаимодействует только с соседними уровнями, протокол взаимодействия стандартизован. Это позволяет использовать реализации сетевого и программного обеспечения от разных производителей в различных комбинациях. *Например, протоколы высокого уровня (HTTP, FTP) не зависят от физических параметров используемой сети.*

Большинство сетевых приложений можно классифицировать как клиент-серверные приложения.

Модель «Клиент-сервер»

Каждая из сторон виртуального соединения называется «**сокет**» (**socket**). Приложение-сервер инициализируется при запуске и далее бездействует, ожидая поступления запроса от клиента. **Типы приложений-серверов** - Сервер последовательной обработки запросов и Сервер параллельной обработки запросов. Процесс-клиент посылает запрос на установление соединения с сервером, требуя выполнить для него определенную функцию.

Понятие порта

Компьютер (обычно) имеет только одно физическое соединение с сетью. Соединение описывается, например, IP-адресом (32 бита на сегодняшний день). Как различать информацию для различных приложений? Сокет

привязывается к порту. Порт описывается 16-битным числом. Порты 0-1023 зарезервированы

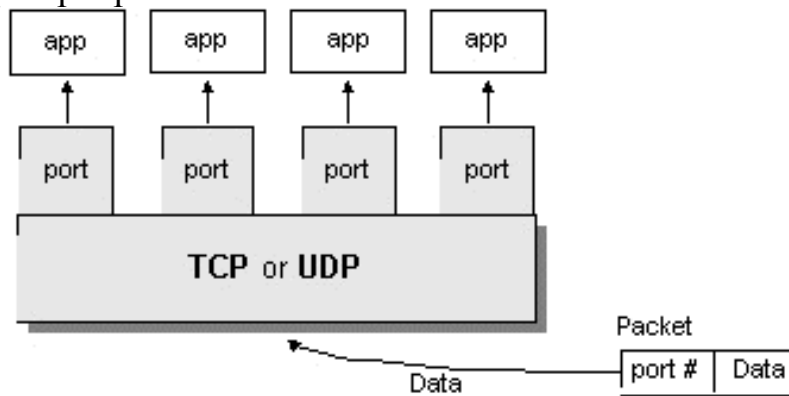


Рисунок 4 - Понятие порта

Абстракция сокета

Сетевое соединение – это процесс передачи данных по сети между двумя компьютерами или процессами. Сокет – конечный пункт передачи данных. Для программ сокет – одно из окончаний сетевого соединения. Для установления соединения каждая из сетевых программ должна иметь свой собственный сокет. Связь между двумя сокетами может быть ориентированной или не ориентированной на соединение. Сокет связан с номером порта.

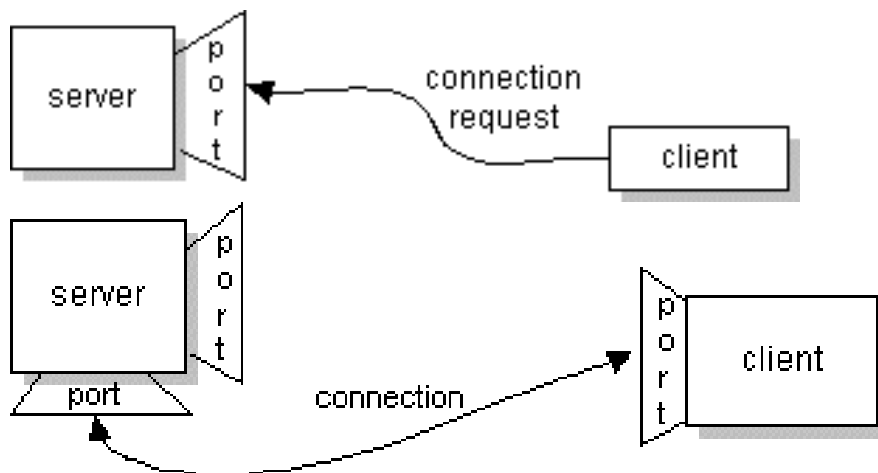


Рисунок 5 - Абстракция сокета

А что же на Java?

Сокеты инкапсулированы в экземпляры специальных классов. Все низкоуровневое взаимодействие скрыто от пользователя. Существует семейство классов, обеспечивающих настройку сокетов и работу с ними. Классы для работы с сетью в Java располагаются в пакете `java.net`.

Пакет `java.net`

- Адресация.
- Установление TCP-соединения.
- Передача/прием дейтаграмм через UDP.
- Обнаружение/идентификация сетевых ресурсов.
- Безопасность: авторизация / права доступа.

Адресация

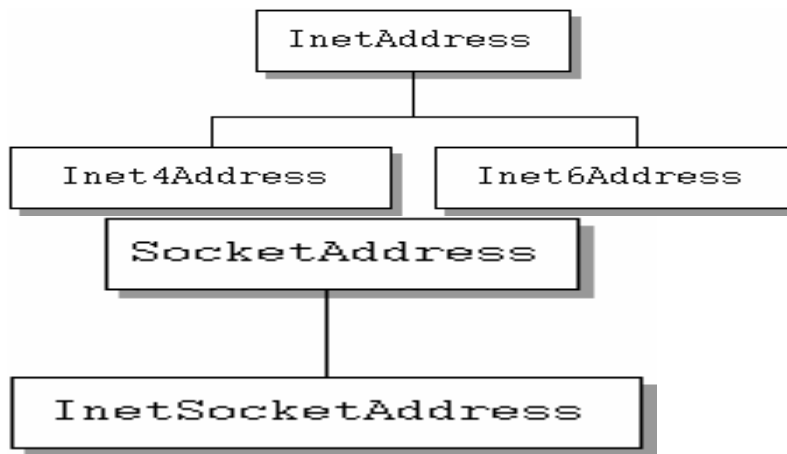


Рисунок 6 - Адресация

Класс `InetAddress` является интернет-адресом, или IP. Экземпляры этого класса создаются не с помощью конструкторов, а с помощью статических методов:

- `InetAddress getLocalHost()` - возвращает IP-адрес машины, на которой выполняется Java-программа.
- `InetAddress getByName(String name)` - возвращает адрес сервера, чье имя передается в качестве параметра. Это может быть как DNS-имя, так и числовой IP, записанный в виде текста, например, "67.11.12.101".
- `InetAddress[] getAllByName(String name)` - определяет все IP-адреса указанного сервера.

Класс `Inet4Address` служит для описания адреса, состоящего из 4 байтов, с помощью класса `Inet6Address` описывается адрес, состоящий из 16 байтов. Начиная с JDK 1.4. существует класс `InetSocketAddress(String hostname, int port)`, который создаёт объект адреса для указанного узла и порта.

Общая схема соединения

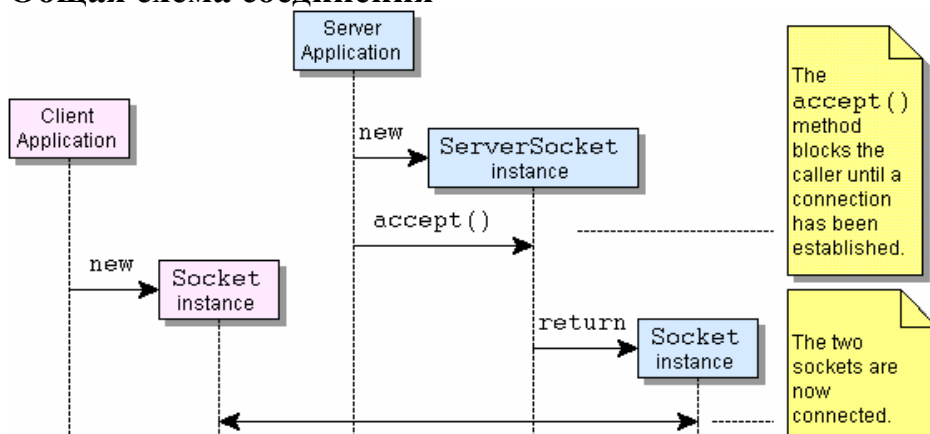


Рисунок 7 - Общая схема соединения

Класс `Socket`

- Реализует клиентский сокет и его функции
- Конструкторы
 - `Socket()`
 - `Socket(InetAddress address, int port)`
 - `Socket(InetAddress address, int port, InetAddress localAddr, int localPort)`

- Socket(String host, int port)
- Socket(String host, int port, InetAddress localAddr, int localPort)
- **Методы**
 - void close() – закрывает используемый сокет.
 - InetAddress getLocalAddress() -
 - InputStream getInputStream() – возвращает поток, позволяющий читать данные, переданные по сети.
 - OutputStream getOutputStream() - возвращает поток, позволяющий передавать данные по сети.
 - static void setSocketImplFactory(SocketImplFactory fac)
 - И прочие...

Порядок работы с сокетом клиента

- Открытие сокета.
- Открытие потока ввода и/или потока вывода для сокета.
- Чтение и запись в потоки согласно установленному протоколу общения с сервером.
- Закрывание потоков ввода-вывода.
- Закрывание сокета.

Пример 19.

```
import java.io.*;
import java.net.*;
```

```
public class EchoClient {
    public static void main(String[] args) throws IOException {
        Socket echoSocket = null;
        PrintWriter out = null;
        BufferedReader in = null;
        try {
            echoSocket = new Socket("taranis", 7);
            out = new PrintWriter(echoSocket.getOutputStream(), true);
            in = new BufferedReader(new InputStreamReader(
                echoSocket.getInputStream()));
        } catch (UnknownHostException e) {
            System.err.println("Don't know about host: taranis.");
            System.exit(1);
        } catch (IOException e) {
            System.err.println("Couldn't get I/O for the connection to:" +
                "taranis.");
            System.exit(1);
        }
        BufferedReader stdIn = new BufferedReader(
            new InputStreamReader(System.in));
        String userInput;

        while ((userInput = stdIn.readLine()) != null) {
            out.println(userInput);
            System.out.println("echo: " + in.readLine());
        }
    }
}
```

```

    out.close();
    in.close();
    stdIn.close();
    echoSocket.close();
}
}

```

Класс ServerSocket

- Реализует серверный сокет и его функции
- Конструкторы
 - ServerSocket()
 - ServerSocket(int port)
 - ServerSocket(int port, int backlog)
- Методы
 - void close() – закрывает используемый сокет
 - Socket accept() – приостанавливает выполнение программы и ожидает обращения клиента
 - void bind(SocketAddress endpoint)
 - И прочие...

Пример 20.

```

try {
    serverSocket = new ServerSocket(4444);
} catch (IOException e) {
    System.out.println(
        "Could not listen on port: 4444");
    System.exit(-1);
}
Socket clientSocket = null;
try {
    clientSocket = serverSocket.accept();
} catch (IOException e) {
    System.out.println("Accept failed: 4444");
    System.exit(-1);
}

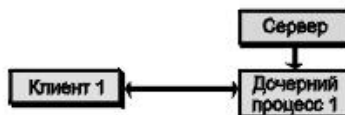
```

Сервер параллельной обработки запросов

Стадия 1: Установление соединения клиент-сервер



Стадия 2: Сервер параллельной обработки передает управление дочернему процессу



Стадия 3: Если в момент обработки запроса от первого клиента поступает запрос от второго, сервер параллельной обработки передает управление второму дочернему процессу



Рисунок 8 - Сервер параллельной обработки запросов

Дейтаграммы

Дейтаграмма – независимое, самодостаточное сообщение, посылаемое по сети, чья доставка, время (порядок) доставки и содержимое не гарантируются. Могут использоваться как для адресной, так и для широковещательной рассылки.

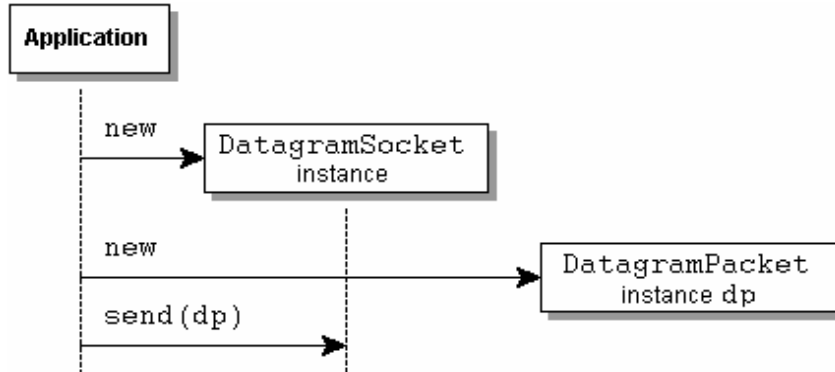


Рисунок 9 - Передача дейтаграмм

DatagramPacket

- Экземпляры являются прототипами дейтаграмм-сообщений
- Конструкторы
DatagramPacket(byte[] buf, int offset, int length, InetAddress address, int port)
И прочие...
- Методы
 - **byte[] getData()**
 - **int getLength()**
 - **int getOffset()**
 - **SocketAddress getSocketAddress()**
 - **void setSocketAddress(SocketAddress address)**
 - **void setData(byte[] buf, int offset, int length)**
 - И прочие...

DatagramSocket

- Экземпляры являются не ориентированными на соединение сокетами
- Конструкторы
DatagramSocket()
DatagramSocket(int port, InetAddress laddr)
И другие...
- Методы
 - void bind(SocketAddress addr)**
 - void close()**
 - void connect(InetAddress address, int port)**
 - void send(DatagramPacket p)**
 - void receive(DatagramPacket p)**
 - И другие...

Uniform Resource Locator

- URL – адрес ресурса в Интернет

- Имя протокола
Протокол, используемый для связи
 - Имя хоста
Имя компьютера, на котором расположен ресурс
 - Имя файла
Путь к файлу на компьютере
 - Номер порта
Номер порта для соединения (необязателен, если порт не указан, то используется значение по умолчанию для указанного протокола)
 - Ссылка
Ссылка на именованный якорь (необязательна)
 - Может быть абсолютным и относительным
- URL gamelan = new URL("http", "www.gamelan.com", 80, "pages/Gamelan.network.html");
Для простого извлечения содержимого заданного ресурса достаточно использовать метод `openStream()` класса `URL`. Этот метод возвращает объект `InputStream`. Прямое чтение из URL – пример 21.

Пример 21.

```
import java.net.*;
import java.io.*;

public class URLReader {
    public static void main(String[] args) throws Exception
    {
        URL yahoo = new URL("http://www.yahoo.com/");
        BufferedReader in = new BufferedReader(
            new InputStreamReader(
                yahoo.openStream()));

        String inputLine;
        while ((inputLine = in.readLine()) != null)
            System.out.println(inputLine);
        in.close();
    }
}
```

Для получения дополнительной информации о ресурсе потребуется использовать класс `URLConnection`, который предоставляет гораздо больше средств управления доступом к Web-ресурсам. Для получения объекта `URLConnection` нужно вызвать метод `openConnection()` класса `URL`. Чтение из URL-соединения – пример 22.

Пример 22.

```
import java.net.*;
import java.io.*;

public class URLConnectionReader {
    public static void main(String[] args) throws Exception
    {
        URL yahoo = new URL("http://www.yahoo.com/");
        URLConnection yc = yahoo.openConnection();
        BufferedReader in = new BufferedReader(
            new InputStreamReader(
                yc.getInputStream()));
    }
}
```

```

String inputLine;
while ((inputLine = in.readLine()) != null)
    System.out.println(inputLine);
in.close();
}
}

```

Существует несколько методов класса `URLConnection`, предназначенных для указания свойств соединения ещё до подключения к серверу. По умолчанию соединение получает входной поток для чтения, но не получает выходной поток для записи. Для получения выходного потока нужно вызвать метод `setDoOutput(true)`. Запись в URL-соединение – пример 23.

Пример 23.

```

import java.io.*;
import java.net.*;
public class Reverse {
    public static void main(String[] args) throws Exception {
        if (args.length != 1) {
            System.err.println("Usage: java Reverse" +
                               "string_to_reverse");
            System.exit(1);
        }
        String stringToReverse = URLEncoder.encode(args[0], "US-ASCII");
        URL url = new URL("http://java.sun.com/cgi-bin/backwards");
        URLConnection connection = url.openConnection();
        connection.setDoOutput(true);
        PrintWriter out = new PrintWriter(
            connection.getOutputStream());
        out.println("string=" + stringToReverse);
        out.close();
        BufferedReader in = new BufferedReader(
            new InputStreamReader(
                connection.getInputStream()));
        String inputLine;
        while ((inputLine = in.readLine()) != null)
            System.out.println(inputLine);
        in.close();
    }
}

```

Класс Graphics

`java.awt.Graphics` - базовый класс, предназначенный для рисования в контекстах компонентов, в изображениях в памяти и т.д.

- **`void drawArc(int x, int y, int width, int height, int startAngle, int arcAngle)`** - Этот метод вычерчивает (`fillArc` заполняет) дугу, ограниченную прямоугольником (`x, y, width, height`), начинающуюся с угла `startAngle` и имеющую угловой размер `arcAngle`. Ноль градусов соответствует положению часовой стрелки на 3 часа, угол отсчитывается против часовой стрелки (например, 90 градусов соответствуют 12 часам, 180 - 9 часам, и так далее).
- **`void drawString(String str, int x, int y)`** - Этот метод выводит строку с использованием текущего шрифта и цвета. Точка с координатами (`x, y`)

соответствует левой границе базовой линии символов, а не левому верхнему углу, как это принято в других методах рисования. Базовая линия (baseline) - линия, по которой выравниваются нижние границы символов.

- и т.д.

Работа с цветом

Класс **java.awt.Color**.

- Можно использовать константы класса (Color.BLUE, Color.RED) для задания какого-либо из общеупотребительных цветов (blue, red).
- Для создания нового цвета можно использовать один из описанных ниже конструкторов:
Color(int, int, int) - параметрами для этого конструктора являются три целых числа в диапазоне от 0 до 255 для красного, зеленого и голубого компонентов цвета.
Color(int) - у этого конструктора - один целочисленный аргумент, в котором в упакованном виде заданы красный, зеленый и голубой компоненты цвета. Красный занимает биты 16-23, зеленый - 8-15, голубой - 0-7.
Color(float, float, float) - последний из конструкторов цвета, принимает в качестве параметров три значения типа float (в диапазоне от 0.0 до 1.0) для красного, зеленого и голубого базовых цветов.
- Методы получения параметров цвета
getRed(), getGreen(), getBlue() - каждый из этих методов возвращает в младших восьми битах результата значение соответствующего базового компонента цвета.

Работа со шрифтами

- Класс **java.awt.Font**
 - Константы
 - Конструкторы
Font(String name, int style, int size) - создает новый шрифт с указанным именем, стилем и размером. Имена шрифтов: Dialog, Helvetica, TimesRoman, Courier и Symbol. Для указания стиля шрифта внутри данного семейства предусмотрены три статические переменные. - Font.PLAIN, Font.BOLD и Font.ITALIC, что соответствует обычному стилю, курсиву и полужирному.
 - Методы модификации и получения параметров шрифта (getSize - метод возвращает целое число, представляющее собой размер шрифта, isBold, isItalic, isPlain - методы возвращают true в том случае, если стиль шрифта - полужирный (bold), курсив (italic) или обычный (plain), соответственно).
- Класс **java.awt.FontMetrics**
 - Содержит методы определения геометрических характеристик шрифтов (getAscent, getDescent, getHeight - эти методы возвращают подъем, снижение и ширину шрифта. Сумма

подъема и снижения дают полную высоту шрифта. Высота шрифта - это не просто расстояние от самой нижней точки букв g и y до самой верхней точки заглавной буквы Т и символов вроде скобок. Высота включает подчеркивания и т.п.).

Высота (height) - размер от верхней до нижней точки самого высокого символа в шрифте.

Базовая линия (baseline) - линия, по которой выравниваются нижние границы символов (не считая снижения (descent)).

Подъем (ascent) - расстояние от базовой линии до верхней точки символа.

Снижение (descent) - расстояние от базовой линии до нижней точки символа.

Abstract Window Toolkit (AWT)

Существуют два взгляда на проблему создания независимой от платформы библиотеки. Один подход заключается в том, что упор делается на графику низкого уровня - рисование пикселей, при этом разработчики библиотеки сами заботятся о внешнем виде каждого компонента. При другом подходе создаются абстракции, подходящие для библиотек каждой из операционных систем, и именно "родные" пакеты данной операционной системы служат подъемной силой для архитектурно-нейтральной библиотеки на каждой из платформ. В Java при создании библиотеки *Abstraction Window Toolkit (AWT)* выбран второй подход.

Особенности AWT:

- Компоненты являются компонентами ОС
 - большое количество native-кода
 - отображение изменяется при смене ОС
- Применение:
 - в апплетах
 - в оконных приложениях
- Абстрактный класс *Component* определяет базовую функциональность всех компонентов. Это класс, который инкапсулирует все атрибуты визуального интерфейса - обработка ввода с клавиатуры, управление фокусом, взаимодействие с мышью, уведомление о входе/выходе из окна, изменения размеров и положения окон, прорисовка своего собственного графического представления, сохранение текущего текстового шрифта, цветов фона и переднего плана (более 100 методов).

Иерархия классов

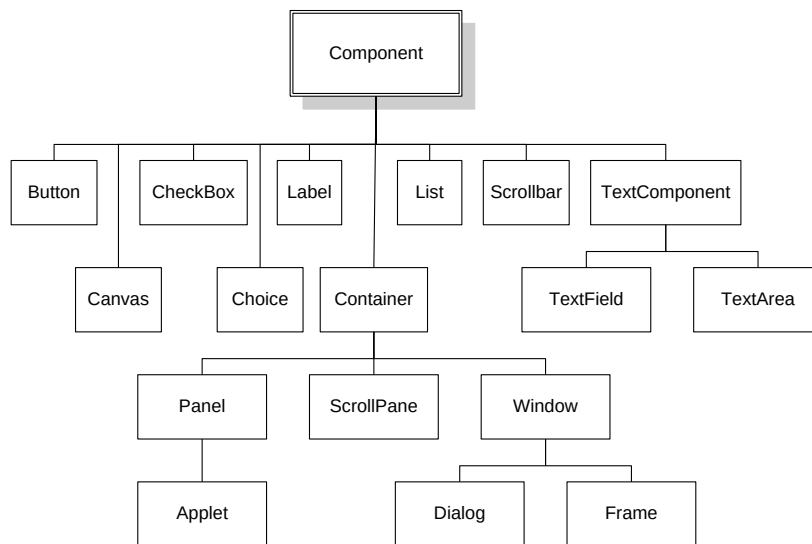


Рисунок 10

Container

Container - это абстрактный подкласс класса Component, определяющий дополнительные методы, которые дают возможность помещать в него другие компоненты, что дает возможность построения иерархической системы визуальных объектов. Container отвечает за расположение содержащихся в нем компонентов с помощью интерфейса LayoutManager, описание которого будет позднее.

Panel

Класс Panel - это очень простая специализация класса Container. В отличие от последнего, он не является абстрактным классом. Поэтому о Panel можно думать, как о допускающем рекурсивную вложенность экранном компоненте. С помощью метода add в объекты Panel можно добавлять другие компоненты. После того, как в него добавлены какие-либо компоненты, можно вручную задавать их положение и изменять размер с помощью методов setLocation, setSize и setBounds класса Component.

Мы уже использовали один из подклассов Panel - Applet. Каждый раз, когда мы создавали Applet, методы paint и update рисовали его изображение на поверхности объекта Panel. Давайте познакомимся с основными компонентами AWT, которые можно вставлять в пустую Panel при создании графических приложений.

Canvas

- Семантически «пустой» компонент.
- Позволяет отрисовать на себе произвольный вид компонента.

Label

- Отображает строку.
- Позволяет управлять параметрами текста, выравнивания.

Button

- Реализует кнопку.
- Создает события.

Checkbox

- Реализует «флажок»

- Имеет методы установки значения
- Создает события

CheckboxGroup

- Реализует выбор одного из вариантов
- Имеет методы установки значения
- Создает события

Choice

- Реализует выпадающий список
- Имеет методы управления состоянием
- Создает события

List

- Реализует список с возможностью выбора нескольких вариантов и прокруткой.
- Имеет методы управления состоянием.
- Создает события.

Scrollbar

- Реализует полосу прокрутки.
- Может иметь два положения.
- Имеет методы управления состоянием.
- Создает события.

TextField

- Реализует однострочную область для ввода текста.
- Имеет методы управления состоянием.
- Создает события.

TextArea

- Реализует простой многострочный редактор.
- Имеет методы управления состоянием и изменения текста.
- Создает события.

Менеджеры компоновки

- Управляют размещением компонентов в контейнере, учитывая параметры этих компонентов (например, предпочтительный размер).
- Реализуют интерфейс **java.awt.LayoutManager**.
- Устанавливаются с помощью метода **setLayout()** контейнера.
- Примеры простых компоновок:
- **Простая поточная компоновка FlowLayout.** Класс **FlowLayout** реализует простой стиль размещения, при котором компоненты располагаются, начиная с левого верхнего угла, слева направо и сверху вниз. Если в данную строку не помещается очередной компонент, он располагается в левой позиции новой строки. Справа, слева, сверху и снизу компоненты отделяются друг от друга небольшими промежутками. Ширину этого промежутка можно задать в конструкторе **FlowLayout**. Каждая строка с компонентами выравнивается по левому или правому краю, либо центрируется в зависимости от того, какая из констант **LEFT**, **RIGHT** или **CENTER**

была передана конструктору. Режим выравнивания по умолчанию - *CENTER*, используемая по умолчанию ширина промежутка - 5 пикселей.

- **Граничная компоновка BorderLayout.** Класс `BorderLayout` реализует стиль размещения, в котором предусмотрено четыре узких компонента фиксированной ширины по краям, и одна большая область в центре, которая может расширяться и сужаться в двух направлениях, занимая все свободное пространство окна. У каждой из этих областей есть строки-имена: *String.North*, *String.South*, *String.East* и *String.West* соответствуют четырем краям, а *Center* - центральной области.
- **Компоновка в сетку GridLayout.** Класс `GridLayout` размещает компоненты в простой равномерной сетке. Конструктор этого класса позволяет задавать количество строк и столбцов.
- **Компоновка «Колода карт» CardLayout.** Класс `CardLayout` по своему уникален. Он отличается от других программ управления размещением компонентов тем, что представляет несколько различных вариантов размещения, которые можно сравнить с колодой карт. Колоду можно тасовать так, чтобы в данный момент времени наверху была только одна из карт. Это может быть полезно при создании интерфейсов пользователя, в которых есть необязательные компоненты, включаемые и выключаемые динамически в зависимости от реакции пользователя.
- **Сетка с настраиваемыми размерами GridBagLayout.**

Модель делегирования обработки событий

- Источник генерирует событие и посылает его одному или нескольким слушателям.
- Слушатель просто ждет поступления события.
- Получив событие, слушатель обрабатывает его и затем возвращает управление.
- Слушатели должны зарегистрироваться у источника – события посылаются только зарегистрировавшимся слушателям.
- Логика кода, обрабатывающего события, отделена от логики интерфейса, генерирующего события.

Событие

Событие – это объект, описывающий изменение состояния источника. В модели обработки событий Java разным типам событий соответствуют различные классы Java. Каждое событие является подклассом класса **java.util.EventObject**. События пакета AWT являются подклассом **java.awt.AWTEvent**. Для удобства события различных типов пакета AWT (например, `MouseEvent` или `ActionEvent`) помещены в новый пакет **java.awt.event**. Для каждого события существует порождающий его объект, который можно получить с помощью метода **public Object getSource()**, и каждому событию пакета AWT соответствует определенный идентификатор, который позволяет получить метод **public int getID()**. Это значение используется для того, чтобы отличать события различных типов, которые могут описываться одним и тем же классом событий. Например, для класса

FocusEvent возможны два типа событий: *FocusEvent.FOCUS_GAINED* и *FocusEvent.FOCUS_LOST*.

Источник

- **Источник** – объект, генерирующий события. Источник может генерировать несколько типов событий.
- Регистрация слушателя:
public void addTypeListener(TypeListener el) throws java.util.TooManyListenersException
public void removeTypeListener(TypeListener el)

Слушатель

- Слушатель – объект, получающий уведомление о событии.
- Может быть зарегистрирован одним или несколькими источниками.
- Должен реализовывать методы для приема и обработки уведомлений.
- Существует набор интерфейсов **TypeListener**, описывающих методы обработки событий. Например, объекты слушателей событий *ActionEvent* должны реализовывать интерфейс *ActionListener*. В пакете *java.awt.event* содержатся интерфейсы слушателей для каждого из определенных в нем типов событий (например, для событий *MouseEvent* здесь определено два интерфейса слушателей: *MouseListener* и *MouseMotionListener*). Все интерфейсы слушателей событий являются расширениями интерфейса *java.util.EventListener*. В этом интерфейсе не определяется ни один из методов, но он играет роль интерфейса-метки, в котором однозначно определены все слушатели событий как таковые.

Таблица 1 - Элементы AWT и порождаемые события

Элемент	Событие	Значение
Component	ComponentEvent	Элемент либо перемещен, либо он стал скрытым, либо видимым
	FocusEvent	Элемент получил или потерял фокус ввода
	KeyEvent	Пользователь нажал или отпустил клавишу
	MouseEvent, MouseMotionEvent	Пользователь нажал или отпустил кнопку мыши, либо курсор мыши вошел или покинул область, занимаемую элементом, либо пользователь просто переместил мышь или переместил мышь при нажатой кнопке мыши
Button	ActionEvent	Пользователь нажал кнопку

Checkbox	ItemEvent	Пользователь установил или сбросил флажок
CheckboxMenuItem	ItemEvent	Пользователь установил или сбросил флажок рядом с пунктом меню
Choice	ItemEvent	Пользователь выбрал элемент списка или отменил его выбор
Container	ContainerEvent	Элемент добавлен в контейнер или удален из него
List	ActionEvent	Пользователь выполнил двойной щелчок мыши на элементе списка
	ItemEvent	Пользователь выбрал элемент списка или отменил выбор
MenuItem	ActionEvent	Пользователь выбрал пункт меню
Scrollbar	AdjustmentEvent	Пользователь осуществил прокрутку
TextComponent	TextEvent	Пользователь внес изменения в текст элемента
TextField	ActionEvent	Пользователь закончил редактирование текста элемента
Window	WindowEvent	Окно было открыто, закрыто, представлено в виде пиктограммы, восстановлено или требует восстановления

Таблица 2 - События, слушатели и методы

Класс события	Интерфейс слушателя	Методы слушателя
ActionEvent	ActionListener	actionPerformed()
AdjustmentEvent	AdjustmentListener	adjustmentValueChanged()
ComponentEvent	ComponentListener	componentHidden() componentMoved() componentResized() componentShown()

ContainerEvent	ContainerListener	componentAdded() componentRemoved()
FocusEvent	FocusListener	focusGained() focusLost ()
ItemEvent	ItemListener	itemStateChanged()
KeyEvent	KeyListener	keyPressed() keyReleased() keyTyped()
MouseEvent	MouseListener	mouseClicked() mouseEntered() mouseExited() mousePressed() mouseReleased()
MouseEvent	MouseMotionListener	mouseDragged() mouseMoved()
TextEvent	TextListener	textValueChanged()
WindowEvent	WindowListener	windowActivated() windowClosed() windowClosing() windowDeactivated() windowDeiconified() windowIconified() windowOpened()

Классы-адаптеры

Для каждого интерфейса слушателей событий, содержащего несколько методов, в пакете `java.awt.event` определен простой класс-адаптер, который обеспечивает пустое тело для каждого из методов соответствующего интерфейса. Когда нужен только один или два таких метода, иногда проще получить подкласс класса-адаптера, чем реализовать интерфейс самостоятельно. При получении подкласса адаптера требуется лишь переопределить те методы, которые нужны, а при прямой реализации интерфейса необходимо определить все методы, в том числе и ненужные в данной программе. Заранее определенные классы-адаптеры называются так же, как и интерфейсы, которые они реализуют, но в этих названиях `Listener` заменяется на `Adapter`: `MouseAdapter`, `WindowAdapter` и т.д.

Пример обработки событий представлен примером 24.

Пример 24.

```
/* <applet code = "Scribble2" width=640 height=480> </applet> */
import java.applet.*;
import java.awt.*;
import java.awt.event.*;
```

```

public class Scribble2 extends Applet implements
    MouseListener, MouseMotionListener {
    private int last_x, last_y;
    public void init() {
        this.addMouseListener(this);
        this.addMouseMotionListener(this);
    }
    public void mousePressed(MouseEvent e) {
        last_x = e.getX();
        last_y = e.getY();
    }

    public void mouseDragged(MouseEvent e) {
        Graphics g = this.getGraphics();
        int x = e.getX(), y = e.getY();
        g.drawLine(last_x, last_y, x, y);
        last_x = x; last_y = y;
    }

    public void mouseReleased(MouseEvent e) {}
    public void mouseClicked(MouseEvent e) {}
    public void mouseEntered(MouseEvent e) {}
    public void mouseExited(MouseEvent e) {}
    public void mouseMoved(MouseEvent e) {}
}

```

Результат работы примера 24 – рисунок 11.

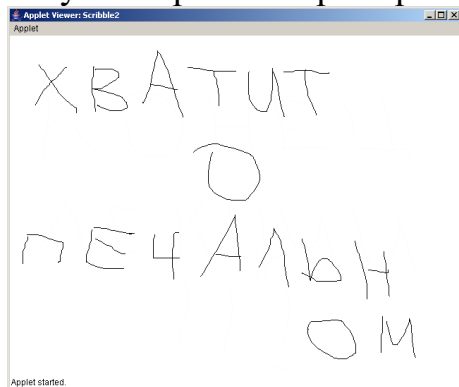


Рисунок 11 - Результат работы примера 24.

Вложенные классы и интерфейсы

Классы и интерфейсы могут быть объявлены в составе других классов и интерфейсов, как в качестве членов, так и внутри блоков кода. Такие вложенные классы и вложенные интерфейсы способны принимать различные формы, каждая из которых обладает собственными свойствами и назначением.

Возможность определять вложенные типы, предусмотренная в языке Java служит нескольким целям:

- Вложенные типы позволяют представить тип в виде логически связанных структурных групп и контекстов.
- Вложенные типы – это простой и эффективный инструмент объединения семантики соотносимых объектов. Эта возможность

активно применяется при создании программных сред, управляемых событиями, примерами могут служить пакет AWT и компонентная архитектура JavaBeans.

Вложенные типы считаются частью внешнего типа, и оба типа находятся во взаимно «доверительных» отношениях — один из них обладает правом обращаться ко всем членам другого. Основные различия между вложенными типами обусловлены тем, является ли вложенный тип классом или интерфейсом, и тем, к какой категории относится внешний тип — к классам или интерфейсам.

Пример вложенного класса – пример 25.

Пример 25.

```
package ru.smr.ipsi.ConformingEstimation;
```

```
abstract public class ConformingEstimation {
    private double[][] x;
    private double[] y;
    protected final int n;
    protected final int m;

    protected class LowerSubsystem {
        public int[] code;
        public double[] estimate;
        private void calculateEstimate() {
            estimate = solveLinearEquationSystem(x, y);
        }
    }
    // ...
}
```

Вложенные типы могут быть объявлены как статические или нестатические. *Статические вложенные типы более просты для восприятия и использования, поэтому с них мы и начнём.*

Статические вложенные классы

Статический вложенный класс - это простейшая форма вложенного класса, в его объявлении присутствует служебное слово *static*. Если класс вложен в интерфейс, он получает статус статического по умолчанию, и модификатор *static*, в соответствии с принятым соглашением, опускается. Статический вложенный класс может наследовать другие классы, реализовывать любые интерфейсы и сам по себе служить объектом расширения для любого класса, обладающего необходимыми правами доступа. В объявлении статического вложенного класса, как и обычного класса, разрешается применять модификаторы *final* и *abstract*. Статические классы, вложенные в другой класс, являются членами последнего и допускают применение любых модификаторов доступа.

Пример 26.

```
abstract public class ClassA {
    protected static class ClassB {
        ...
    }
}
```

Вложенные интерфейсы

Вложенные интерфейсы всегда статичны, хотя соответствующий модификатор, `static`, в соответствии с принятым соглашением, принято опускать. Доступ определяется доступом внешнего класса или интерфейса.

Пример 27.

```
public class ClassA {  
    protected interface InterfaceB {  
        ...  
    }  
}
```

Нестатические вложенные классы

Нестатические вложенные классы называются внутренними классами. Объект внутреннего класса всегда ассоциируется с т.н. внешним объектом (с соответствующим объектом внешнего класса). Элементы внутреннего класса имеют доступ к полям и методам внешнего объекта. Внутренние классы не способны содержать статические члены, кроме полей вида `final static`, инициализируемых константами или постоянными выражениями. Внутренние классы, как и обычные, способны расширять любые другие классы, реализовывать интерфейсы и выступать в роли объектов наследования. Вполне допустимо использовать в объявлении внутреннего класса модификаторы `final` и `abstract`.

Пример 28.

```
abstract public class ClassA {  
    private class ClassB {  
        ...  
    }  
}
```

Локальные классы

Разрешается объявлять вложенные классы внутри блоков кода, таких как тело метода, конструктор или блок инициализации. Подобный локальный внутренний класс не является членом класса, к которому относится блок, а принадлежит самому блоку – точно так же, как обычная локальная переменная. Такие классы недоступны за пределами внешнего класса, т.к. не существует способов обращения к ним, но их экземпляры – это обычные объекты, которые позволяет передавать в качестве аргументов или возвращать из методов. Единственный модификатор, который допускается применять в объявлении локального класса, - это `final` (*модификаторы доступа запрещены*). Экземпляры имеют доступ не только к полям и методам внешнего объекта, но и к локальным переменным. Однако такие переменные должны быть объявлены как `final`.

Пример 29.

```
public class ClassA {  
    public Iterator iterator() {  
        class Iter implements Iterator {  
            ...  
        }  
        ...  
    }  
}
```

```
    return new Iter();  
}
```

Анонимные классы

Анонимные классы описываются непосредственно в выражении new и служат его частью. Тип, указанный после new, является базовым для объявляемого анонимного класса. Могут расширять один класс или реализовывать один интерфейс. Явно писать implements или extends нельзя. Не могут иметь конструкторов, так как имя конструктора должно совпадать с именем класса, а в данном случае класс не имеет имени. Параметры, необходимые для создания объекта, передаются конструктору суперкласса.

Пример 30.

```
import java.awt.*;  
import java.applet.*;  
  
public class MyApplet extends Applet {  
    ...  
    public void init() {  
        Button btn = new Button("?");  
        add(btn);  
        btn.addActionListener(new ActionListener() {  
            public void actionPerformed(ActionEvent e) {  
                ((Button)e.getSource()).setLabel("!");  
            }  
        });  
    }  
}
```

Технология Swing

Проблемы AWT

1. Сходные элементы в различных ОС могут иметь некоторые различия (например, меню, панели прокрутки).
2. ОС часто имеют элементы GUI, отсутствующие в других ОС (некоторые графические среды программирования, например, X11/Motif, не имеют такого богатого набора компонентов пользовательского интерфейса, как ОС Windows или Macintosh).
3. Использование native-методов в AWT приводит к возникновению ошибок на конкретных платформах (на разных платформах возникали разные ошибки, приходилось тестировать каждое приложение на каждой платформе).

В 1996 г. компания Netscape создала библиотеку программ для создания графического пользовательского интерфейса и назвала её IFC (Internet Foundation Classes). Эта библиотека была основана на совершенно других принципах. Элементы пользовательского интерфейса рисовались в пустом окне. Единственная функциональность, которая требовалась от оконной системы платформы, заключалась в отображении окна и рисовании в нём. Таким образом, элементы графического пользовательского интерфейса, созданные с помощью библиотеки IFC, выглядели и вели одинаково, но не зависели от платформы, на которой запускалась программа. Компании Sun и

Netscape объединили свои усилия и усовершенствовали этот подход, создав библиотеку под названием Swing. С тех пор слово Swing стало официальным названием набора инструментальных средств для создания машиннонезависимого пользовательского интерфейса.

Особенности Swing

1. Элементы GUI (меню, кнопки) отрисовываются в пустых окнах.
2. «Нативные» функции используются только для вывода окна, отрисовки и получения информации о действиях пользователя.
3. Обладает свойством «легковесности» (lightweight).
4. Набор элементов GUI более широк, чем в AWT, и может быть еще расширен.
5. Сильная привязка к «нативным» методам отсутствует, что снижает вероятность возникновения ошибок.
6. Отображение на различных платформах единообразно.

Look And Feel

Программисты, использующие Swing, могут придать своей программе внешний вид в заданном стиле. Программе можно придать различный вид, изменив «стиль» отрисовки компонентов. Есть разработанные «стили» для базовых платформ. Существует возможность создания собственных «стилей». *Sun разработала независимый от платформы стиль, названный Metal, который прозвали стилем Java. На рисунках 12-14 показано, как выглядит одна и та же программа при использовании различных стилей.*

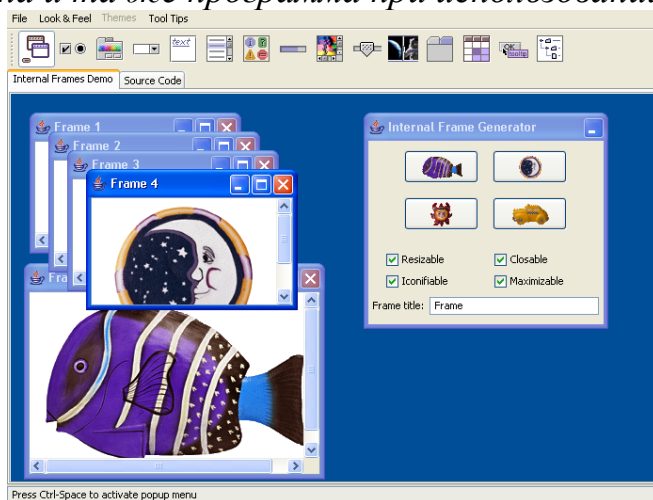


Рисунок 12 - Windows

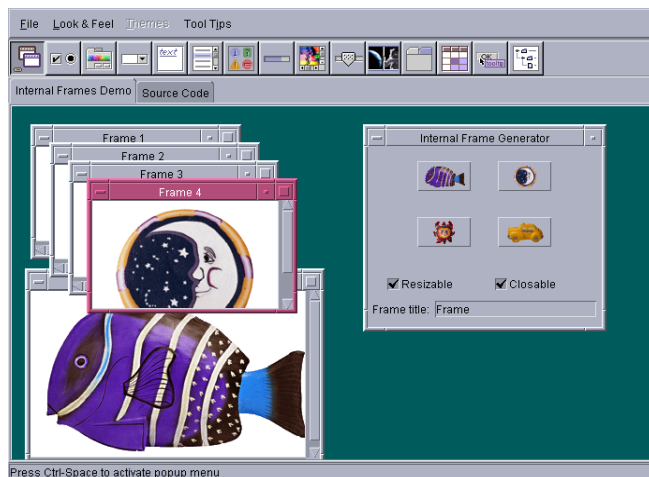


Рисунок 13 - Motif

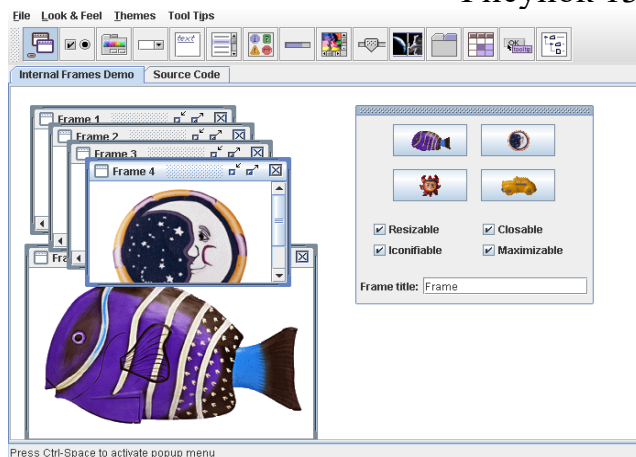


Рисунок 14 - Java (Metal)

Внешний вид программы может изменяться во время исполнения.
Перерисовка должна вызываться принудительно.

На рисунках 15-16 показана иерархия классов.

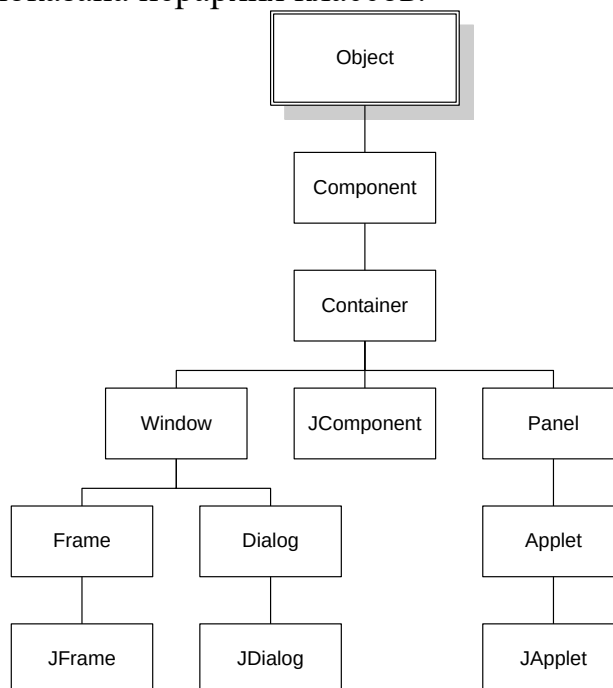


Рисунок 15 - Иерархия классов

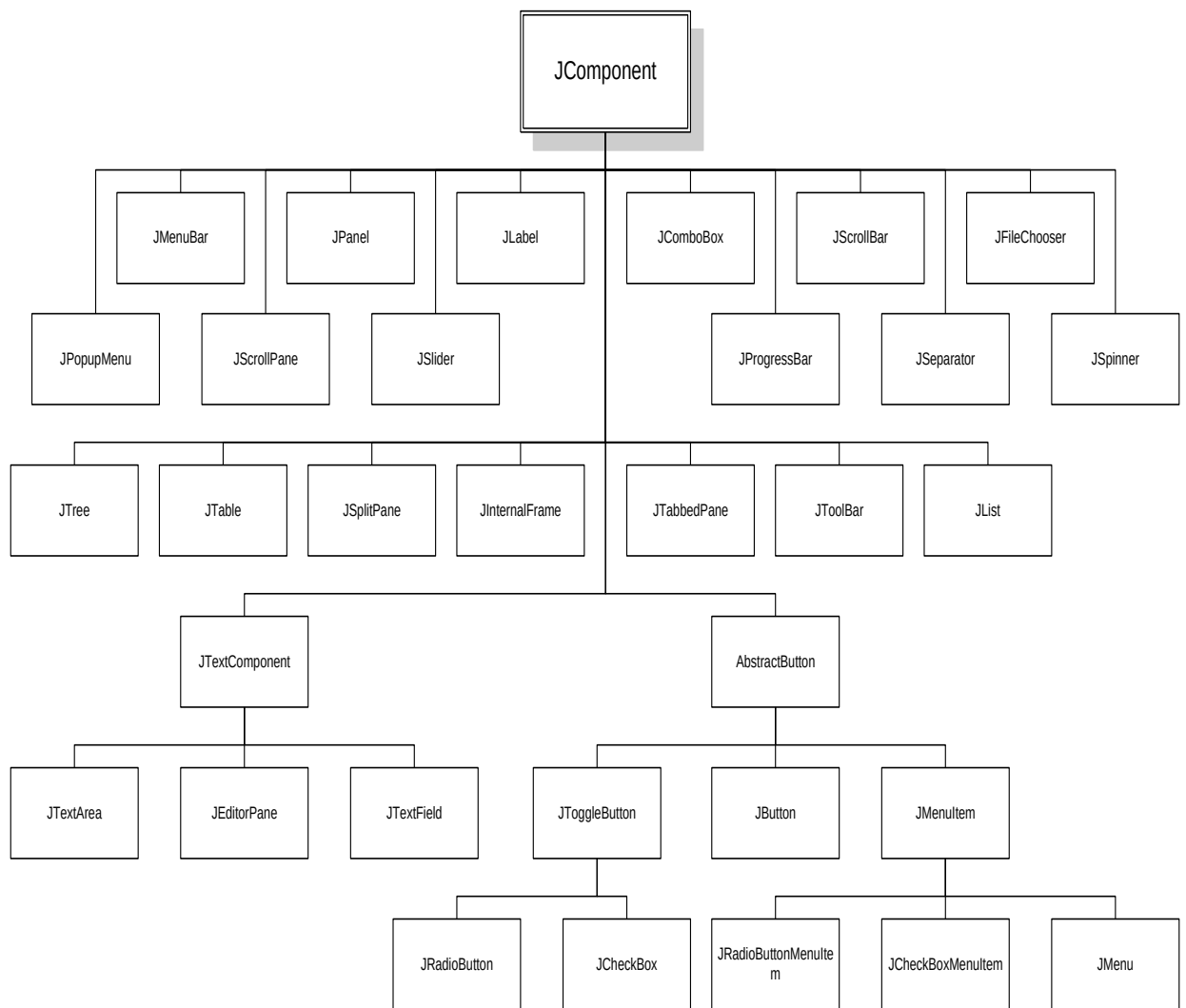


Рисунок 16 - Иерархия классов

Создание оконных приложений

- Используется класс **JFrame**.
- Содержимое окна находится на панели, ссылку на которую можно получить вызовом метода **getContentPane()**.
- Параметрами окна можно управлять.

Пример 31.

```

import java.awt.*;
import javax.swing.*;
public class CenteredFrameTest {
    public static void main(String[] args) {
        CenteredFrame frame = new CenteredFrame();
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
class CenteredFrame extends JFrame {
    public CenteredFrame() {
        Toolkit kit = Toolkit.getDefaultToolkit();
        Dimension screenSize = kit.getScreenSize();
        int screenHeight = screenSize.height;
        int screenWidth = screenSize.width;
        setSize(screenWidth / 2, screenHeight / 2);
    }
}
  
```

```

        setLocation(screenWidth / 4, screenHeight / 4);
        Image img = kit.getImage("icon.gif");
        setIconImage(img);    setTitle("CenteredFrame");
    }
}

```

Комментарий к примеру 31.

Определяем, что должно произойти, если пользователь закроет фрейм. В данном случае программа должна завершить свою работу. Для этого используется строка кода

```
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

Если бы в программе использовалось несколько фреймов, завершать работу только по тому, что пользователь закрыл один из них, было бы необязательно. По умолчанию закрытый фрейм исчезает с экрана, а программа продолжает работу.

Простое создание фрейма не приводит к его автоматическому появлению на экране. В начале своего существования все фреймы невидимы. Это дает возможность добавлять к фрейму компоненты еще до того, как он впервые появится на экране. Чтобы показать фрейм на экране, вызывается метод setVisible().

Если размер не задан явно, все фреймы по умолчанию имеют размер 0x0 пикселей. В профессиональных приложениях нужно сначала проверить разрешающую способность экрана и написать код, изменяющий размер фрейма в соответствии с полученной величиной: окно, которое прекрасно смотрится на экране портативного компьютера, на экране дисплея с большой разрешающей способностью будет выглядеть как почтовая марка. Для того, чтобы определить размер экрана, надо выполнить следующие действия: сначала следует вызвать статический метод getDefaultToolkit(), класса Toolkit, который вернет объект Toolkit. Класс Toolkit содержит много методов, предназначенных для взаимодействия с оконной системой конкретной платформы. Затем надо вызвать метод getScreenSize(), который вернет размер экрана в виде объекта Dimension. Этот объект содержит ширину и высоту в общедоступных переменных width и height. Мы также предоставим пиктограмму для фрейма. Поскольку процесс изображения рисунков на экране также зависит от ОС, для загрузки рисунка нам снова нужен объект Toolkit. Затем этот рисунок устанавливается в качестве пиктограммы.

```

Image img = kit.getImage("icon.gif");
setIconImage(img);    setTitle("CenteredFrame");

```

Отрисовка компонентов

- Отрисовка производится в методе **paintComponent()**.
- Для перерисовки по требованию вызывается метод **repaint()**.

Пример 32.

```

class NotHelloWorldPanel extends JPanel {
    public void paintComponent(Graphics g) {
        super.paintComponent(g);
        g.drawString("Not a Hello, World program",
            MESSAGE_X, MESSAGE_Y);
    }
}

```

```

    }
    public static final int MESSAGE_X = 75;
    public static final int MESSAGE_Y = 100;
}

```

Комментарий к примеру 32.

Можно было расширить класс JComponent.

JPanel предназначен служить контейнером, который может содержать в себе другие компоненты, но можно также рисовать непосредственно в нем. Однако следует иметь в виду, что панель непрозрачна, а это значит, что она отвечает за рисование всех пикселей в ее пределах. Простейший способ достичь того же - залить панель цветом фона, вызвав super.paintComponent(), в методе paintComponent каждого подкласса панели.

В целях экономии времени на перерисовку логично запоминать однажды нарисованный статичный объект как рисунок в памяти. При использовании Swing для этого не надо предпринимать дополнительных действий по выделению памяти и.д. Используется т.н. механизм «двойной буферизации», реализующий сохранение информации на уровне механизмов отрисовки. Для одного участка «видимой области» приложения используется не более одного изображения-буфера. Методы класса JComponent:

- **setDoubleBuffered(boolean aFlag).** Устанавливает, буферизует ли объект свой вывод.
- **boolean isDoubleBuffered().** Возвращает булевское значение, показывающее, используется ли двойная буферизация, или нет

Работа с графикой

- **java.awt.Graphics2D**
Класс-наследник класса **java.awt.Graphics**, обеспечивающий большую функциональность
- **java.awt.geom.***
Содержит набор классов работы с графическими примитивами

Работа с меню

- **javax.swing.JMenuBar**
Панель меню в верхней части окна
- **javax.swing.JMenu**
Меню как таковое
- **javax.swing.JMenuItem**
Единичный элемент меню
- **javax.swing.JCheckBoxMenuItem**
Элемент меню «флажок»
- **javax.swing.JRadioButtonMenuItem**
Элемент меню «радио-кнопка»
- **javax.swing.JPopup**
Всплывающее меню

Некоторые замечания

- Swing предоставляет более широкие и надежные возможности, чем AWT
- Эти возможности касаются в основном отрисовки компонентов
- Модель обработки событий и другие внутренние механизмы приложения относятся к AWT, а не к Swing
- Некорректно говорить, что Swing заменил AWT
- Современные средства разработки имеют визуальные редакторы для создания пользовательских интерфейсов

Нерассмотренные возможности

- Дополнительные события **javax.swing.event**
- Создание своих «стилей» отображения **javax.swing.plaf**
- «Высокоинтеллектуальные» компоненты **JTree**, **JTable** и т.д.
- Drag&Drop **java.awt.dnd**
- Вывод на печать **java.awt.print**

• Нововведения Java 7.0

1) Структура языка

1.1 Использование строк в операторе switch

Пример 33.

```
switch (param) {
    case "plain":
        System.err.println("bear");
        break;
    case "mountain":
        System.err.println("tiger");
        break;
    case "swamp":
        System.err.println("cockroach");
        break;
    case "forest": case "island":
        System.err.println("snake");
        break;
}
```

1.2 Подчеркивание в числах

```
int one_million = 1_000_000;
```

1.3 Multi-Catch

Пример 34.

```
try {
    ...// we are using reflection here
} catch (ClassCastException e) {
    doSomethingClever(e);
    throw e;
} catch (InstantiationException |
        NoSuchMethodException |
        InvocationTargetException e) {
    log(e);
    throw e;
}
```

1.4 оператор $\diamond$ - diamond (бриллиант)

```
List<String> strList = new ArrayList<>();
```

2) • Библиотеки

a. • NIO2

Имеется полный доступ к файловой системе: можно писать в файл через потоки или каналы, перебирать, копировать, перемещать и удалять файлы, создавать ссылки, получать атрибуты, сравнивать пути, «подсматривать» за файловой системой.

b. • Concurrency

- Удобный фреймворк для распараллеливаемых задач Fork/Join
- Новый класс синхронизации Phaser (барьер, в дополнение к CyclicBarrier и CountdownLatch)
- Новый интерфейс TransferQueue (расширяет BlockingQueue, имплементирован в LinkedTransferQueue)

c. • Client features

- Nimbus LaF – новый кроссплатформенный look-and-feel
- JLayer – декоратор для swing-компонент
- XRender pipeline – использование ресурсов графических видеокарт для отрисовки Java2D для повышения производительности.
- новый API AWT (полупрозрачные и фигурные окна, смешивание "тяжелых" и "лёгких" компонент (HW/LW))

3) • Виртуальная Машина

a. • The DaVinci Machine project

Этот проект прототипирует ряд расширений для JVM, таким образом, что она может запускать приложения, написанные не на Java, с уровнем производительности, сравнимым с самой Java. Акцент делается на доработке существующего байт-кода и архитектуры выполнения с расширениями общего назначения для работы с отличными от Java языками.

b. • InvokeDynamic bytecode

Исполнение метода требовало точного имени метода, списка параметров и возвращаемого значения на этапе компиляции. Новая инструкция Байткода InvokeDynamic позволяет использовать позднее связывание для методов с помощью MethodHandler. При первом вызове метода находится метод и создаётся handler. Последующие вызовы его переиспользуют.

• Нововведения Java 8.0

- **Java**
 - Ламбда-выражения;
 - Ссылки на методы;
 - Повторяемые аннотации;
 - Аннотации на типы данных;
 - Рефлексия для параметров методов;
 - Методы по-умолчанию.
- **Коллекции**
 - Новый API для потоков;
 - Параллельная сортировка массивов;
 - Улучшение производительности NashMaps.
- **Компактные сборки Java**
 - Возможность создания профилей для платформы Java SE, которые включают в себя не всю платформу целиком, а некоторую ее часть.
- **Безопасность**
 - Новая реализация `AccessController.doPrivileged`, позволяющая устанавливать подмножество привелегий без необходимости проверки всех остальных уровней доступа;
 - Password-based алгоритмы стали более устойчивыми;
 - Добавлена поддержка SSL/TLS Server Name Indication (NSI) в JSSE Server;
 - Улучшено хранилище ключей (KeyStore);
 - Добавлен алгоритм SHA-224;
- **Инструменты**
 - Добавлена команда `jjs` для использования нового JavaScript-движка Nashorn;
 - Команда `java` может запускать JavaFX приложения;
 - Добавлена команда `jdeps` для анализа `.class`-файлов.
- **Интернационализация**
 - Добавлена поддержка Unicode 6.2.0;
 - Добавлен новый API для `Calendar` и `Locale`;
- Новый API `Date/Time`;
- Новый движок JavaScript Nashorn;
- **[N]IO**
 - Улучшена производительность конструктора `java.lang.String(byte[], *)` и метода `java.lang.String.getBytes()`.
- Добавлен стандартный класс для работы с Base64;
- Добавлена поддержка беззнаковой арифметики;
- Удален мост JDBC-ODBC;
- Удален `PermGen`, изменен способ хранения мета-данных классов.
- Добавлено несколько новых классов для потокобезопасной работы с `ConcurrentHashMap`, `Atomics`, `ForkJoinPool`, `Locks`.

1. Лямбда-выражения

Наиболее значимым нововведением в Java 8 являются лямбда-выражения (Проект Ламбда). Они позволяют заменить анонимные классы на функции.

Структура лямбда-выражения выглядит так:

параметры -> тело

Лямбда-выражения обладают несколькими свойствами:

1. Задание типов параметров не является обязательным;
2. Использование скобок, заключающих параметры лямбда-выражения не обязательно;
3. Использование фигурных скобок в теле лямбда-выражения не обязательно, если тело состоит из одного оператора;
4. Ключевое слово `return` не является обязательным, если тело лямбда-выражения состоит из одного оператора.

Примеры лямбда-выражений:

Вот так будет выглядеть сортировка массива строк:

```
String [] strArray = {"1", "30", "200"};
```

```
Arrays.sort(strArray, (String s1, String s2) -> s2.length() - s1.length());
```

В лямбда-выражениях могут использоваться методы класса, в котором определено выражение, а также переменные с модификатором `final` и переменные, инициализированные только раз.

Например, так выглядит доступ к экземпляру класса и его методам:

Пример 35.

```
import static java.lang.System.out;
public class Hello {
    Runnable r1 = () -> out.println(this);
    Runnable r2 = () -> out.println(toString());
    public String toString() { return "Hello, world!"; }

    public static void main(String... args) {
        new Hello().r1.run(); //Hello, world!
        new Hello().r2.run(); //Hello, world!
    }
}
```

2. Ссылки на методы

Ссылки на методы могут указывать на:

1. Статические методы;
2. Методы объектов;
3. Конструкторы.

Пример 36.

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6);
numbers.forEach(value -> System.out.println(value));
numbers.forEach(System.out::println);
```

3. Методы по умолчанию

Для реализации нового потокового API, в языке Java потребовалось ввести новую конструкцию - **методы по умолчанию (Default Methods, также известные как Defender Methods или Virtual Extension Methods)**.

Методы по умолчанию могут быть добавлены в любой интерфейс, и такие методы в интерфейсах имеют реализацию, что необычно для интерфейсов, т.к. ранее запрещалось размещать тело функции в описании интерфейса.

Допустим, класс наследует два интерфейса, и в каждом из них есть default метод. При компиляции возникнет ошибка.

Пример 37.

```
public interface A {  
    default void hello() { System.out.println("Hello World from A"); }  
}  
public interface B {  
    default void hello() { System.out.println("Hello World from B"); }  
}  
public class D implements A,B {}
```

Разберем другой пример:

Пример 38.

```
public interface A {  
    default void hello() { System.out.println("Hello World from A"); }  
}  
public interface B extends A {  
    default void hello() { System.out.println("Hello World from B"); }  
}  
public class C implements B, A {  
    public static void main(String... args) {  
        new C().hello();  
    }  
}
```

В данном случае победит ближайший в иерархии, то есть интерфейс B. Ну а если хочется использовать метод интерфейса A, то нужно:

Пример 39.

```
interface A {  
    default void hello() { System.out.println("Hello World from A"); }  
}  
interface B {  
    default void hello() { System.out.println("Hello World from B"); }  
}  
class C implements B, A {  
    public void hello() {  
        A.super.hello();  
    }  
}
```

```

        public static void main(String... args) {
            new C().hello();
        }
    }
}

```

Разберем третий пример: есть конфликтующий метод у абстрактного класса и интерфейса. В этом случае побеждает всегда класс.

Пример 40.

```

abstract class A {
    public void hello() {System.out.println("Hello World from A");}
}
interface B {
    default void hello() { System.out.println("Hello World from B"); }
}
class C extends A implements B {

    public static void main(String... args) {
        new C().hello();
    }
}

```

Таким образом, проблема решена:

- если конфликт на одном уровне иерархии — компилятор не даст ему случиться;
- если на разных уровнях — победит ближайший;
- если интерфейс против класса — побеждает класс.

В интерфейсах появилась возможность определять полноценные статические методы.

4. Новый API для потоков

Поток или стрим представляет собой последовательность элементов и поддерживает различные виды операций для выполнения вычислений:

Пример 41.

```
List<String> mList = Arrays.asList("aa1", "cc2", "cc1", "aa2", "bb1");  
mList  
    .stream()  
    .filter(s -> s.startsWith("a"))  
    .map(String::toUpperCase)  
    .sorted()  
    .forEach(System.out::println);  
//Результат выполнения:  
// AA1  
// AA2
```

Операции над потоком относятся либо к промежуточным, либо к терминальным. Все промежуточные операции возвращают поток, так что мы можем объединять несколько промежуточных операций без использования точки с запятой. Терминальные операции возвращают void или непотоковый результат, так что терминальная операция в цепочке может быть только одна. В приведенном выше примере **filter**, **map** и **sorted** являются промежуточными операциями, тогда как **forEach** является терминальной операцией.

У потоков есть некоторые особенности. Во-первых, обработка не начнется до тех пор, пока не будет вызвана терминальная операция. Во-вторых, поток после обработки нельзя переиспользовать.

Источник

of(T value)

of(T... values)

Поток для одного или нескольких перечисленных элементов. Часто используют запись:

Пример 42.

```
Arrays.asList(1,2,3).stream()  
    .forEach(System.out::println);
```

А можно:

```
Stream.of(1,2,3)  
    .forEach(System.out::println);
```

Промежуточные операции

filter (Predicate predicate) - фильтрует поток, принимая только те элементы, которые удовлетворяют заданному условию.

map (Function mapper) - применяет функцию к каждому элементу и затем возвращает поток, в котором элементами будут результаты функции. map можно применять для изменения типа элементов.

sorted()

sorted(Comparator comparator) - сортирует элементы потока. Причём

работает этот оператор следующим образом: если поток уже помечен как отсортированный, то сортировка проводиться не будет, иначе соберёт все элементы, отсортирует их и вернёт новый поток, помеченный как отсортированный.

distinct() - убирает повторяющиеся элементы и возвращаем поток с уникальными элементами. Как и в случае с `sorted`, смотрит, состоит ли уже поток из уникальных элементов и если это не так, отбирает уникальные и помечает поток как содержащий уникальные элементы.

limit(long maxSize) - ограничивает поток `maxSize` элементами.

Терминальные операции

void forEach(Consumer action) - выполняет указанное действие для каждого элемента потока.

long count() - возвращает количество элементов потока.

R collect(Collector collector) - один из самых мощных операторов Stream API. С его помощью можно собрать все элементы в список, множество или другую коллекцию, сгруппировать элементы по какому-нибудь критерию, объединить всё в строку и т.д.. В классе `java.util.stream.Collectors` очень много методов на все случаи жизни. При желании можно написать свой коллектор, реализовав интерфейс `Collector`.

Пример 43.

```
List<Integer> list = Stream.of(1,2,3)
    .collect(Collectors.toList());
// list: [1, 2, 3]
```

```
String s=Stream.of(1,2,3)
    .map(String::valueOf)
    .collect(Collectors.joining("-", "<", ">"));
// s: "<1-2-3>"
```

JavaFX

JavaFX представляет инструментарий для создания кроссплатформенных графических приложений на платформе Java.

С помощью JavaFX можно создавать программы для различных операционных систем: Windows, MacOS, Linux и для самых различных устройств: десктопы, смартфоны, планшеты, встроенные устройства, ТВ. Приложение на JavaFX будет работать везде, где установлена исполняемая среда Java (JRE).

JavaFX предоставляет большие возможности по сравнению с рядом других подобных платформ, в частности, по сравнению со Swing. Это и большой набор элементов управления, и возможности по работе с мультимедиа, двухмерной и трехмерной графикой, декларативный способ описания интерфейса с помощью языка разметки FXML, возможность стилизации интерфейса с помощью CSS, интеграция со Swing и многое другое.

История JavaFX фактически началась в первой половине 2000-х годов, когда разработчик по имени Крис Оливер (Chris Oliver), будучи работником компании SeeBeyond, разработал для создания графических интерфейсов новый язык F3 (Froms Follows Functions). Впоследствии в 2005 году SeeBeyond была приобретена компанией Sun Microsystems (которая на тот момент развивала язык Java до покупки компанией Oracle). F3 был переименован в JavaFX, а Крис Оливер продолжил работу над новой платформой уже в рамках компании Sun. И в мае 2007 года Sun Microsystems публично анонсировала новую платформу для создания графических приложений. А 4 декабря 2008 года вышел JavaFX 1.0 SDK.

После приобретения Sun Microsystems компанией Oracle в 2010 году была анонсирована, а в 2011 году вышла в релиз версия JavaFX 2.0. В первой версии JavaFX фактически представлял скриптовый язык. Во второй версии был полностью изменен подход. Скриптовый язык был убран, а платформа была полностью переписана фактически с нуля. Теперь создавать приложения можно было с помощью любого языка, который поддерживала JVM. Были добавлены новые API, интеграция со Swing и много других вещей.

Следующими важными вехами в развитии платформы стали версии JavaFX 8 и особенно JavaFX 9, которая вышла в сентябре 2017 года вместе с Java 9 и привнесла в платформу модульность. И если раньше JavaFX поставлялась вместе с Java SE, то сейчас JavaFX отделена от основной функциональности Java SE и используется как отдельный модуль. Последняя версия фреймворка - JavaFX 12 - вышла в марте 2019 года.

На данный момент JavaFX представляет предпочтительный способ для создания графических приложений с помощью языка Java, который пришел на смену AWT и Swing. Также стоит отметить, что для работы с JavaFX вместо Java теоретически можно использовать любой язык программирования, который поддерживается JVM.

Пример 44.

```
import javafx.application.Application;  
import javafx.stage.Stage;
```

```

import javafx.scene.Scene;
import javafx.scene.Group;
import javafx.scene.text.Text;
public class Main extends Application{
    public static void main(String[] args) {
        Application.launch(args);
    }
    @Override
    public void start(Stage stage) {
        // установка надписи
        Text text = new Text("Hello from JavaFX!");
        text.setLayoutY(80); // установка положения надписи по оси Y
        text.setLayoutX(100); // установка положения надписи по оси X
        Group group = new Group(text);
        Scene scene = new Scene(group);
        stage.setScene(scene);
        stage.setTitle("First Application");
        stage.setWidth(300);
        stage.setHeight(250);
        stage.show();
    }
}

```

Вкратце разберем этот класс. Главный класс приложения JavaFX должен наследоваться от класса `javafx.application.Application`. Этот класс имеет один абстрактный метод, который нам нужно реализовать:

```
public void start(Stage stage) {}
```

В этот метод в качестве параметра передается объекта **Stage**, который представляет пользовательский интерфейс. Например, на десктопах Stage будет представлять графическое окно. При запуске приложения, когда среда JavaFX будет вызывать данный метод и передавать в него объект Stage.

Что нам нужно для создания графического интерфейса? Прежде всего нам нужны различные визуальные элементы, например, кнопки, текстовые поля, списки, изображения и так далее. JavaFX предоставляет большой набор встроенных визуальных элементов. В данном случае мы используем визуальный компонент **javafx.scene.text.Text**, который представляет простую текстовую надпись. При создании элемента Text мы можем передать ему выводимый текст, а также настроить его положение с помощью его методов:

```

Text text = new Text("Hello from JavaFX!"); // создание надписи
text.setLayoutY(80); // установка положения надписи по оси Y
text.setLayoutX(100); // установка положения надписи по оси X

```

Все визуальные элементы, которые мы хотим отобразить в Stage, помещаются в объект **javafx.scene.Scene** или на сцену. Scene - это контейнер верхнего уровня для всех графических элементов. Однако напрямую в Scene объект Text мы поместить не можем. Класс Scene предусматривает установку

корневого элемента или контейнера, который содержит все остальные элементы.

Таким образом, вначале элемент Text помещается в элемент Group, который представляет контейнер для группы элементов. Затем элемент Group устанавливается в качестве корневого элемента Scene. И в конце элемент Scene устанавливается для объекта Stage.

```
Group group = new Group(text); // создание и установка группы элементов
Scene scene = new Scene(group); // создание сцены
stage.setScene(scene); // установка сцены для объекта Stage
```

В конце метода start мы можем настроить объект Stage, например, задать заголовок окна, а также его размеры:

```
stage.setTitle("First Application"); // установка заголовка окна
stage.setWidth(300); // установка ширины окна
stage.setHeight(250); // установка длины окна
stage.show(); // отображаем окно на экране устройства
```

С помощью метода show объект Stage отображается на экране устройства.

Но класс Main, как и всякий главный класс приложения Java, начинает свою работу с метода main. И в методе, чтобы запустить само приложение JavaFX, представленное классом Application, вызывается метод launch():

```
public static void main(String[] args) {
    Application.launch(args);
}
```

Метод launch собственно и запускает приложение JavaFX. После этого вызывается метод start и окно приложения будет отображаться на экране.

Приложение JavaFX по сути представляет класс, который унаследован от класса javafx.application.Application. Объект этого класса проходит ряд этапов жизненного цикла. Все эти этапы представлены методами класса Application, которые вызываются автоматически средой JavaFX:

init(): инициализирует приложение до его запуска. Стоит отметить, что метод не должен использоваться для создания графического интерфейса или отдельных его частей.

start(Stage stage): здесь собственно и определяется графический интерфейс.

stop(): вызывается после закрытия приложения, например, после того, как пользователь нажал на крестик в правом верхнем углу

В итоге весь процесс работы приложения выглядит следующим образом:

1. Запускает исполняемая среда JavaFX (если она не запущена).
2. Она вызывает конструктор класса, который расширяет класс Application, тем самым создавая экземпляр данного класса.
3. Затем среда JavaFX вызывает метод init().
4. Вызывается метод start(javafx.stage.Stage), в который среда JavaFX передает созданный объект Stage. Таким образом, приложение начинает работать.

5. Далее среда ожидает, пока либо в приложении не будет вызван программным способом метод `Platform.exit()`, либо пока не будет закрыто последнее окно программы.
6. После завершения работы приложения среда JavaFX вызывает метод `stop()`.

Все эти методы мы можем переопределить в главном классе приложения. Но, как правило, определяется один метод `start()`, в котором определяются настройки графического интерфейса. Но, к примеру, переопределим все методы:

Пример 45.

```
import javafx.application.Application;
import javafx.stage.Stage;
import java.util.List;
public class Main extends Application{
    public static void main(String[] args) {
        System.out.println("Launching Application");
        Application.launch(args);
    }
    @Override
    public void init() throws Exception {
        System.out.println("Application inits");
        super.init();
    }
    @Override
    public void start(Stage stage) {
        System.out.println("Application starts");
        // получаем переданные параметры
        Application.Parameters params = getParameters();
        List<String> unnamedParams = getParameters().getUnnamed();
        int i = 0;
        for(String param: unnamedParams){
            i++;
            System.out.printf("%d - %s \n", i, param);
        }
        stage.show();
    }
    @Override
    public void stop() throws Exception {
        System.out.println("Application stops");
        super.stop();
    }
}
```

В данном случае просто выводим на консоль индикацию о вызове метода.

Фактически работа любой программы на Java начинается с метода `main`, который является входной точкой в приложение. В этом методе вызывается статический метод `Application.launch(args)`. При запуске в метод `launch` мы можем передать аргументы в виде массива строк. Например, мы можем взять те значения, которые получает метод `main` через командную строку/терминал. Затем эти аргументы мы можем получить через `getParameters`, например, в методе `start`.

Затем среда создает объект класса `Main`, вызывает у него последовательно методы `init` и `start`. После вызова в методе `start` метод `stage.show()` нам отобразится окно приложения.

Заккрытие окна приведет к вызову метода `stop`. И таким образом приложение завершит свое выполнение.

Стоит отметить, что ни метод `init`, ни конструктор класса, который наследуется от `Application`, не должен использоваться для создания графического интерфейса. Потому что и метод `init`, и конструктор запускаются в потоке, который называется потоком запуска или `launcher thread`.

А создание, а также изменение графического интерфейса должно производиться в потоке приложения или `application thread`. Именно в таком потоке и запускается метод `start` (а также метод `stop`).

Основой для создания графического интерфейса в JavaFX является класс **`javafx.stage.Stage`**. По сути он является контейнером, в который помещаются все остальные компоненты интерфейса. Его конкретная реализация зависит от платформы, на которой запускается приложение. Так, на десктопах это будет отдельное графическое окно, а на мобильных устройствах интерфейс может представлять весь экран устройства.

В программе на JavaFX можно использовать множество объектов `Stage`, но один из них является основным. При запуске приложения основной объект `Stage` создается средой JavaFX и передается в метод **`start()`**:

`@Override`

```
public void start(Stage primaryStage) {}
```

Этот объект `Stage` определяет главное окно или экран программы. Однако внутри приложения мы также можем создавать другие объекты `Stage` - другие графические окна.

Класс `Stage` унаследован от класса **`javafx.stage.Window`**, который определяет базовые возможности окна приложения.

`Stage` позволяет управлять позиционированием, размерами и некоторыми другими настройками окна приложения. Рассмотрим некоторые основные методы класса `Stage`:

- **`close()`**: закрывает объект `Stage` (на десктопах по сути закрывает окно)
- **`hide()`**: скрывает окно
- **`centerOnScreen()`**: располагает окно в центре экрана
- **`toBack()`**: перемещает окно на задний план
- **`toFront()`**: перемещает окно на передний план
- и т.д.

Пример 46

```
import javafx.application.Application;
import javafx.stage.Stage;

public class Main extends Application{

    public static void main(String[] args) {

        Application.launch(args);
    }

    @Override
    public void start(Stage stage) {

        stage.setTitle("Hello JavaFX"); // установка заголовка
        stage.setWidth(250);           // установка ширины
        stage.setHeight(150);          // установка длины
        stage.show();                   // отображение окна на экране
    }
}
```

Класс **javafx.scene.Scene** представляет контейнер для всех графических элементов внутри объекта Stage в виде графа, который называется **Scene Graph**. Все узлы этого графа, то есть по сути все вложенные элементы должны представлять класс **javafx.scene.Node**. Но корневой узел этого графа должен представлять объект класса, который унаследован от **javafx.scene.Parent**. По сути Parent - это контейнер, который может содержать другие элементы.

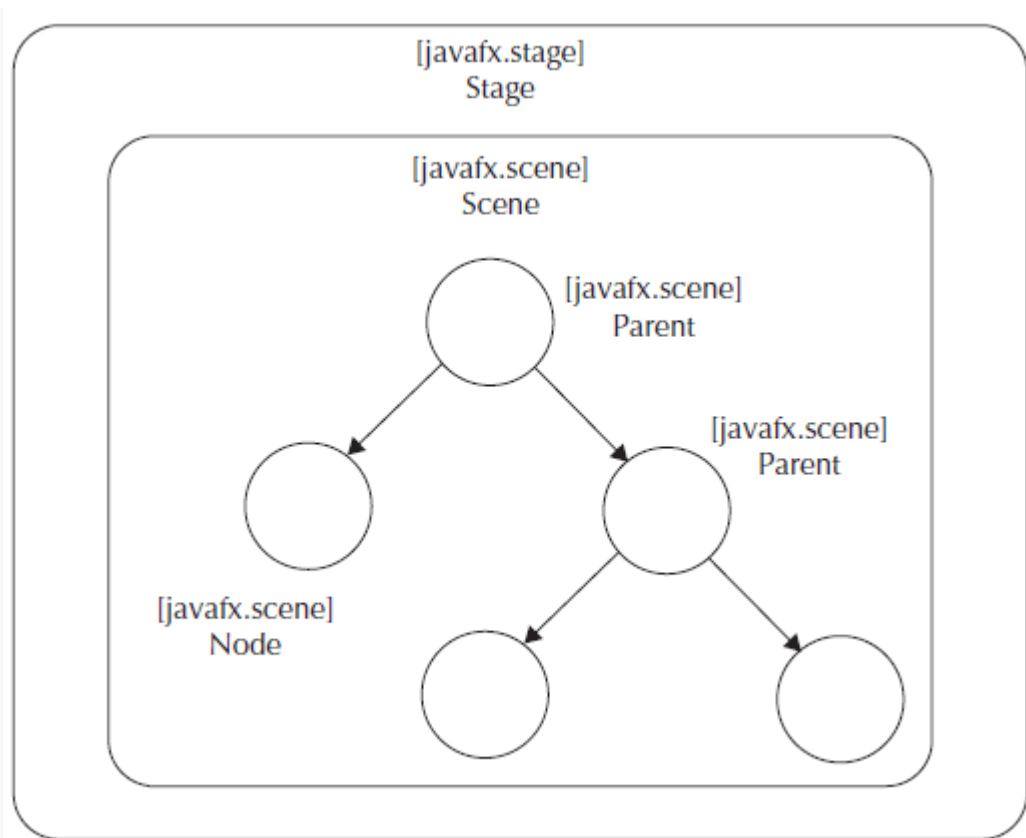


Рисунок 17

Для установки корневого узла в Scene применяется один из конструкторов объекта Scene. Основные из них:

- `Scene(Parent root)`: создает Scene с корневым узлом `root`
- `Scene(Parent root, double width, double height)`: создает Scene с корневым узлом `root`, с шириной `width` и высотой `height`
- `Scene(Parent root, Paint fill)`: создает Scene с корневым узлом `root` и устанавливает фоновый цвет
- `Scene(Parent root, double width, double height, Paint fill)`: создает Scene с корневым узлом `root`, с шириной `width` и высотой `height` и устанавливает фоновый цвет

При этом все конструкторы принимают в качестве первого параметра корневой узел. То есть при создании объекта Scene нам та или иначе придется установить и корневой узел.

Пример 47

```
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.Group;
import javafx.scene.layout.FlowPane;
import javafx.scene.control.Button;
```

```

import javafx.scene.control.Label;

public class Main extends Application{

    public static void main(String[] args) {

        Application.launch(args);
    }

    @Override
    public void start(Stage stage) {

        Label label = new Label("Hello");           // текстовая метка
        Button button = new Button("Button");        // кнопка
        Group group = new Group(button);             // вложенный узел Group

        FlowPane root = new FlowPane(label, group);  // корневой узел
        Scene scene = new Scene(root, 300, 150);     // создание Scene
        stage.setScene(scene);                       // установка Scene для Stage

        stage.setTitle("Hello JavaFX");

        stage.show();
    }
}

```

Здесь в качестве корневого узла выступает класс *FlowPane*, который располагает вложенные элементы подряд и который, как и *Group*, унаследован от класса *Parent*. *FlowPane* содержит объект *Label* (текстовая метка) и объект *Group*. А объект *Group*, в свою очередь, содержит объект *Button* (кнопка).

Визуально это будет выглядеть следующим образом:

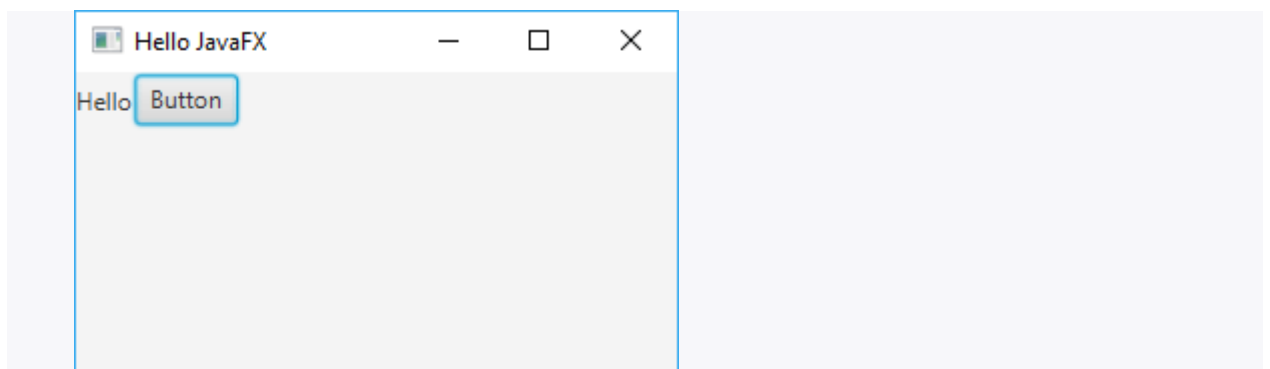


Рисунок 18

А сам *Scene Graph* в данном случае будет представлять следующую структуру:

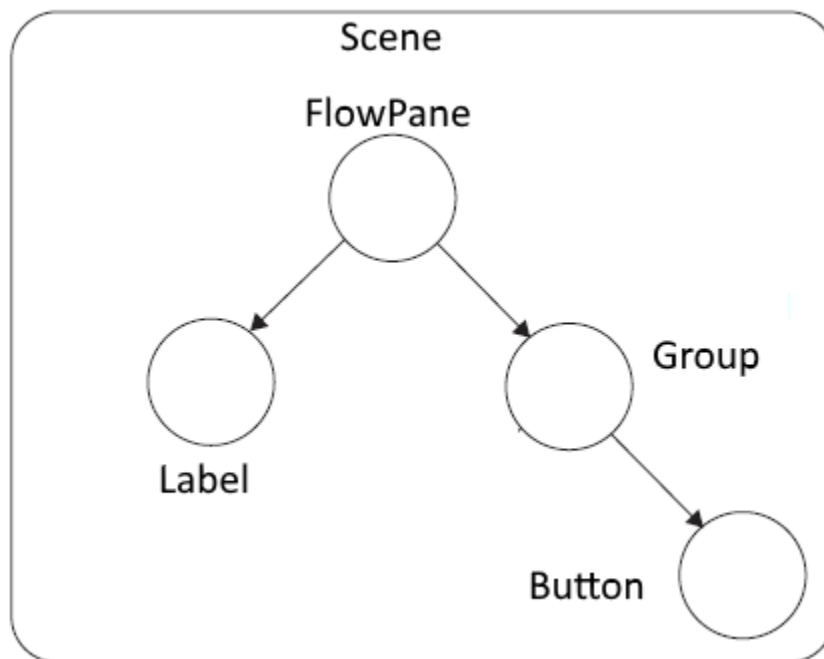


Рисунок 19

Все графические элементы, которые используются в объекте Scene и добавляются в Scene Graph, должны представлять класс **javafx.scene.Node** или иначе узел. Все встроенные классы визуальных графических элементов или узлы, например, кнопки, текстовые поля и другие, наследуются от класса Node.

При этом одни узлы Node могут содержать несколько других узлов Node. Например, класс Parent наследуется от Node, но при этом сам может содержать другие узлы Node.

Базовую иерархию классов можно представить следующим образом:

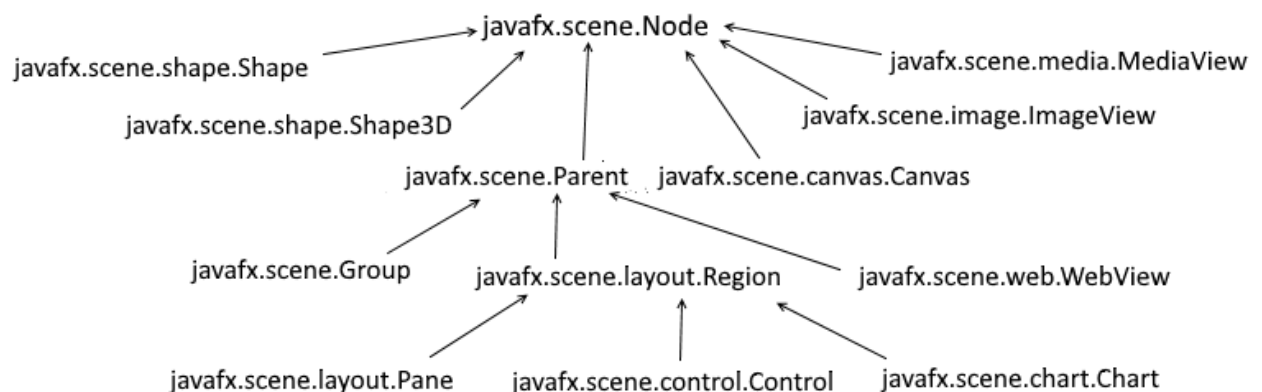


Рисунок 20

Вкратце рассмотрим эту иерархию. Основные классы, которые наследуются от класса Node:

- **javafx.scene.shape.Shape:** является базовым классом для создания геометрических двумерных примитивов (например, линия, прямоугольник, эллипс)
- **javafx.scene.shape.Shape3D:** является базовым классом для создания трехмерных объектов

- **javafx.scene.canvas.Canvas:** представляет полотно для отрисовки различного содержимого
- **javafx.scene.image.ImageView:** элемент для отображения изображений
- **javafx.scene.media.MediaView:** элемент для работы с мультимедиа
- **javafx.embed.swing.SwingNode:** элемент для встраивания содержимого Swing в JavaFX
- **javafx.scene.Parent:** базовый класс для всех элементов, которые могут содержать другие элементы

Класс Parent

В данном случае следует выделить класс `javafx.scene.Parent`, который представляет функциональность для управления вложенными узлами, их добавления и удаления и прочие операции с ними. От него наследуются следующие классы:

- **javafx.scene.web.WebView:** элемент, который позволяет отображать веб-содержимое.
- **javafx.scene.Group:** представляет контейнер для группы объектов
- **javafx.scene.layout.Region:** базовый класс для всех элементов управления, панелей компоновки и диаграмм. Его отличительная особенность состоит в том, что он добавляет функциональность управления границами и размерами элементов.

Класс Region

Здесь выделим класс `javafx.scene.layout.Region`, который определяет свойства и методы для управления и получения ширины, высоты элемента, для управления его отступами, границами, позиционированием.

Соответственно все классы-наследники смогут использовать эти методы.

От него происходят три больших группы узлов:

- **javafx.scene.chart.Chart:** базовый класс для всех диаграмм.
- **javafx.scene.layout.Pane:** базовый класс для всех панелей компоновки, которые позволяют управлять расположением вложенных элементов, определенным образом упорядочивать их.
- **javafx.scene.control.Control:** базовый класс для всех элементов управления (кнопок, текстовых полей ввода, списков и т.д.).

Для взаимодействия с пользователем в JavaFX используется событийная модель. В этой модели есть источник события - некоторый элемент управления, который генерирует событие, и есть один или несколько слушателей или обработчиков события, которые подписываются на событие. Когда элемент управления генерирует событие, то обработчик обрабатывает это событие.

Рассмотрим небольшой базовый пример по обработке событий:

Пример 48

```

import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.Group;
import javafx.scene.control.Button;
import javafx.event.ActionEvent;
import javafx.event.EventHandler;

public class Main extends Application{

    public static void main(String[] args) {

        Application.launch(args);
    }

    @Override
    public void start(Stage stage) {

        Button btn = new Button();
        btn.setText("Click!");

        btn.setOnAction(new EventHandler<ActionEvent>() {

            @Override
            public void handle(ActionEvent event) {

                btn.setText("You've clicked!");
            }
        });

        Group root = new Group(btn);
        Scene scene = new Scene(root);
        stage.setScene(scene);

        stage.setTitle("Hello JavaFX");
        stage.setWidth(250);
        stage.setHeight(200);

        stage.show();
    }
}

```

Здесь мы создаем кнопку, которая представлена объектом классом Button. При нажатии на кнопку возникает событие нажатия, которое мы можем обработать.

Базовым классом для всех событий является класс **javafx.event.Event**, который унаследован от класса **java.util.EventObject**. При генерации события, например, при нажатии на кнопку, создается объект Event, через который передается информация о событии. В данном случае при нажатии на кнопку будет генерироваться событие типа **javafx.event.ActionEvent** (который наследуется от Event).

Для определения обработчика события используется функциональный интерфейс **EventHandler**.

```
public interface EventHandler<T extends Event>{  
    void handle(T event);  
}
```

Интерфейс EventHandler типизируется типом, который унаследован от класса Event и который по сути представляет событие. В нашем случае это класс ActionEvent.

Для прикрепления обработчика события EventHandler к событию элемента управления применяется метод btn.setOnAction(), в который передается реализация интерфейса EventHandler. В методе handle определяются действия, которые будут вызываться при нажатии на кнопку:

```
btn.setOnAction(new EventHandler<ActionEvent>() {
```

```
    @Override  
    public void handle(ActionEvent event) {
```

```
        btn.setText("You've clicked!");
```

```
    }
```

```
});
```

В самом обработчике просто меняется текст кнопки.

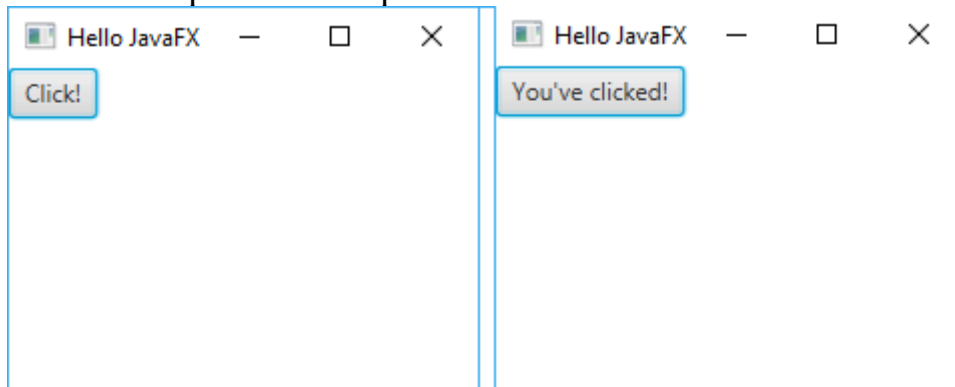


Рисунок 21

Нередко вместо определения явной реализации интерфейса EventHandler применяются лямбда-выражения. Например, мы могли бы переписать метод btn.setOnAction() следующим образом:

```
btn.setOnAction(e -> {
```

```
    btn.setText("You've clicked!");
```

```
});
```

Web-приложения и их особенности

Распределенная система – это собрание независимых компьютеров (узлов), соединенных сетью и программным обеспечением, обеспечивающим их совместное функционирование.

Причины создания распределенных приложений

1. **Необходимость совместного использования общих ресурсов** (данных, устройств, приложений).
2. **Улучшение функциональных характеристик системы** (производительности, повышения надежности и устойчивости к сбоям, упрощение и удешевление за счет специализации компонентов, отношения цена/производительность).

Принципы построения

1. **Функциональное разделение.** Узлы выполняют различные задачи (клиент / сервер, хост / терминал, сбор данных / обработка данных). Решение – создание разделяемых сервисов, которые управляют набором ресурсов и предоставляют услуги пользователям.
2. **Естественное разделение.** Разделение определяется задачей (система обслуживания сети супермаркетов, сеть для обеспечения коллективной работы). Функциональная нагрузка одинаковая у однотипных узлов.
3. **Балансировка нагрузки.** Функциональность может повторяться. Задачи назначаются на процессоры таким образом, чтобы нагрузка была равномерной. Такой подход обеспечивает наибольшую эффективность использования вычислительных ресурсов.

Последствия распределенности

1. **Параллельность.** Параллельное выполнение (вопросы синхронизации, «гонки потоков»), совместное использование ресурсов (коллизии при доступе к данным, сервисам, устройствам), типичные проблемы (взаимные блокировки, ненадежные коммуникации).

2. **Нет “глобального” времени** (асинхронная передача сообщений, ограниченная точность синхронизации часов).
3. **Нет состояния системы.** В распределенной системе нет ни одного процесса, который бы знал текущее глобальное состояние системы. Это следствие параллелизма и механизма передачи данных.
4. **Сбои.** Процессы выполняются автономно, изолированно. Неудачи отдельных процессов могут остаться необнаруженными. Отдельные процессы могут не подозревать об общесистемном сбое. Сбои происходят чаще, чем в централизованной системе. Новые причины сбоев (которых не было в монолитных системах). Сетевые сбои изолируют процессы и фрагментируют систему на изолированные части.

Требования к распределенным системам

1. Открытость.
 2. Безопасность.
 3. Масштабируемость.
 4. Механизмы обработки ошибок и восстановления после сбоев.
 5. Методы решения проблем параллелизма.
 6. Прозрачность.
 7. Управляемость.
1. **Открытость.** Гарантирует расширяемость, возможность повторного использования, использование открытых протоколов и стандартов. Важными факторами являются наличие четких спецификаций и полной документации, опубликованные интерфейсы, тестирование и проверка на многих платформах.
 2. **Безопасность.** Физическая распределенность означает возможность доступа злоумышленников к компонентам. Необходимо обеспечить посылку важной информации по сети безопасно и эффективно. Это достигается, например, с помощью аутентификации (процедуры проверки подлинности данных) и авторизации (предоставление определённого лицу или группе лиц прав на выполнение определённых действий; а также

процесс проверки данных прав при попытке выполнения этих действий), шифрования данных.

3. **Масштабируемость.** Распределенная система масштабируема, если она остается эффективной при увеличении числа обслуживаемых пользователей или ресурсов. Возникает проблема стоимости физических ресурсов (растет при увеличении числа пользователей) и потери производительности (увеличиваются с ростом размера данных и количества пользователей). Для «обхода узких мест» используется децентрализация алгоритмов (Пример – Domain Name Service (DNS)), тиражирование и кэширование данных.
4. **Обработка сбоев.** Сбои возникают чаще, чем в централизованных системах, но обычно носят локальный характер. Обработка сбоев включает в себя: диагностику (например, ошибки передачи могут быть обнаружены с помощью контрольных сумм. Однако, в ряде случаев, может быть невозможна. Например, невозможно определить, удаленный сервер не работает или просто очень загружен?), маскирование (многие сбои могут быть скрыты, однако, в ряде случаев маскирование может быть невозможно или не всегда приводит к правильным последствиям) и восстановление.
5. **Прозрачность.** Это сокрытие гетерогенной (разнородной) и распределенной структуры системы таким образом, чтобы пользователю система представлялась монолитной. Степень прозрачности взаимосвязана с производительностью. Различают:
 - **Прозрачность** **доступа.**
Скрывается разница в представлении данных и доступе к ресурсам.
 - **Прозрачность** **расположения.**
Скрывается местоположения ресурса.
 - **Прозрачность** **переноса.**
Скрывается факт перемещения ресурса в другое место.

■ **Прозрачность** смены местоположения.

Скрывается факт перемещения ресурса в другое место в процессе обработки.

■ **Прозрачность** репликации.

Скрывается факт и особенности репликации (механизм синхронизации содержимого нескольких копий объекта (например, содержимого базы данных)) ресурса.

■ **Прозрачность** параллелизма.

Возможность нескольким процессам параллельно работать с ресурсами, не оказывая влияния друг на друга.

■ **Прозрачность** обработки ошибок.

Защита программных компонентов от сбоев, произошедших в других программных компонентах; восстановление после сбоев.

Критически важными являются:

- Прозрачность доступа.
- Прозрачность расположения.
- Прозрачность параллелизма.

6. **Управляемость.** Распределенные ресурсы не имеют центральной точки управления. Локальная оптимизация не всегда означает глобальную оптимизацию (нужно учитывать гетерогенность и другие особенности, нужен глобальный взгляд на проблему, он не всегда возможен (есть системы, никому конкретно не принадлежащие)).

Сложности при реализации

1. **Выбор архитектуры.** Важнейшие характеристики системы очень сильно зависят от выбранной архитектуры. Практически невозможно отказаться от принятых в начале проектирования неверных решений.
Решение: использование широко известных шаблонов (образцов) архитектур.
2. **Гетерогенность среды (разнородность).** Различаются:
 - Сетевые инфраструктуры.

- Оборудование (например, Intel и Motorola).
- Программное обеспечение (например, UNIX sockets и Winsock calls).
- Языки программирования.
- Представления данных.

Различные компоненты системы выполняются на различных платформах. Различия должны быть скрыты. Интерфейсы и реализация могут быть разными, но базовые концепции обычно неизменны. Средства борьбы с гетерогенностью – стандарты.

Решения:

- Использование распространенных открытых стандартов и протоколов.
- Использование промежуточного программного обеспечения (middleware).

Middleware: промежуточный программный слой:

- Позволяет гетерогенным узлам взаимодействовать.
- Определяет однородную вычислительную модель.
- Поддерживает один или несколько языков программирования.
- Обеспечивает поддержку распределенных приложений (вызов удаленных объектов, удаленный вызов SQL, распределенную обработку транзакций).

Примеры: CORBA, JavaEE, Microsoft DCOM.

3. Сложность развертывания.

- Фрагментация (разделение приложения на модули развертывания).
- Конфигурация (связь модулей друг с другом (зависимости)).
- Размещение (выгрузка модулей в целевую систему, распределение вычислительных модулей между узлами (статическое или динамическое), использование специализированных инструментальных средств).

4. **Сложность отладки.** Нет глобального состояния. Параллельность приводит к неповторяемости (невоспроизводимости) результатов выполнения. Компоненты распределены по разным узлам.

Модели распределенных систем

1. **Модели архитектуры.** Модель архитектуры распределенной системы должна содержать решение двух проблем: физическое размещение компонентов между узлами и взаимодействие между компонентами. Реальные функции отдельных компонентов не указываются, указываются:
- Расположение (размещение по узлам).
 - Шаблоны распределения данных и задач по их обработке.
 - Взаимодействие компонентов (роли компонентов, шаблоны взаимодействия).
2. **Другие модели.** Формальное описание различных параметров и свойств системы, модели взаимодействия, обработки ошибок, безопасности, и т.д.

Архитектура

- Определяет разделение системы на наиболее крупные составные части.
- Определяет конструктивные решения, которые после их принятия с трудом поддаются изменению.
- Отображает общий взгляд разработчиков на результаты проектирования системы: идентификация главных компонентов системы, способов их взаимодействия, выбор основополагающих решений, не подлежащих изменению в будущем.

Основные принципы архитектуры

- **Согласованность.** Частичное знание системы позволяет предсказать остальное.
- **Ортогональность.** Функции независимы и специфицированы по отдельности.
- **Соответствие.** Включаются только функции, соответствующие существенным требованиям к системе, нет ненужных функций.
- **Экономичность.** Отсутствие дублирования.

- **Прозрачность.** Функции должны быть известны пользователю.
- **Общность.** Если функция должна быть введена, ее следует вводить в таком виде, чтобы она отвечала как можно большему числу назначений.
- **Открытость.** Можно использовать функцию иначе, чем это предполагалось при проектировании.
- **Полнота.** Введенные функции должны с учетом экономических и технологических ограничений как можно полнее соответствовать требованиям и пожеланиям пользователя.

Преодоление сложности:

1. Использование типовых решений - образцов проектирования.

Каждое типовое решение описывает некую повторяющуюся проблему и ключ к ее разгадке, причем таким образом, что вы можете пользоваться этим ключом многократно, ни разу не придя к одному и тому же результату.

Структура типовых решений

- Название решения.
 - Назначение (аннотация).
 - Мотивация, применимость.
 - Принцип действия (структура, участники, отношения).
 - Результаты.
 - Реализация.
- 2. Разделение системы на слои (расслоение).** Основная идея: независимость нижележащих уровней от вышележащих. Основная задача: уменьшать сложность систем, разделяя их на слои и сервисы:
- **Слой (уровень):** группа сильно связанных и закрытых элементов, реализующих одну функциональность.
 - **Сервис:** функциональность, обеспечиваемая для вышестоящего слоя.

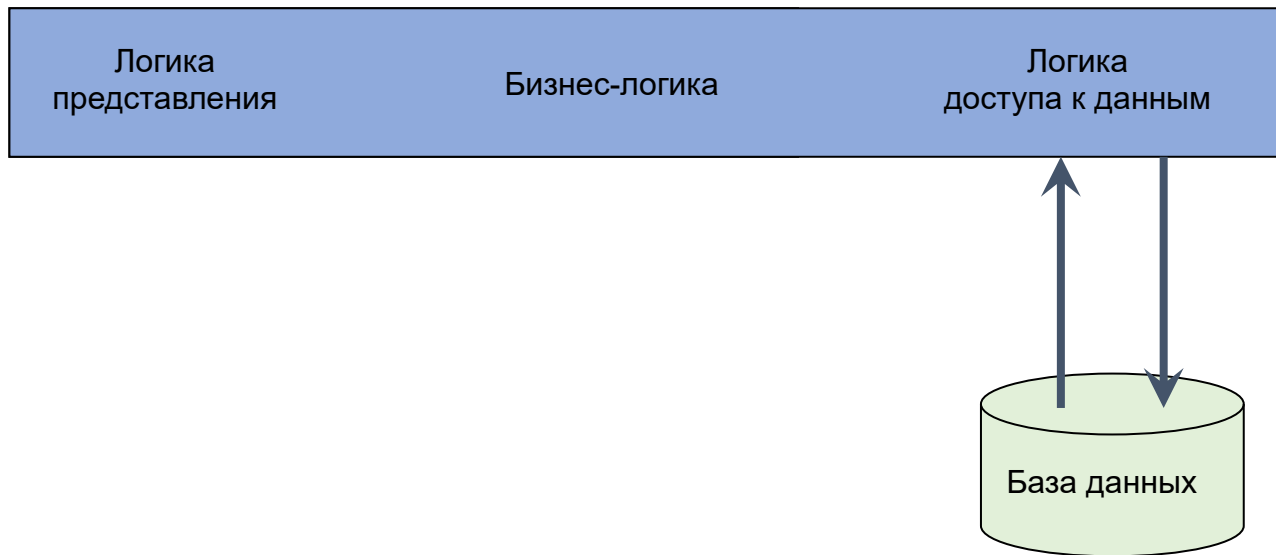


Рисунок 22 – Монолитная система

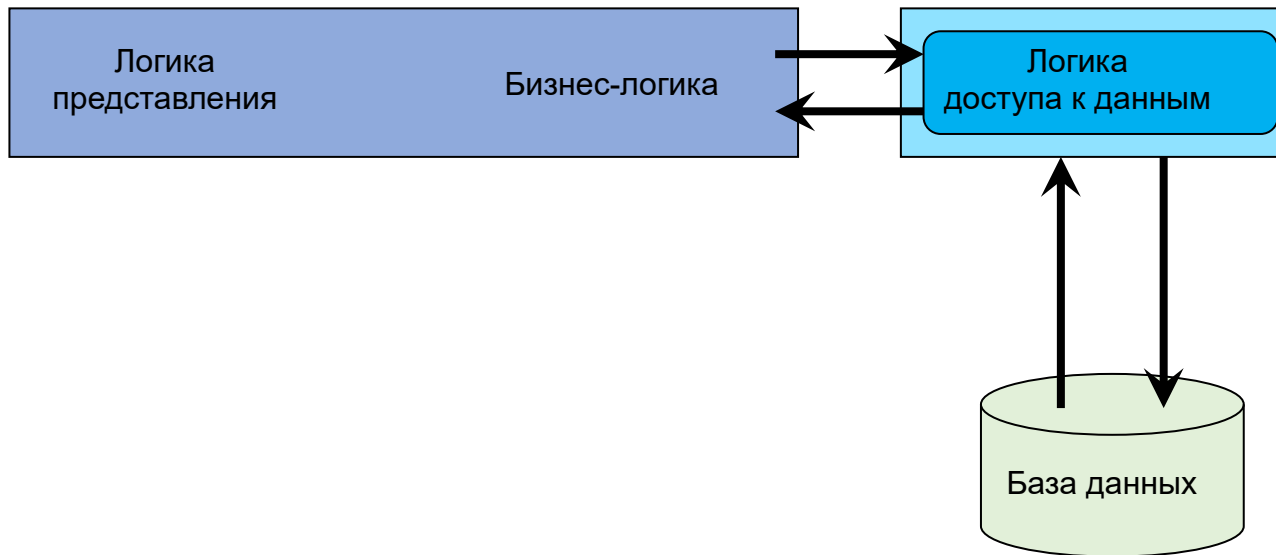


Рисунок 23 – Двухслойная система

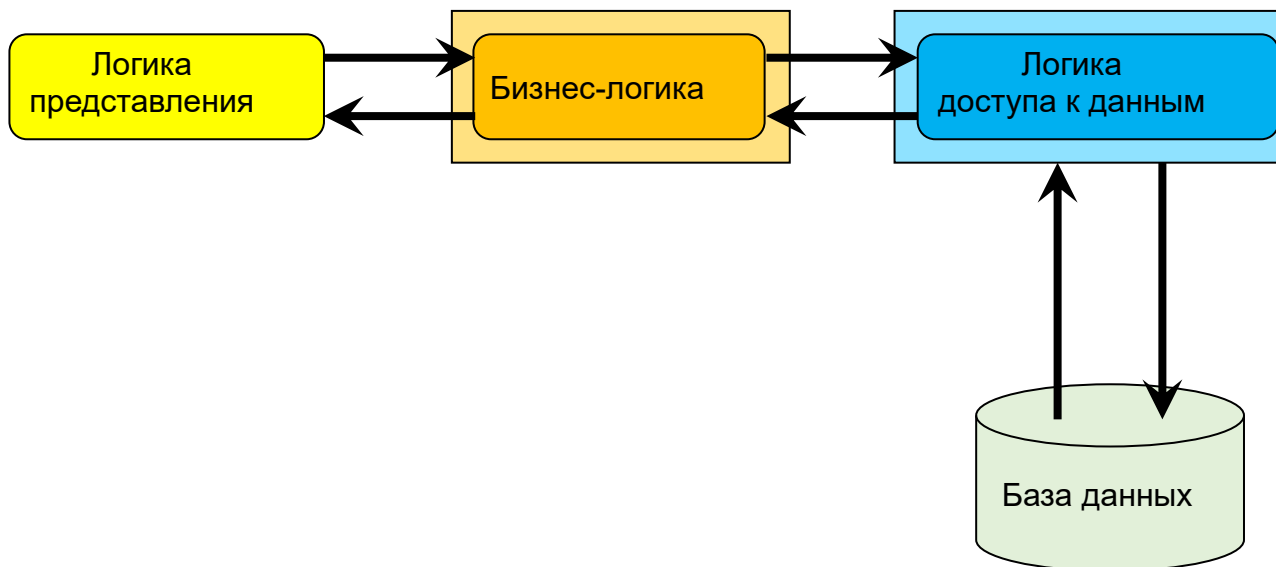


Рисунок 24 – Трехслойная система

Три основных слоя

■ Слои:

- представление (presentation).
- домен (domen) – предметная область, бизнес-логика.
- работа с данными (data source).

■ Рекомендуется различать:

- **Слой** (layer) – логическое разделение.
- **Уровень** или **Ярус** (tier) – физическое разделение.

Итоги

- Распределенная система – это автономные (но соединенные средой передачи данных) узлы. Взаимодействие осуществляется посредством передачи сообщений.
- Много доводов в пользу того, что распределенные системы нужны и их нужно уметь строить.
- Распределенные системы существуют и их нужно уметь развивать и поддерживать.
- При разработке распределенных систем возникают специфические проблемы.
- Используется модель слоев для снижения сложности системы. Middleware обеспечивает дополнительное удобство и дополнительные сервисы.
- Выбор модели архитектуры зависит от особенностей задачи.
 - Клиент – сервер.
 - Модель предоставления услуг пулом серверов.
 - Модель прокси- и кэш-серверов.
 - Модель равных процессов.

JDBC

В 1996 году компания Sun выпустила первую версию интерфейса для организации доступа Java – приложений к базам данных (БД) JDBC (Java DataBase Connectivity). Настоящий интерфейс позволяет программистам соединяться с БД, запрашивать и обновлять данные с помощью языка SQL (Structured Query Language), который фактически стал стандартным средством взаимодействия с реляционными базами данных.

Java и JDBC имеют весомое преимущество по сравнению с другими инструментами для работы с БД. Программы, созданные с помощью Java и JDBC, не зависят от используемой платформы и поставщика программного обеспечения.

Средства JDBC неоднократно обновлялись. В состав JDK 1.2, выпущенной в 1998 году, была включена версия 2 JDBC. В JDK 1.4 и 1.5 была включена версия 3 JDBC и т.д.

С самого начала поставщики программного обеспечения для БД поддерживали идею создания компанией Sun стандартного сетевого протокола доступа к БД, но при условии, что за основу будет принят их собственный протокол. В конечном итоге поставщики СУБД и инструментов доступа к базам всё-таки согласились с тем, что сотрудники Sun создали Java API для SQL-доступа к данным и диспетчер драйверов, который позволил бы подключать к базам драйверы независимых производителей. Такой подход позволял производителям СУБД создавать собственные драйверы, которые подключались бы с помощью диспетчера и должны лишь соответствовать требованиям API диспетчера драйверов.

В результате было создано два интерфейса. Разработчики приложений используют JDBC API, а поставщики БД и инструментальных средств – JDBC Driver API. Подход, используемый при создании JDBC, основан на модели ODBC (Open Database Connectivity) — интерфейса доступа к БД, разработанного фирмой Microsoft: программы, соответствующие API, могут

взаимодействовать с диспетчером драйверов JDBC, который, в свою очередь, использует присоединенные драйверы для организации взаимодействия с БД.

Типы JDBC – драйверов:

1. **Драйвер типа 1.** Транслирует JDBC в ODBC и для взаимодействия с БД использует драйвер ODBC. Sun включила в состав JDK один такой драйвер – мост JDBC/ODBC. Но для его использования требуется соответствующим образом установить и конфигурировать ODBC драйвер. В первом выпуске JDBC этот мост предполагалось использовать только для тестирования, а не для рабочего применения. В настоящее время имеется большое количество более удачных драйверов.
2. **Драйвер типа 2.** Создается преимущественно на Java и частично на собственном языке программирования, который используется для взаимодействия с клиентским API БД. Для использования такого драйвера нужно помимо библиотеки Java установить специфический для данной платформы код.
3. **Драйвер типа 3.** Создается только на основе библиотеки Java, в которой используется независимый от БД протокол взаимодействия сервера и базы. Этот протокол позволяет транслировать запросы в соответствии со спецификой конкретной базы. Если код, зависящий от БД, находится только на сервере, доставка программ существенно упрощается.
4. **Драйвер типа 4.** Представляет собой библиотеку Java, которая транслирует JDBC-запросы непосредственно в протокол конкретной БД.

Большинство поставщиков БД применяют драйверы типа 3 или 4. Кроме того, некоторые компании специализируются на создании драйверов, которые позволяют добиться более полного соответствия принятым стандартам, поддерживают большее количество платформ, обладают более высокой

производительностью или надежностью, чем драйверы, предлагаемые производителями СУБД.

Основные цели интерфейса JDBC можно сформулировать следующим образом:

1) Разработчики создают программы на языке Java, пользуясь для доступа к БД стандартными средствами языка SQL, при этом они следуют только соглашениям языка Java.

2) Производители СУБД и инструментальных средств поставляют драйверы только низкого уровня.

Примеры использования JDBC

Согласно традиционной модели клиент/сервер, графический пользовательский интерфейс реализуется на стороне клиента, а БД располагается на стороне сервера. В этом случае драйвер JDBC находится на стороне клиента (рис.25).



Рисунок 25 – Традиционная структура приложения клиент/сервер

Однако современная тенденция развития программного обеспечения заключается в переходе от архитектуры клиент/сервер к «трехуровневой модели» (рис.26).

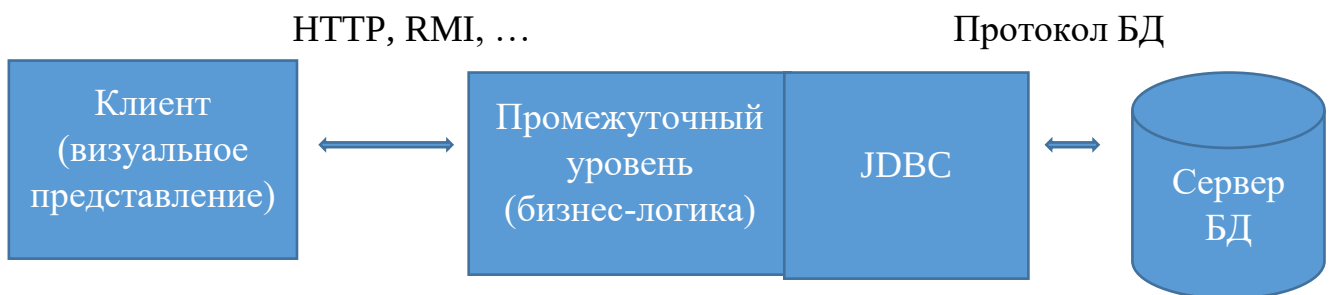


Рисунок 26 – Приложение, созданное в соответствии с трехуровневой моделью

В ней клиент не формирует обращений к БД. Вместо этого он обращается к средствам промежуточного уровня на сервере, который, в свою

очередь, выполняет запросы к БД. Трехуровневая модель отделяет визуальное представление (на компьютере-клиенте) от бизнес-логики (промежуточный уровень) и данных (в базе данных). Таким образом, становится возможным доступ к тем же данным и тем же бизнес-правилам посредством клиентов различных типов, например, Java-приложений, апплетов и Web-форм.

Взаимодействие между клиентом и промежуточным уровнем может быть реализовано на основе протокола HTTP (при использовании Web-браузера в качестве клиента), средств RMI (при использовании приложений или апплетов) или с помощью какого-либо другого механизма. JDBC используется для управления взаимодействием между промежуточным уровнем и БД.

Основы программирования JDBC

Классы, используемые для программирования JDBC, содержатся в пакетах `java.sql` и `javax.sql`.

Класс *DriverManager* отвечает за выбор драйверов БД и создание нового соединения с БД. Однако перед тем, как диспетчер активизирует драйвер, его надо зарегистрировать. Имя драйвера можно задать в командной строке:

```
java -Djdbc.drivers=org.postgresql.Driver MyClass
```

Кроме того, приложение может установить системное свойство с помощью вызова:

```
System.setProperty("jdbc.drivers", "org.postgresql.Driver");
```

Драйвер также можно зарегистрировать вручную, загрузив его класс:

```
Class.forName("org.postgresql.Driver");
```

Такой подход применяют, если диспетчер не может загрузить драйвер из-за ограничений конкретного драйвера либо в том случае, когда программа выполняется в среде контейнера, например, сервлета.

После регистрации драйвера можно установить соединение с БД. При работе с JDBC база данных представляется с помощью URL (Uniform Resource Locator). Для PostgreSQL он принимает одну из следующих форм:

- `jdbc:postgresql:database`

- `jdbc:postgresql://host/database`
- `jdbc:postgresql://host:port/database`

Для подключения к базе нужно использовать класс `Connection`:

```
Connection connection = DriverManager.getConnection(url, username, password);
```

Например:

```
Connection connection = DriverManager.getConnection("jdbc:postgresql://localhost:5432/MyTest", "postgres", "root");
```

Для указания драйвера, URL, имени пользователя и пароля удобно применять файл свойств:

Пример 49

```
jdbc.drivers = org.postgresql.Driver
jdbc.url = jdbc:postgresql://localhost:5432/MyTest
jdbc.username = postgres
jdbc.password = root

//Далее читаем файл свойств и устанавливаем соединение с БД:
Properties props = new Properties();
FileInputStream in = new FileInputStream("database.properties");
props.load(in);
in.close();

String drivers = props.getProperty("jdbc.drivers");
if(drivers != null) System.setProperty("jdbc.drivers", drivers);
String url = props.getProperty("jdbc.url");
String username = props.getProperty("jdbc.username");
String password = props.getProperty("jdbc.password");
Connection connection = DriverManager.getConnection(url, username, password);
```

Метод `getConnection()` возвращает объект `Connection`, с помощью которого создаётся объект `Statement`, который используется для выполнения

SQL – команд. Метод *executeQuery()* возвращает объект *ResultSet*, который можно использовать для построчного просмотра результатов.

Пример 50

//Извлечение данных:

```
Statement statement = connection.createStatement();
ResultSet resultSet = statement.executeQuery("SELECT * FROM item");
while (resultSet.next()) {
    String str = resultSet.getString("name");
    System.out.println(str);
}
```

В классе *ResultSet* итератор располагается в начале работы перед первой строкой, поэтому для перемещения его к первой строке нужно вызвать метод *next()*. Строки в наборе результатов располагаются совершенно произвольно. Если порядок следования важен, его необходимо установить с помощью оператора *ORDER BY*.

При обработке каждой отдельной строки нужно с помощью специальных методов получить содержимое каждого столбца.

```
resultSet.getString(1);
resultSet.getDouble("Price");
```

Для каждого типа данных языка Java предусмотрен отдельный метод извлечения информации, например, *getString()* и *getDouble()*. Для каждого из них реализовано два варианта, один из которых предполагает числовой, а второй – строковый параметр. При использовании числового параметра метод извлечет данные из столбца с указанным номером, нумерация столбцов начинается с 1. При использовании строкового параметра метод извлечет данные из столбца с указанным именем.

Если указанный тип не соответствует фактическому типу, метод извлечения данных выполняет преобразование. Например, метод *resultSet.getString("Price")* преобразует число с плавающей точкой из столбца *Price* в строку.

Типы данных SQL и Java не всегда совпадают. В таблице 3 приведен список основных типов данных языка SQL и их эквивалентов в Java.

Таблица 3

Типы данных языка SQL	Типы данных языка Java
INTEGER или INT	int
SMALLINT	short
NUMERIC(m,n), DECIMAL(m,n) или DEC(m,n)	java.math.BigDecimal
REAL	float
DOUBLE	double
CHARACTER(n) или CHAR(n), VARCHAR	String
BOOLEAN	boolean
DATE	java.sql.Date
TIME	java.sql.Time

Метод *executeUpdate()* возвращает количество строк, полученных в результате выполнения SQL-команды. Он может применяться для выполнения команд INSERT, UPDATE и DELETE, а также команд определения данных, в частности, CREATE TABLE и DROP TABLE.

Закончив работу с классом ResultSet, Statement или Connection нужно вызвать метод *close()*, так как перечисленные объекты используют структуры данных большого объема и не стоит ждать, пока процедура сборки мусора освободит ресурсы. Метод *close()* класса Statement автоматически закрывает связанные с ним наборы результатов(если они открыты). Аналогично, метод *close()* класса Connection закрывает все объекты Statement для этого соединения. Если время существования соединения невелико, нужно поместить вызов метода *close()* в блок finally, гарантируя, что объект соединения не останется открытым.

Предварительно подготовленные команды

Вместо создания отдельной SQL-команды для каждого запроса следует предварительно подготовить запрос с подстановочной переменной и многократно использовать его, меняя только значение переменной. Эта возможность существенно повышает эффективность работы программы. Перед каждым выполнением запроса СУБД создает план его эффективного исполнения. Предварительно подготавливая запрос для последующего многократного повторного использования, можно избежать повторного создания плана. Подстановочная переменная в запросе обозначается символом `?`. При использовании нескольких подстановочных переменных нужно следить за их расположением, чтобы правильно выполнять подстановку фактических значений.

```
String s = "SELECT name, price FROM tovar WHERE price >? AND price <?";  
PreparedStatement ps = connection.prepareStatement(s);  
ps.setDouble(1,2000);  
ps.setDouble(2,6000);  
ResultSet rs1 = ps.executeQuery();
```

Перед выполнением предварительно подготовленного запроса нужно с помощью метода `set()` связать подстановочные переменные с их фактическими значениями. Как и при использовании методов `get()`, класса `ResultSet`, для различных типов предусмотрены разные методы `set()`. Например:

```
ps.setDouble(1,2000);
```

Первый параметр обозначает номер позиции подстановочной переменной, второй – её фактическое значение. При повторном использовании предварительно подготовленного запроса с несколькими подстановочными переменными все связи остаются в силе, если только они не изменены с помощью метода `set()`. Это значит, что методы `set()` нужно вызывать только для тех подстановочных переменных, которые изменяются в последующих запросах. Для изменения данных используется метод `executeUpdate()`.

Метаданные

В JDBC предусмотрены дополнительные возможности для получения информации о структуре таблиц и самой БД. Например, можно получить список всех таблиц БД либо имена всех столбцов с типами данных в них. Эти сведения пригодятся, если вы пишете программный продукт для работы с произвольной БД.

В языке SQL информация о структуре БД и её компонентов называется **метаданными**. Информация о структуре БД хранится в классе `DatabaseMetaData`. Ниже приведен пример получения набора результатов, содержащего информацию обо всех таблицах БД.

```
DatabaseMetaData meta = connection.getMetaData();
```

```
ResultSet mrs = meta.getTables(null, null, null, new String[] {"TABLE"});
```

Метод `getTables(String catalog, String schemaPattern, String tableNamePattern String types[])` представляет описание всех таблиц в каталоге, которые соответствуют шаблонам схемы и имен таблиц, а также заданным типам. **Схемой** называется группа связанных таблиц и полномочий доступа. **Каталогом** называется связанная группа схем. Параметры `catalog` и `schemaPattern` могут быть пустыми строками(" ") для извлечения таблиц без каталога и схемы либо иметь значение `null`, если нужно возвращать таблицы без учета каталога или схемы. Массив `types` содержит имена типов таблиц: `TABLE`, `VIEW`, `ALIAS` и т.д. Если для `types` задано значение `null`, то будут возвращены таблицы всех типов. Результат выполнения запроса содержит 5 столбцов, которые имеют тип `String` и показаны в таблице 4.

Таблица 4

1	TABLE_CAT	Каталог таблиц (значение может быть равно null)
2	TABLE_SCHEM	Схема (значение может быть равно null)
3	TABLE_NAME	Имя таблицы
4	TABLE_TYPE	Тип таблицы

5	REMARKS	Комментарии для таблицы
---	---------	----------------------------

Если нас интересует имя таблицы, т.е. третий столбец, то нужно вызвать метод *mrs.getString(3)*.

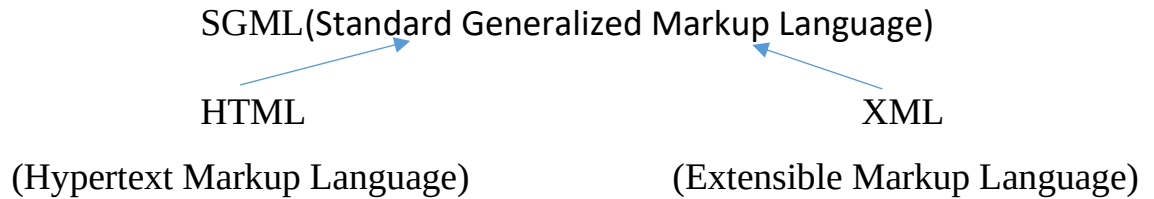
Сведения о структуре набора результатов хранятся в классе *ResultSetMetaData*.

```
ResultSet rs = statement.executeQuery("SELECT * FROM item");
```

```
ResultSetMetaData rsmd= rs.getMetaData();
```

После получения результатов выполнения запроса можно определить количество столбцов методом *getColumnCount()*, имена столбцов *getColumnLabel(int column)* и т.д.

XML



- SGML – с 1970-х годов использовался для описания структуры сложных документов в некоторых отраслях промышленности с высокими требованиями к документации.
- HTML – язык разметки гипертекста.
- XML – язык для описания иерархических данных.

Отличия XML от HTML

- В отличие от HTML, в языке XML учитывается регистр символов.
- В языке HTML некоторые закрывающие дескрипторы могут отсутствовать, в XML это не разрешается.
- Для элементов, тело которых отсутствует, в XML предусмотрена сокращенная запись открывающего дескриптора, совмещённого с закрывающим. В этом случае открывающий дескриптор заканчивается символом /
- В XML значения атрибутов должны быть помещены в кавычки, а в HTML кавычки могут отсутствовать.
- В HTML допускается использование имен атрибутов без указания их значений, а в XML все атрибуты должны иметь значения.

Пример 51

Пример XML:

```
<configuration>
  <title>
    <font>
      <name>Helvetica</name>
      <size>36</size>
    </font>
```

```

</title>
<body>
  <font>
    <name>Times Roman</name>
    <size>12</size>
  </font>
</body>
<window>
  <width>400</width>
  <height>200</height>
</window>
<color>
  <red>0</red>
  <green>50</green>
  <blue>100</blue>
</color>
<menu>
  <item>Times Roman</item>
  <item>Helvetica</item>
</menu>
</configuration>

```

Структура XML - документа

- **XML- документ начинается с заголовка (необязательно)**
 <?xml version="1.0"?>
 <?xml version="1.0" encoding="UTF-8"?>
- **Далее идёт DTD (Document Type Definition) (необязательно)**
 <!DOCTYPE web-app PUBLIC "-//Sun Microsystems, Inc.
 //DTD Web Application 2.2//EN"
 "http://java.sun.com/j2ee/dtds/web-app_2_2.dtd">

- **Наконец, тело XML-документа содержит корневой элемент, в состав которого могут входить другие элементы**

`<configuration>` – корневой элемент

- Каждый элемент может содержать дочерние элементы, текст либо и то, и другое
- XML-документы рекомендуется формировать так, чтобы элементы содержали либо дочерние элементы, либо текст, т.е. следует избегать создания структур:

`<font>`

Helvetica

`<size>36</size>`

`</font>`

Такая структура называется **смешанным содержимым**/

- XML-элементы могут содержать атрибуты

`<size unit="pt">36</size>`

- Элементы и текст являются основными компонентами XML-документов, но возможны и другие инструкции:

- Ссылки на символы `&#d` или `&#xh`, где d – десятичное, а h – шестнадцатеричное Unicode-значение. Например, `é`, `™`
- Ссылки на примитивы имеют вид `&name`. Например, `<`, `>`, `&`, `"`, `'`
- CDATA-разделы ограничены группами символов `<![CDATA[` и `]]>`, предназначены для включения строк с символами `<`, `>`, `&`, которые не следует интерпретировать как символы разметки
`<![CDATA[ < & > are my favorite delimiters ] ]>`
- Инструкции обработки ограничиваются символами `<?` и `?>`, нужны приложению в котором используется данный XML-

документ

<?xml version="1.0"?>

- Комментарии ограничиваются символами <! -- и -->. В комментариях нельзя использовать строки типа --. Не следует включать в комментарии скрытые команды, для их выполнения предназначены инструкции обработки

<!-- This is a comment. -->

DTD и XML-схема

- XML-анализатор может автоматически проверять корректность структуры документа
- Для указания структуры документа нужно создать DTD либо XML-схему
- Они указывают допустимые дочерние узлы элементов и атрибутов каждого элемента
- XML-схема позволяет формулировать сложные правила, которые нельзя выразить средствами DTD
- В отличие от DTD, правила, соответствующие схеме, выражаются на языке XML, что упрощает их обработку
- XML-схема сложнее DTD

Document Type Definition (DTD)

- Существует несколько способов связывания DTD с XML-документом:

- Первый способ
- ```
<?xml version="1.0"?>
<!DOCTYPE configuration [
 <!ELEMENT configuration ...>
 другие правила
 ...
]>
< configuration >
```

...

</ configuration>

- Второй способ  
<!DOCTYPE configuration SYSTEM "config.dtd">
- Третий способ  
<!DOCTYPE configuration SYSTEM  
"http://myserver.com/config.dtd">
- Рассмотрим правила, которые могут задаваться посредством DTD.
- Правило ELEMENT указывает дочерние узлы данного элемента в виде регулярного выражения, которое состоит из перечисленных в таблице 5 компонентов.

Таблица 5

| Правило              | Описание                                                                                                                |
|----------------------|-------------------------------------------------------------------------------------------------------------------------|
| E*                   | 0 или более элементов E                                                                                                 |
| E+                   | 1 или более элементов E                                                                                                 |
| E?                   | 0 или 1 элемент E                                                                                                       |
| E1 E2 ... En         | Одно из элементов E1, E2, ..., En                                                                                       |
| E1, E2, ..., En      | Элемент E1, за которым следуют E2, ..., En                                                                              |
| #PCDATA              | Текст                                                                                                                   |
| (#PCDATA E1 ... En)* | 0 или более текстовых элементов и элементы E1, E2, ..., En, расположенные в произвольном порядке (смешанное содержание) |
| ANY                  | Любой дочерний узел                                                                                                     |
| EMPTY                | Нет дочерних узлов                                                                                                      |

#### Примеры правил

- <!ELEMENT menu (item)\*>
- <!ELEMENT font (name,size)>
- <!ELEMENT name (#PCDATA)>

<!ELEMENT size (#PCDATA)>

- <!ELEMENT chapter (intro,(heading,(para|image|table|note)+)+)>
- Для описания допустимых атрибутов элементов используется приведенный ниже синтаксис:  
<!ATTLIST элемент атрибут тип поведение\_по\_умолчанию>
- В таблицах 6 и 7 приведены допустимые типы атрибутов и синтаксис параметра поведение\_по\_умолчанию.

Таблица 6 – Типы атрибутов

| Тип               | Описание                                              |
|-------------------|-------------------------------------------------------|
| CDATA             | Произвольная символьная строка                        |
| (A1 A2 ... An)    | Один из строковых атрибутов A1  A2  ... An            |
| NMTOKEN, NMTOKENS | Одна или несколько лексем, соответствующих имени      |
| ID                | Уникальный идентификатор                              |
| IDREF, IDREFS     | Одна или несколько ссылки на уникальный идентификатор |
| ENTITY, ENTITIES  | Один или несколько непроанализированных примитивов    |

Таблица 7 – Поведение по умолчанию

| Тип       | Описание                                                                                   |
|-----------|--------------------------------------------------------------------------------------------|
| #REQUIRED | Атрибут является обязательным                                                              |
| #IMPLIED  | Атрибут не является обязательным                                                           |
| A         | Атрибут не является обязательным, анализатор возвращает значение A, если атрибут не указан |
| #FIXED A  | Атрибут не должен быть указан, либо должен быть равен A, в любом                           |

|  |                                         |
|--|-----------------------------------------|
|  | случае анализатор возвращает значение A |
|--|-----------------------------------------|

### Примеры выражений:

```
<!ELEMENT gridbag (row)*>
<!ELEMENT row (cell)*>
<!ATTLIST cell gridwidth CDATA "1">
<!ATTLIST cell gridheight CDATA "1">
<!ATTLIST cell fill (NONE|BOTH|HORIZONTAL|VERTICAL) "NONE">
<!ATTLIST cell anchor (CENTER|NORTH|NORTHEAST|EAST
|SOUTHEAST|SOUTH|SOUTHWEST|WEST|NORTHWEST)
"CENTER">
<!ATTLIST cell ipadx CDATA "0">
<!ATTLIST cell ipady CDATA "0">
```

### DTD и соответствующий XML-документ

```
<?xml version="1.0"?>
<!DOCTYPE employee [
<!ELEMENT employee (Name, Dept, Title)>
<!ELEMENT Name (#PCDATA)>
<!ELEMENT Dept (#PCDATA)>
<!ELEMENT Title (#PCDATA)>
]>
<employee>
<Name>John Goodman</Name>
<Dept>Manufacturing</Dept>
<Title>Supervisor</Title>
</employee>
```

### XML-схема

- Для включения в документ ссылку на файл схемы нужно указать в корневом элементе соответствующие атрибуты, например:  

```
<?xml version="1.0"?>
```

*< configuration xmlns: xsi = http://www.w3.org/2001/XMLSchema-instance*

*xsi: noNamespaceSchemaLocation = "config.xsd">*

*...*

*< /configuration >*

- В данном случае в составе корневого элемента содержится информация о том, что при проверке документа должен использоваться файл схемы config.xsd.
- Если применяется пространство имен – синтаксис усложняется.
- Схема определяет тип каждого элемента
- Это может быть простой или сложный тип
- Некоторые простые типы предопределены в XML-схеме:
  - *xsd:string*
  - *xsd:int*
  - *xsd:boolean*
- Можно определять собственные простые типы
- Вместо префикса *xsd* (обозначает пространство имен XML Schema Definition), можно применять *xs*:
- Определяя элемент, следует указать его тип:
  - *<xsd:element name="name" type="xsd:string"/>*
  - *<xsd:element name="size" type="xsd:int"/>*
- Простые типы можно объединять в сложные:

*<xsd:complexType name = "FontType">*  
*<xsd:sequence>*  
*<xsd:element ref = "name" />*  
*<xsd:element ref = "size" />*  
*<xsd:element ref = "style" />*  
*</xsd:sequence>*  
*</xsd:complexType >*
- *FontType* – последовательность элементов *name*, *size* и *style*.

- Атрибут `ref` используется для ссылки на определения, находящиеся в схеме.
- Возможны вложенные определения.
- Для определения атрибутов необходимо включать элементы `xsd:attribute`.

## **Dom и SAX**

- Для разборки XML-документа необходимо выполнить синтаксический анализ или разбор его содержимого.
- **Анализатором (parser)** называется программа, которая считывает файл, подтверждает корректность его формата, разбивает данные на составные элементы и предоставляет программисту доступ к ним.
- Существует два основных типа XML-анализаторов:
  - **DOM-анализатор (Document Object Model)** считывает XML-документ и представляет его в виде древовидной структуры.
  - **SAX-анализатор (Simple API for XML)** генерирует события по мере чтения XML-документа.
- DOM-анализатор представляет документ как древовидную структуру, SAX-анализатор генерирует события в ходе разбора документа.
- При обработке больших документов DOM-анализатор неэффективен, так как требуется память для хранения всей древовидной структуры, SAX-анализатор более экономичный.
- Сформированную древовидную структуру можно многократно и по-разному обходить, при использовании SAX-анализатора события генерируются последовательно в порядке возникновения и повторный обход документа невозможен.

## **Dom**

- Пакет org.w3c.dom содержит определения типов интерфейсов, например, Document и Element.
- Ряд компаний, например, IBM и Apache, разработали собственные варианты DOM-анализаторов, которые реализуют эти интерфейсы.
- В API обработки XML-документов(Java API for XML Processing-JAXP) компании Sun предусмотрена возможность подключения данных анализаторов.
- Кроме того, компания Sun включила в состав пакета JDK собственный анализатор.

Для считывания XML-документа потребуется объект **DocumentBuilder**, который можно получить с помощью класса **DocumentBuilder Factory**.

```
DocumentBuilder Factory factory = DocumentBuilder
Factory.newInstance();
```

```
DocumentBuilder builder = factory.new DocumentBuilder();
```

```
File f = ... // можно использовать URL или InputStream
Document doc = builder.parse(f);
```

- Объект **Document** является внутренним представлением древовидной структуры XML-документа.
- Он состоит из экземпляров классов, реализующих интерфейс **Node** и его дочерние интерфейсы (**Attr**, **Comment**, **Document**, **Element**, **Text**).
- Анализ содержимого документа начинается с вызова метода **getDocumentElement()**, который возвращает корневой элемент  
Element root = doc.getDocumentElement();
- Метод **getTagName()** возвращает имя дескриптора элемента.
- Для извлечения элементов, дочерних по отношению к данному (вложенных элементов, текста, комментариев или других узлов)

предназначен метод **getChildNodes()**, возвращающий набор данных типа **NodeList**.

- Метод **item()** возвращает элемент набора с заданным индексом.
- Метод **getLength()** возвращает общее количество элементов.
- Методы **getFirstChild()** и **getLastChild()** используются для получения первого и последнего дочернего узла.
- Метод **getNextSibling()** позволяет получить следующий узел того же уровня.
- Метод **getAttributes()** возвращает карту **NameNodeMap** с объектами типа **Node**, описывающими атрибуты.
- Методы **getNodeName()** и **getNodeValue()** извлекают имя и значение атрибута соответственно.
- Метод **getAttributes("имя ")** извлекает значение атрибута по имени.

#### Пример 52

- *import org.w3c.dom.\*;*
- *import javax.xml.parsers.\*;*
- *public class MyXml {*
- *public static void main(String[] args) {*
- *try {*
- *DocumentBuilderFactory factory =*  
*DocumentBuilderFactory.newInstance();*
- *DocumentBuilder builder = factory.newDocumentBuilder();*
- *Document doc = builder.parse("G:\\Haulmont\\java\_SE\\config.xml");*
- *System.out.print("The elements are: ");*
- *printElements(doc);*
- *System.out.println("The attributes of each element are: ");*

- `printElementAttributes(doc);`
- `} catch (Exception e) { System.out.println(e.toString()); }`
- `}`
- `static void printElements(Document doc) {`
- `NodeList nl = doc.getElementsByTagName("*");`
- `for (int i = 0; i < nl.getLength(); i++) {`
- `System.out.print(nl.item(i).getNodeName() + " ");`
- `}`
- `System.out.println();`
- `}`
- `static void printElementAttributes(Document doc) {`
- `NodeList nl = doc.getElementsByTagName("*");`
- `for (int j = 0; j < nl.getLength(); j++) {`
- `Element e = (Element) nl.item(j);`
- `System.out.println(e.getTagName() + ":");`
- `NamedNodeMap nnm = e.getAttributes();`
- `if (nnm != null) {`
- `for (int i = 0; i < nnm.getLength(); i++) {`
- `Node n = nnm.item(i);`
- `System.out.print("    " + n.getNodeName() + " = " +`
- `n.getNodeValue());`
- `}`
- `}`

- `System.out.println();`
- `} } }`

## SAX

- Для использования SAX-анализатора необходимо создать обработчик событий, которые происходят во время разбора XML-документа.
- В интерфейсе **ContentHandler** определено несколько методов, к которым обращается анализатор:

- Методы **startElement()** и **endElement()** вызываются при получении открывающего и закрывающего дескрипторов.
- Метод **characters()** вызывается при получении символьных данных.
- Методы **startDocument()** и **endDocument()** вызываются в начале и в конце документа.

- Для получения SAX-анализатора используется следующая команда:

```
SAXParserFactory factory = SAXParserFactory.newInstance();
```

```
SAXParser parser = factory.newSAXParser();
```

- Для обработки документа следует использовать команду:

```
parser.parse(source, handler);
```

Здесь `source` – это источник входных данных, который может быть файлом, URL или входным потоком; `handler` – подкласс класса **DefaultHandler**, в котором определены методы, объявленные в интерфейсах **ContentHandler**, **DTDHandler**, **EntityResolver**, **ErrorHandler**.

- Эти методы не выполняют никаких действий.

## Пример 53

- `import java.io.IOException;`
- `import javax.xml.parsers.*;`

- `import org.xml.sax.*;`
- `import org.xml.sax.helpers.*;`
- `class CustomSAXParser extends DefaultHandler{`
- `public void startDocument() throws SAXException {`
- `System.out.println("Start parse XML...");`
- `}`
- `public void startElement(String namespaceURI, String localName,`
- `String qName, Attributes atts) throws SAXException {`
- `System.out.println("Start element " + qName + ":");`
- `}`
- `public void endElement(String namespaceURI, String localName,`
- `String qName) throws SAXException {`
- `System.out.println("End element " + qName + ":");`
- `}`
- `public void endDocument() {`
- `System.out.println("Stop parse XML...");`
- `}`
- `public static void main(String[] args) {`
- `try {`
- `SAXParserFactory factory = SAXParserFactory.newInstance();`
- `SAXParser parser = factory.newSAXParser();`
- `CustomSAXParser custom = new CustomSAXParser();`
- `parser.parse("G:\\Haulmont\\java_SE\\config.xml", custom);`

- `} catch (IOException ex) {`
- `ex.printStackTrace();`
- `} catch (ParserConfigurationException ex) {`
- `ex.printStackTrace();`
- `} catch (SAXException ex) {`
- `ex.printStackTrace();`
- `} } }`

### Создание XML-документов

- Используется пакет **javax.xml.transform**.
- Удобнее создать древовидную структуру DOM, представляющую документ, а затем записать содержимое этого дерева в файл.
- В первую очередь следует сгенерировать пустой документ с помощью метода **newDocument()** класса **DocumentBuilder**  
`Document doc = builder.newDocument();`
- Затем для создания элементов документа нужно применить метод **createElement()** класса **Document**  
`Element rootElement = doc.createElement(rootName);`  
`Element childElement = doc.createElement(childName);`
- Далее создаем текстовые узлы с помощью метода **createTextNode()**.  
`Text textNode = doc.createTextNode(textContents);`
- Для включения в документ корневого элемента и для связывания дочерних узлов с родительскими используется приведенный ниже код:  
`doc.appendChild(rootElement);`  
`rootElement.appendChild(childElement);`  
`childElement.appendChild(textNode);`

- При формировании дерева DOM атрибуты элементов задаются с помощью метода **setAttribute()** класса **Element**  
*rootElement.setAttribute(name, value);*

Для записи дерева DOM в выходной поток к документу применяется преобразование (XSLT-преобразование), в процессе которого не выполняется никаких действий и результат записывается в выходной поток.

- Для включения узла DOCTYPE в состав выходных данных нужно указать в качестве свойств идентификаторы SYSTEM и PUBLIC  
*Transformer t = TransformerFactory.newInstance().newTransformer();*  
*t.setOutputProperty(OutputKeys.DOCTYPE\_SYSTEM, systemIdentifier);*  
*t.setOutputProperty(OutputKeys.DOCTYPE\_PUBLIC, publicIdentifier);*  
*t.transform(new DOMSource(doc), new StreamResult(new*  
*FileOutputStream(file)));*

Пример 54

```
import org.w3c.dom.*;
import javax.xml.parsers.*;
import javax.xml.transform.*;
import javax.xml.transform.dom.*;
import javax.xml.transform.stream.*;
import java.io.*;

public class WriteXML {
 public static void main(String[] args) {
 try {
 DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();
 DocumentBuilder builder = factory.newDocumentBuilder();
 Document doc = builder.parse("G:\\Haulmont\\java_SE\\config.xml");
 Transformer t = TransformerFactory.newInstance().newTransformer();
 t.setOutputProperty(OutputKeys.DOCTYPE_SYSTEM,
 doc.getDoctype().getSystemId());
```

```
FileOutputStream f = new FileOutputStream("G:\\Haulmont\\java_SE\\config1.xml");
t.transform(new DOMSource(doc), new StreamResult (f));
 } catch (Exception e) {
 e.printStackTrace();
 }
}
}
```

# **Hibernate**

ORM (Object-relational mapping) — это отображение объектов какого-либо объектно-ориентированного языка в структуры реляционных баз данных.

## **Преимущества ORM**

- Нет необходимости писать рутинные insert/update/delete/select для CRUD(create, read, update, delete) операций.
- Условия связи между объектами (строками таблиц) указываются декларативно в одном месте.
- Возможность использовать полиморфные запросы для иерархий классов.
- Высокая степень независимости от конкретной СУБД.

## **Недостатки ORM**

- Возможны проблемы с производительностью для сложных запросов на объектном SQL.
- Затрудняет использование специфических конструкций языка SQL конкретной СУБД.
- Потеря объектно-реляционного соответствия.

ORM-решением для языка Java является технология Hibernate, которая не только связывает Java классы с таблицами базы данных, но также предоставляет средства для автоматического построения запросов и извлечения данных и может значительно уменьшить время разработки, которое обычно тратится на ручное написание SQL и JDBC кода. Hibernate генерирует SQL вызовы и освобождает разработчика от ручной обработки результирующего набора данных и конвертации объектов.

Mapping (сопоставление, проецирование) Java-классов с таблицами БД осуществляется с помощью конфигурационных XML файлов или Java-аннотаций. Обеспечиваются возможности по организации отношений между классами «один-ко-многим» и «многие-ко-многим».

Для применения Hibernate на практике во-первых, необходимо описать XML файл, содержащий информацию о настройках связи с БД и о способе Mapping. Располагаться он должен в самом корне дерева классов.

Пример 55

Файл hibernate.cfg.xml.

```
<hibernate-configuration>
 <session-factory>
 <!-- PostgreSQL connection settings -->
 <property name="connection.driver_class">org.postgresql.Driver</property>
 <property name="connection.url">jdbc:postgresql://localhost:5433/orm</property>
 <property name="connection.username">postgres</property>
 <property name="connection.password">sa</property>
 <property name="dialect">org.hibernate.dialect.PostgreSQLDialect</property>
 <property name="hbm2ddl.auto">update</property>

 <!-- Other hibernate settings -->
 <property name="show_sql">true</property>

 <!-- Annotated classes (Mapping from Annotations) -->
 <mapping package="org.haulmont.catalog.model"/>
 <mapping class="org.haulmont.catalog.model.Shop"/>
 <mapping class="org.haulmont.catalog.model.Book"/>
 <mapping class="org.haulmont.catalog.model.Magazine"/>
 <mapping class="org.haulmont.catalog.model.Author"/>

 <!-- Mapping from configuration file (as an alternative to annotations) -->
 <!--<mapping resource="mapping.hbm.xml"/>-->
 </session-factory>
</hibernate-configuration>
```

- `connection.driver_class` Показывает класс драйвера, который используется для соединения с базой данных.
- `connection.url` –URL для соединения с базой данных.
- `connection.username` – логин к базе данных.
- `connection.password` – пароль к базе данных.
- `dialect` – т.к. Hibernate может работать с разными базами данных, и каждая имеет какие-то особенности (генерация первичного ключа, страничный вывод, функции), нам надо указать, с какой базой мы работаем. В данном случае у нас PostgreSQL, что мы и указываем.
- `hbm2ddl.auto` - свойство, которое указывается что нужно сделать со схемой БД при инициализации. Может принимать такие значения:
  - update** - сверяет схему БД с имеющимися конфигурациями классов, если мы внесли какие-то изменения, они автоматически занесутся в БД. При этом данные, которые были занесены в базу не изменятся - даже, если мы решили удалить некоторые поля из таблицы, они все одно останутся;
  - create** - каждый раз при запуске приложения, схема БД будет создаваться заново. Все данные, которые были занесены раньше, будут удалены;
  - create-drop** - каждый раз при запуске приложения, схема БД будет создаваться заново, а при завершении - удаляться. Все данные, которые были занесены во время работы приложения, будут удалены по завершению приложения;
  - validate** - при инициализации будет совершена проверка соответствуют ли конфигурации классов и схема БД. Если мы внесли изменение в конфигурацию какого-то класса, а схема в БД не была изменена, выбросится исключение;
- `show_sql` – данное свойство указывает, будут ли выводиться SQL-запросы, которые генерит Hibernate на консоль. В процессе отладки это бывает очень удобно.

- тэг – mapping. Он используется для перечисления всех классов, которые имеют связь с базой данных. Т.е. если вы хотите использовать какой-то класс для связи с какой-то таблицей, вы должны его здесь указать.

Во-вторых, нужно описать классы для работы с данными.

Пример 56

```

package org.haulmont.catalog.model;

import java.util.List;

import javax.persistence.*;

@javax.persistence.SequenceGenerator(name =
"SEQ_ID", sequenceName = "SEQ_ID")
@Entity
@Table(name = "SHOP")
public class Shop {
 private Long id;
 private String name;
 private List<Item> items;

 @Id
 @GeneratedValue(generator = "SEQ_ID")
 @Column(name = "ID")
 public Long getId() {
 return id;
 }

 public void setId(Long id) {
 this.id = id;
 }

 @Column(name = "NAME")
 public String getName() {
 return name;
 }
}

```

```

 public void setName(String name) {
 this.name = name;
 }

 @OneToMany(cascade = CascadeType.ALL, fetch =
FetchType.LAZY)
 @JoinColumn(name = "SHOP_ID", nullable = false)
 public List<Item> getItems() {
 return items;
 }

 public void setItems(List<Item> items) {
 this.items = items;
 }
}

```

- Класс **Shop** связан с таблицей **SHOP**.
- Первичный ключ – **ID**, связан со свойством **id** (функции **getId()/setId()**).
- Для генерации значений первичного ключа **ID** используется sequence **SEQ\_ID**.
- Атрибут **NAME** связан со свойством **name** (функции **getName()/setName()**).
- Объекты **Shop** содержат список **Item**:
  - связь “один-ко-многим” (**@OneToMany**)
  - обязательная (**nullable=false**)
  - Внешний ключ, определяющий связь – **SHOP\_ID**.
- Список **Item** загружаются по требованию (**FetchType.LAZY**).
- При сохранении объекта **Shop** автоматически сохраняются все его **items** (**CascadeType.ALL**).

Рассмотрим подробнее используемые аннотации:

- **@Entity** – говорит о том, что данный класс представляет собой сущность, которую надо сохранять в базе данных.
- **@Table** – показывает, какая таблица используется для хранения.
- **@Id** – говорит о том, что данное поле является идентификатором.
- **@GeneratedValue** – аннотация дает указание, что данная величина будет генериться.
- **@Column** – аннотация для указания имени столбца, с которым связано поле класса.
- **@JoinColumn** – данная аннотация по сути похожа на **@Column**. Только она указывает, что колонка является связующей.
- **@ManyToOne** – аннотация указывает, что поле указывает на другой класс, который связан с текущим классом связью многие-к-одному. Здесь также указывается правило каскада **cascade=CascadeType.REFRESH** – в упрощенном виде оно гласит “что сделали с тобой, то сделай и со связью. **CascadeType.ALL** обозначает выполнение каскадных операций при **save-update** и **delete**, по умолчанию **Hibernate**

будет игнорировать ассоциации, поэтому “разруливать” зависимости придется самостоятельно. Запись `fetch=FetchType.LAZY` указывает, что загрузка данного поля будет только в случае обращения к данному полю. Пока программа этого не делает, поле будет пустым.

- `@OneToMany` – такая запись говорит о том, что поле служит для связи один-ко-многим (потому и список).
- `@ManyToMany` – эта аннотация говорит о том, что поле будет использовано для связи много-ко-многим.
- `@JoinTable` – указывает имя таблицы, которая используется для организации связи много-ко-многим. Параметр `joinColumns` указывает название столбца, который является ссылкой на текущий класс, параметр `inverseJoinColumns` указывает на колонку, в которой находится ссылка на класс с другой стороны.

В качестве альтернативы аннотациям возможно использование XML-файла с настройками (файл `mapping.cfg.xml`). Во-третьих, нужно описать класс для работы с Hibernate API.

- Работая с данными в объектно-ориентированном языке, мы работаем с объектами, заполняя и считывая значения полей, создавая новые или изменяя существующие объекты, определяя зависимости между объектами.
- При операции **`save()`** мы передаем объект типа **`Shop`**, который сохраняется в базу данных по описанным правилам отображения. В том числе сохраняются и все зависимые объекты (**`Item`**).
- Составляя запросы к базе данных, мы уже указываем не столбцы таблицы, а свойства объектов.

Обратите внимание на работу с транзакциями: начало транзакции( метод `beginTransaction()`), подтверждение транзакции (метод `commit()`), откат транзакции (метод `rollback()`).

Пример 57, 58

```

<hibernate-mapping
package="org.haulmont.catalog.model">

 <class name="Author" table="AUTHOR">
 <id name="id" type="long" column="ID">
 <generator class="sequence">
 <param name="sequence">SEQ_ID</param>
 </generator>
 </id>
 <property name="name" column="NAME"/>
 <property name="birthdate" column="BIRTHDATE"
type="date"/>
 <bag name="items" cascade="refresh">
 <key column="AUTHOR_ID"/>
 <one-to-many
class="org.haulmont.catalog.model.Item"/>
 </bag>
 </class>

 <class name="org.haulmont.catalog.model.Shop"
table="SHOP">
 <id name="id" type="long" column="ID">
 <generator class="sequence">
 <param name="sequence">SEQ_ID</param>
 </generator>
 </id>
 <property name="name" column="NAME"/>
 <bag name="items" cascade="all">
 <key column="SHOP_ID" not-null="true"/>
 <one-to-many
class="org.haulmont.catalog.model.Item"/>
 </bag>
 </class>

 <class name="org.haulmont.catalog.model.Item"
table="ITEM">

```

```

 <id name="id" type="long" column="ID">
 <generator class="sequence">
 <param name="sequence">SEQ_ID</param>
 </generator>
 </id>
 <discriminator column="ITEM_TYPE" type="string"/>

 <property name="name" column="NAME"/>
 <many-to-one name="shop" column="SHOP_ID" insert="false"
update="false"/>
 <many-to-one name="author" cascade="save-update"
column="AUTHOR_ID"/>

 <subclass name="org.haulmont.catalog.model.Book"
discriminator-value="BOOK">
 <property name="ISBN" column="ISBN"/>
 </subclass>

 <subclass name="org.haulmont.catalog.model.Magazine"
discriminator-value="MAGAZINE">
 <property name="type" column="TYPE">
 <type name="org.hibernate.type.EnumType">
 <param
name="enumClass">org.haulmont.catalog.model.Magazine$MagazineTy
am>
 <param name="type">12</param>
 </type>
 </property>
 </subclass>
 </class>

</hibernate-mapping>

```

```
public class HibernateSample {

 public static void main(String[] args) {
 // Получим сессию Hibernate
 Session session =
HibernateUtil.getSession();

 // Начало транзакции
 session.beginTransaction();

 // Создание нового магазина и
заполнение данными
 Shop shop = new Shop();
 shop.setName("Central shop");

 List<Item> items = new
LinkedList<Item>();

 Author eckel = new Author();
 eckel.setName("Bruce Eckel");
 eckel.setBirthdate(new Date());

 Author white = new Author();
 white.setName("Kate White");
 white.setBirthdate(new Date());

 Book book = new Book();
 book.setAuthor(eckel);
 book.setName("Thinking In Java");
 book.setISBN("1-232-12345-2");
 book.setShop(shop);
```

```
 Magazine magazine = new Magazine();
 magazine.setAuthor(white);
 magazine.setName("Cosmopolitan");
 magazine.setType(Magazine.MagazineType.WOMEN);
 magazine.setShop(shop);

 items.add(book);
 items.add(magazine);

 shop.setItems(items);

 // Сохранение магазина в базу данных
 session.save(shop);
 // Завершение транзакции
 session.getTransaction().commit();

 System.out.println("Id=" + shop.getId());

 // Вывод списка магазинов
 List<Shop> shops =
 (List<Shop>) session.createQuery("from
c where c.name like '%shop%'").list();

 for (Shop item : shops) {
 System.out.println(item.getId() + " " +
item.getName());
 }
 }
}
```



## Servlets

**Servlet (сервлет)** – это класс на Java, расширяющий возможности сервера, к которому происходят обращения в рамках модели «запрос-отклик». Теоретически может поддерживать любой тип запросов, но чаще всего используются HTTP-сервлеты.

**Servlet container (контейнер сервлета)** - контейнер, обеспечивающий сетевые службы, при помощи которых посылаются запросы и ответы, декодируются запросы и форматируются ответы. Все контейнеры сервлетов должны поддерживать HTTP протокол, но могут также поддерживать дополнительные протоколы, например, HTTPS.

**Distributed servlet container (распределенный контейнер сервлета)** - контейнер сервлета, запускающий Web-приложения, которые помечены как распределенные и выполняются на нескольких виртуальных машинах Java. При этом виртуальные машины могут быть запущены, как на одном, так и на разных компьютерах.

**Servlet context (контекст сервлета)** - объект, содержащий представление (вид) Web-приложения, в котором запущен сервлет. Используя контекст, сервлет может вести журнал событий, получать URL-ссылки на ресурсы, а также устанавливать и хранить атрибуты, которые могут использоваться другими сервлетами в приложении.

**Servlet mapping (отображение сервлета)** - определяет связь между структурой URL и сервлетом. Используется для отображения запросов в сервлеты. Если контейнер, обрабатывающий запрос, является JSP-контейнером, то неявно отображается URL, содержащий расширение .jsp.

### Жизненный цикл сервлета

- Если экземпляр сервлета не существует, то контейнер:
  1. загружает класс сервлета;
  2. создает его экземпляр;
  3. вызывает метод **init()**.

- При получении запроса вызывается метод **service()**, которому передаются объекты запроса и отклика.
- Если контейнеру требуется удалить объект сервлета, то вызывается метод **destroy()**.

### Особенности сервлетов

- Существуют механизмы реагирования на события, связанные с деятельностью сервлетов в приложении:
  - **javax.servlet.СобытиеListener**
  - **javax.servlet.СобытиеEvent**
- Для обработки события требуется:
  - написать класс, реализующий интерфейс.
  - снабдить этот класс аннотацией **@WebListener** (пакет **javax.servlet.annotation**).
- Требуется учитывать то, что сервлет является разделяемым ресурсом и работает с разделяемыми ресурсами.
- Не следует использовать поля класса для хранения состояния, а особенно для передачи состояния между вызовами.

### Объект запроса

- Реализует интерфейс **javax.servlet.ServletRequest**.
- Позволяет узнать характеристики запроса, в том числе значения параметров:
  - **String getParameter(String name)**
  - **Enumeration getParameterNames()**
  - и ряд других методов
- Позволяет получить доступ к потоку запроса:
  - **ServletInputStream getInputStream()**
  - **BufferedReader getReader()**
  - Можно вызвать только один из этих двух методов
- Позволяет получить параметры запроса
  - **int getContentLength()**

- **String getContentType()**
- **String getProtocol()**
- и т.д.

### Объект отклика

- Реализует интерфейс **javax.servlet.ServletResponse**
- Основные методы:
  - **ServletOutputStream** **getOutputStream()**  
Далее для вывода используются обычные средства бинарного вывода
  - **PrintWriter** **getWriter()**  
Далее для вывода используются обычные средства символьного вывода
  - Можно вызвать только один из этих двух методов.
- Позволяет установить параметры отклика
  - Для установки типа отклика используется метод **setContentType()**
  - Для установки кодировки используется метод **setCharacterEncoding()**
  - Сбрасывает текущий буфер клиенту **flushBuffer()**
  - Очищает буфер и сбрасывает статус **reset()**

### HTTP Servlet

- **javax.servlet.http.HttpServlet** - абстрактный класс, позволяющий создавать сервлеты, удобные для Web.
- Наследные классы должны переопределять хотя бы один из методов обработки запроса.
- Обычно это один из методов:
  - **void doGet(HttpServletRequest req, HttpServletResponse resp)**
  - **void doPost(HttpServletRequest req, HttpServletResponse resp)**
- Классы снабжаются аннотацией **@WebServlet**.

### Пример 59. Класс сервлета

```
import java.io.*;
```

```

import javax.servlet.*;
import javax.servlet.http.*;
import javax.servlet.annotation.*;

@WebServlet("/MyServlet.html")
public class MyServlet extends HttpServlet {
 protected void doGet(HttpServletRequest request,
 HttpServletResponse response) throws IOException {
 PrintWriter out = response.getWriter();
 try {
 out.print("<html><body>This is text document, which has "
 + "link"
 + " to another document</body></html>");
 }
 finally {
 out.close(); }
 }
 }
}

```

### Поддержка сессий

- В HTTP-сервлетах в объектах запроса и отклика добавляются дополнительные методы.
- Среди них появляются методы работы с сессиями, включая работу с **Cookies** (англ. печенье) - небольшой фрагмент данных, отправленный веб-сервером и хранимый на компьютере пользователя. Веб-клиент (обычно веб-браузер) всякий раз при попытке открыть страницу соответствующего сайта пересылает этот фрагмент данных веб-серверу в составе HTTP-запроса. Применяется для сохранения данных на стороне пользователя, на практике обычно используется для аутентификации пользователя, хранения персональных предпочтений и настроек пользователя, отслеживания состояния сеанса доступа пользователя, ведения статистики о пользователях.

- Для передачи значений между запросами можно использовать атрибуты сессии.

- **HttpSession.getAttribute(String name)**
- **HttpSession.setAttribute(String name, Object value)**
- И ряд других методов

#### **Пример 60.**

```
public class CashierServlet extends HttpServlet {
 public void doGet (HttpServletRequest request,
 HttpServletResponse response)
 throws ServletException, IOException {
 // Get the user's session and shopping cart
 HttpSession session = request.getSession();
 ShoppingCart cart = (ShoppingCart)session.getAttribute("cart");
 //...
 }
}

out.println("<a href=\"\" +
response.encodeURL(request.getContextPath() +
"/bookcatalog") + \"\">");
```

#### **Общий дескриптор развёртывания web.xml**

- Определяет параметры развёртывания web-приложения в контейнере.
- Некоторые из этих параметров даже не могут быть указаны с помощью аннотаций классов, т.к. относятся к web-приложению в целом.
- Указанные в дескрипторе параметры перекрывают параметры, указанные с помощью аннотаций классов.

#### **Пример 61. Дескриптор развертывания web.xml**

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app version="3.0" xmlns="http://java.sun.com/xml/ns/javaee"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```

xsi:schemaLocation="http://java.sun.com/xml/ns/javaee
http://java.sun.com/xml/ns/javaee/web-app_3_0.xsd">
 <listener>
 <listener-class>org.tempuri.MyListner</listener-class>
 </listener>
 <servlet>
 <servlet-name>MyServlet</servlet-name>
 <servlet-class>org.tempuri.MyServlet</servlet-class>
 </servlet>
 <servlet-mapping>
 <servlet-name>MyServlet</servlet-name>
 <url-pattern>/MyServlet.html</url-pattern>
 </servlet-mapping>
 <welcome-file-list>
 <welcome-file>/MyServlet.html</welcome-file>
 </welcome-file-list>
</web-app>

```

### Фильтрация

- Получаемые запросы и формируемые отклики могут быть подвергнуты фильтрации.
- API для классов фильтрации определен интерфейсами **Filter**, **FilterConfig**, **FilterChain** пакета **javax.servlet**.
- Включение фильтров осуществляется с помощью аннотации **@WebFilter**.
- Фильтры используются в основном для перехвата и изменения запросов и ответов от сервера. Рассмотрим случай, в котором проверяется сессия для каждого запроса пользователя, и если она верна, то только тогда пользователь может получить доступ к странице. Этого можно добиться путем проверки сессий на всех сервлет страницах (страницы JSP),

которые запрашивают пользователи, или можно сделать это с помощью фильтра.

### Пример 62. Фильтр

```
public class LogFilter implements Filter {
 public void doFilter(ServletRequest req, ServletResponse res,
 FilterChain chain) throws IOException, ServletException {
 HttpServletRequest request = (HttpServletRequest) req;
 String ipAddress = request.getRemoteAddr();
 System.out.println("IP " + ipAddress + ", Time " + new Date());
 chain.doFilter(req, res);
 }
 public void init(FilterConfig config) throws ServletException {
 //Get init parameter
 String testParam = config.getInitParameter("test-param");
 //Print the init parameter
 System.out.println("Test Param: " + testParam);
 }
 public void destroy() {
 //add code to release any resource
 }
}
```

- Метод **init()** используется для инициализации любого кода, который используется фильтром. Также отметим, что **init()** получит объект **FilterConfig**, который содержит различную информацию уровня фильтров, а также параметры инициализации, который передаются из **web.xml** (дескриптора развертывания).
- **doFilter()** фактически выведет информацию. Можно изменить этот метод и добавить код, который может изменить запрос / состояние сессии / ответ, добавить атрибут в запрос и т.д.

- **destroy()** вызывается контейнером, когда он хочет убрать мусор за фильтром. Это обычно делается, когда фильтр не используется в течение длительного времени и сервер хочет выделить память для других приложений.

### Пример 63. Маппинг фильтра в web.xml

```
<filter>
 <filter-name>LogFilter</filter-name>
 <filter-class>sample.LogFilter</filter-class>
 <init-param>
 <param-name>test-param</param-name>
 <param-value>This parameter is for testing.</param-value>
 </init-param>
</filter>
<filter-mapping>
 <filter-name>LogFilter</filter-name>
 <url-pattern>/*</url-pattern>
</filter-mapping>
```

## JSP (Java Server Pages)

**JSP** — это технология, которая упрощает создание web страниц с динамически изменяющимся (во время генерации страницы) содержимым.

```
<html>
<body>
<p>HI<%= (request.getParameter("name")!=null) ? " " +
request.getParameter("name"): "" %>!!!</p>
</body>
</html>
```

**JSP файл** – это текстовый документ, содержимое которого можно разбить на две части:

- статический текст (каркас) – создается при помощи специальных редакторов.
- динамический (генерируемый во время выполнения страницы) – обычно пишется в среде разработки программирования.

Принцип работы (рис. 27):

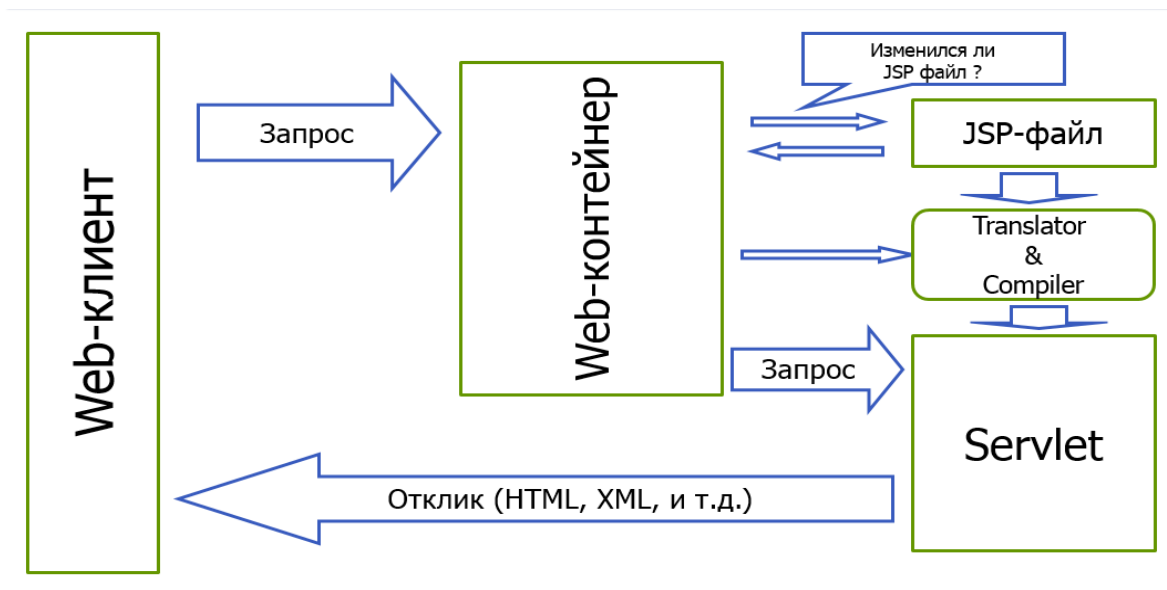


Рисунок 27 – Принцип работы

- JSP-файл транслируется в файл java.
- Процесс трансляции управляется директивами (`<%@ page [...]` `%>`).
- Полученный файл компилируется в класс сервлета.
- Далее с ним идет работа как с сервлетом.
- Конфигурация сервера может включать ряд настроек по работе с jsp-страницами.

### Обработка ошибок

- Во время трансляции и компиляции
  - поведение зависит от используемого сервера;
  - класс сервлета создан не будет.
- Во время выполнения

- если для данной JSP страницы (или приложения) определена «страница на случай ошибки» (errorPage), то будет выведена указанная страница;
- иначе – в зависимости от сервера и его настроек.

### **Виды JSP:**

- JSP-страница
  - Это страница из статических и динамических элементов.
  - Статическая часть имеет произвольный формат.
- JSP-документ
  - XML-файл, содержащий тэги особого вида.
  - Имеет ту же функциональность, что и JSP-страница, но требует представления в виде тэгов ряда элементов.

### **Создание страницы**

- Создание статического содержимого. Для этого удобно использовать редакторы HTML, XML и пр.
- Вставка динамических объектов и элементов сценариев JSP. В созданный файл вставляются элементы `<%? [...] %>`

### **Элементы JSP-страницы**

- Статический текст
- Элементы сценариев
  - Комментарии `<%-- [...] --%>`
  - Объявления `<%! [...] %>`
  - Скриплеты `<% [...] %>`
  - Выражения `<%= [...] %>`
  - Директивы `<%@ ? [...] %>`
- Стандартные объекты.

### **Комментарии**

- `<%-- [...] --%>`
- Содержат текст, который не будет включен в отклик страницы.
- Содержимое комментария игнорируется транслятором.

- Не путайте с комментариями HTML  
`<!-- [...] -->`

### **Пример 64. JSP-страница с комментариями**

JSP-страница

```
<html> <body>

Hello World!

<%-- This is a JSP comment --%>

<!-- This is a HTML comment -->

</body> </html>
```

Фрагмент примерного кода сервлета

```
out.write("<html> <body>\n Hello World!\n ");

out.write("\n <!-- This is a HTML comment -->"

+ "\n</body></html>\n");
```

### **Объявления**

- `<%! [...] %>`
- Обычно содержат объявления переменных, методов, классов и интерфейсов.
- При трансляции преобразуются в элементы класса сервлета (поля, методы и вложенные типы).
- Нельзя использовать поля для хранения значений, использующихся в рамках одного запроса.
- При работе с внутренними классами есть особенности.

### **Пример 65. JSP-страница с объявлениями**

JSP-страница

```
<%!

private String str = "Message: ";

public String createMessage(String arg) {
```

```

 return str + arg;
 }
%>

```

Фрагмент примерного кода сервлета

```

public final class index_jsp extends org.apache.jasper.runtime.HttpJspBase
 implements org.apache.jasper.runtime.JspSourceDependent {
 private String str = "Message: ";
 public String createMessage(String arg) {
 return str + arg;
 }
}

```

### **Скриптлеты**

- `<% [...] %>`
- Содержат динамически выполняемый код.
- Преобразуются во фрагменты методов сервлета, формирующих отклик.
- Синтаксически являются фрагментами обычных Java-методов.
- Каждая завершённая инструкция заканчивается знаком точки с запятой.

### **Пример 66. JSP-страница со скриптлетами**

JSP-страница

```

<html><body>
<% String server_name = System.getProperty("os.name");
 if (server_name == null)
 server_name = "Undefined."; %>
</body></html>

```

Фрагмент примерного кода сервлета

```

out.write("<html><body>\n");
String server_name = System.getProperty("os.name");
if (server_name == null)
 server_name = "Undefined.";
out.write("\n");
out.write("</body></html>\n");

```

## Выражения

- `<%= [...] %>`
- Содержит выражения, значения которых будут выведены в отклик.
- Синтаксически являются вычислимыми выражениями Java, для которых может быть получено текстовое представление.
- В конце выражений точки с запятой нет!

## Пример 67. JSP-страница с выражениями

JSP-страница

```
<html><body>
```

```
OS Name: <%=server_name%>
```

```

<%=createMessage("It works!")%>
```

```
</body></html>
```

Фрагмент примерного кода сервлета

```
out.write("<html><body>\n");
```

```
out.write("OS Name: ");
```

```
out.print(server_name);
```

```
out.write("\n");
```

```
out.write("
");
```

```
out.print(createMessage("It works!"));
```

```
out.write("\n");
```

```
out.write("</body></html>\n");
```

## Директива page

- `<%@ page [...] %>`
- Позволяет управлять процессом трансляции JSP-страницы в сервлет.
- Часто используемые параметры:
  - Тип отклика и кодировка.
  - Автоматическая инициализация сессии.
  - Импортируемые типы.
  - Страница для обработки ошибок:
    - Является ли данная страница обработчиком.

- Какая страница используется как обработчик.

### **Пример 68. Тип отклика, кодировка и импорт типов**

JSP-страница

```
<%@page contentType="text/html; charset=UTF-8"%>
```

```
<%@page session="true"%>
```

```
<%@page import="java.util.*, java.lang.reflect.*" %>
```

Фрагмент примерного кода сервлета

```
import java.util.*;
```

```
import java.lang.reflect.*;
```

```
...
```

```
response.setContentType("text/html; charset=UTF-8");
```

```
session = pageContext.getSession();
```

```
...
```

### **Возникновение и обработка ошибок**

### **Пример 69. Возникновение и обработка ошибок. JSP-страницы**

Основная страница

```
<%@page contentType="text/html; charset=UTF-8"%>
```

```
<%@page errorPage="errorMessage.jsp"%>
```

```
<html><body>
```

```
<% if(true) throw new Exception("Something happened!");%>
```

```
</body></html>
```

Страница-обработчик

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
```

```
<%@page isErrorPage="true" %>
```

```
<html><body>
```

*An error has occurred! Please, contact administrator.*

```
<%someLogMethod(exception);%>
```

```
</body></html>
```

### **Пример 70. Возникновение и обработка ошибок. Примерный код сервлетов**

Основная страница

```
try {
 pageContext = _jspxFactory.getPageContext(this, request,
 response, "errorMessage.jsp", true, 8192, true);
 if(true) throw new Exception("Something happened!");
} catch (Throwable t) {
 _jspx_page_context.handlePageException(t);
}
```

Страница-обработчик

```
Throwable exception =
 org.apache.jasper.runtime.JspRuntimeLibrary.getThrowable(request);
...
out.write("<html><body>\n");
out.write("An error has occurred! Please, contact administrator.\n");
someLogMethod(exception);
out.write("</body></html>\n");
```

### Директива **include**

- Предназначена для включения в формируемый отклик содержимого другого файла.
- Включение статическое: происходит на этапе трансляции.
- Включаться могут как статические, так и динамические ресурсы.
- Существует специальный вид JSP-страниц – JSP Fragments (JSPF).

### Пример 71. Директива **include**. JSP-страницы

Основная страница

```
<%@page contentType="text/html; charset=UTF-8"%>
<html><body>
<%@include file="WEB-INF/jspf/fragment.jspf" %>
</body></html>
```

Включаемая страница

```
<%@ page pageEncoding="UTF-8" %>
```

*Some fragment text*

## **Пример 72. Директива include.Примерный код сервлетов**

Основная страница

```
public final class index_jsp ... {
 private static java.util.Vector _jspx_dependants;
 static {
 _jspx_dependants = new java.util.Vector(1);
 _jspx_dependants.add("/WEB-INF/jspf/fragment.jspf");
 }
 public void _jspService(HttpServletRequest request,
 HttpServletResponse response)
 throws java.io.IOException, ServletException {
 ...
 out.write("<html><body>\n");
 out.write("Some fragment text\n");
 out.write("</body></html>\n");
 ...
 }
}
```

## **Стандартные объекты**

- **page**  
Ссылка на текущий объект (по сути – объект сервлета).
- **config**  
Имеет тип javax.servlet.ServletConfig, позволяет получить контекст сервлета и параметры сервлета.
- **application**  
Имеет тип javax.servlet.ServletContext, определяет контекст сервлета.
- **pageContext**  
Имеет тип javax.servlet.jsp.PageContext, определяет контекст jsp-страницы.

- **session**

Имеет тип `javax.servlet.http.HttpSession`, представляет текущую сессию.

- **request**

Имеет тип `javax.servlet.http.HttpServletRequest`, представляет текущий запрос.

- **response**

Имеет тип `javax.servlet.http.HttpServletResponse`, представляет текущий отклик.

- **out**

Имеет тип `javax.servlet.jsp.JspWriter`, представляет поток для вывода, по функциональности схож с классом `PrintWriter`.

**Пример 73. Стандартные объекты. Примерный код сервлета**

```
PageContext pageContext = null;
```

```
HttpSession session = null;
```

```
ServletContext application = null;
```

```
ServletConfig config = null;
```

```
JspWriter out = null;
```

```
Object page = this;
```

```
...
```

```
pageContext = _jspxFactory.getPageContext(this, request,
 response, null, true, 8192, true);
```

```
application = pageContext.getServletContext();
```

```
config = pageContext.getServletConfig();
```

```
session = pageContext.getSession();
```

```
out = pageContext.getOut();
```

**Тэги <jsp:...>**

- Являются еще одним средством управления ходом трансляции страницы.

- В основном предназначены для формата JSP-документов.

- Позволяют:
  - Описывать теги и их составляющие.
  - Перенаправлять обработку запросов.
  - Работать с компонентами JavaBeans.
  - Включать в код страницы обращения к апплетам.

### Тэги **<jsp:include>**

- `<jsp:include page="inclPage">`
- Включает в страницу статический или динамический ресурс.
- Включение происходит динамически, во время выполнения.

### Пример 74.

```
<jsp:include page="{relativeURL | <%= expression %>}"
 flush="true | false" >
 <jsp:param name="parameterName"
 value="{parameterValue|<%=expr%>}|${...}" />
</jsp:include>
```

JSP-страница

```
<html><body>
```

Before

```
<jsp:include page="simple.html"> </jsp:include>
```

After

```
</body></html>
```

Примерный код сервлета

```
out.write("<html><body>\n");
out.write("Before\n");
org.apache.jasper.runtime.JspRuntimeLibrary.include(request,
 response, "simple.html", out, false);
out.write("\n");
out.write("After\n");
out.write("</body></html>\n");
```

### Тэги **<jsp:forward>**

- `<jsp:forward page="inclPage">`
- Передает обработку запроса другому ресурсу.
- Могут быть указаны дополнительные параметры.

#### **Пример 75.**

```
<jsp:forward page="{relativeURL | <%= expression %>}">
 <jsp:param name="parameterName"
 value="{parameterValue|<%=expr%>}|%{...}" />
</jsp:forward>
```

JSP-страница

```
<html><body>
```

*Before*

```
<jsp:forward page="simple.html"></jsp:forward>>
```

*After*

```
</body></html>
```

Примерный код сервлета

```
out.write("<html><body>\n");
out.write("Before\n");
if (true) {
 _jspx_page_context.forward("simple.html");
 return;
}
out.write("\n");
out.write("After\n");
out.write("</body></html>\n");
```

#### **Взаимодействие с HTML-формами**

- Элементы HTML-управления должны находиться в форме.
- Для формы указывается адрес для перехода.
- Значения элементов ввода записываются в параметры запроса.
- В новой странице значения элементов ввода автоматически не заполняются.

- Элемент управления, активация которого привела к смене страницы, вносится в параметры запроса.

#### **Пример 76. Взаимодействие с HTML-формами.**

```

<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@page import="java.util.*" %>
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">
<html>

 <form name="SimpleForm" action="index.jsp">
 <input type="text" name="NameEditor" value="" size="30" />
 <input type="checkbox" name="SomeCheckBox" value="ON" />

 <input type="submit" value="Action 1" name="Action1Button" />
 <input type="submit" value="Action 2" name="Action2Button" />
 </form>

 <% Enumeration names = request.getParameterNames();
 if (names.hasMoreElements()) {
 String name = null;
 String value = null;%>
<table border="1">
 <thead>
 <tr>
 <th>Parameter name</th>
 <th>Parameter value</th>
 </tr>
 </thead>
 <tbody>
 <% while (names.hasMoreElements()) {
 name = (String) names.nextElement();
 value = request.getParameter(name);%>

```

```
<tr>
 <td><%=name%></td>
 <td><%=value%></td>
</tr>
<% }%>
</tbody>
</table>
<% } %>
</html>
```

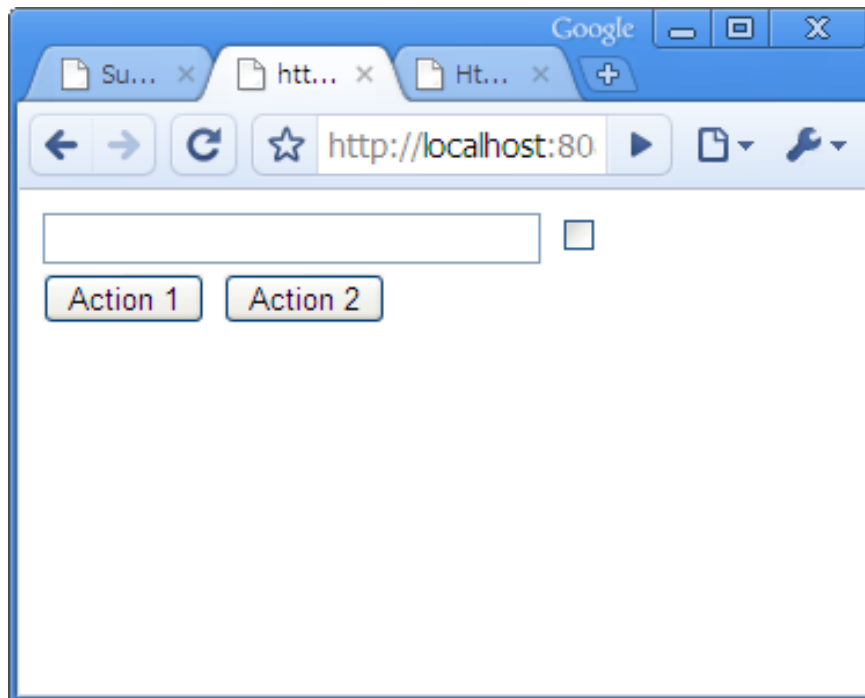


Рисунок 28 – Форма при первом запуске.  
Адрес: <http://host:8080/WebApplication1/index.jsp>

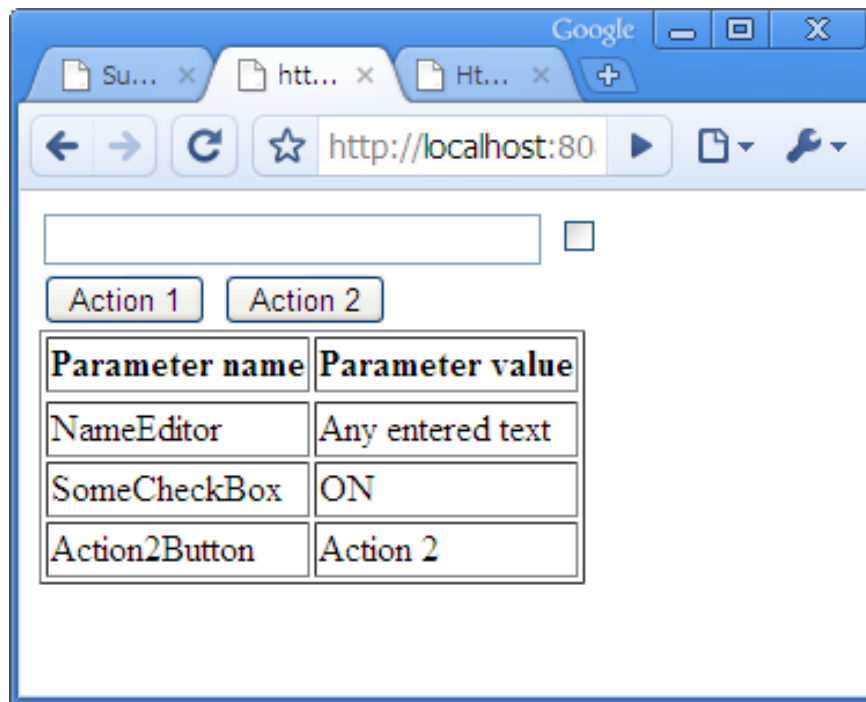


Рисунок 29 – Форма после нажатия кнопки.

Адрес: `http://localhost:8080/WebApplication1/index.jsp?NameEditor=Any+entered+text&SomeCheckBox=ON&Action2Button=Action+2`

### Недостатки раннего JSP:

- Смешение Java-кода и тегов страницы.
- Много «типовых» действий.
- «Некомфортность» работы на Java в ходе обработки запросов из-за:
  - Громоздкости кода.
  - Необходимости знания особенностей преобразования в сервлет.

### Основные идеи смены парадигмы:

- Альтернативный язык Expression Language.
- Использование библиотек тегов, особенно JSTL – JSP Standard Tag Library.
- Ориентация на компоненты в смысле JavaBeans и использование понятия свойств.
- Общие принципы остаются прежними.

### Пример 77. Как писали раньше

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
```

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
```

```

"http://www.w3.org/TR/html4/loose.dtd">
<html> <head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
<title>JSP Page</title> </head> <body>
<h1>Hello page</h1>

<% if (request.getParameter("name") != null) {
 out.print("Hello, ");
 out.print(request.getParameter("name"));
 out.println("!
");
} %>
<form name="oldform" action="old.jsp">
 What's your name?

 <input type="text" name="name" value="" size="50" />
 <input type="submit" value="Tell name" name="tell" />
</form> </body> </html>

```

### Пример 78. Как пишут теперь

```

<%@taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core"%>
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">
<html> <head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
<title>JSP Page</title> </head> <body>
<h1>Hello page</h1>

<c:if test="${!empty param.name}" var="val" scope="request">
 Hello, ${param.name}!

</c:if>
<form name="oldform" action="old.jsp">
 What's your name?

 <input type="text" name="name" value="" size="50" />

```

```
<input type="submit" value="Tell name" name="tell" />
</form>
</body> </html>
```

## **Expression Language**

- Является альтернативой выражениям и скриптелям.
- Динамическое чтение данных из:
  - JavaBeans-компонентов.
  - Неявных объектов.
  - Различных структур данных.
- Динамическая запись данных в:
  - Формы.
  - JavaBeans-компоненты.
- Вызов статических и публичных методов.
- Динамическое выполнение простых операций.

## **Виды выражений EL**

- По моменту вычисления:
  - Немедленное вычисление.
  - Отложенное вычисление.
- По цели выражения:
  - Обращение к данным.
  - Вызов метода.
- По режиму доступа:
  - Только чтение.
  - Чтение и запись данных.

## **Немедленное и отложенное вычисление**

- Немедленное вычисление (JSP и JSF):
  - `${ выражение }`
  - Позволяет лишь получить значение выражения.
- Отложенное вычисление (JSF):
  - `#{ выражение }`

- Выражение может использоваться различным образом в разные моменты жизненного цикла.
- Позволяет не только получать значения, но и записывать их (в т.ч. для свойств бинов).

### **Применение выражений с немедленным вычислением**

- Выражения EL могут применяться:
  - в статическом тексте
  - в качестве значений атрибутов тегов

### **Пример 79.**

*<some:tag>*

*some text \${expr} some text*

*</some:tag>*

*<some:tag value="\${expr}"/>*

*<some:tag value="some\${expr}\${expr}text\${expr}"/>*

### **Обращение к данным**

- Компоненты JavaBeans
  - `${beanName.propertyName}`
  - `${beanName["propertyName"]}`
- Коллекции
  - `${beanName.listName[i]}`
  - `${beanName.mapName["string key"]}`
- Перечислимые типы JavaSE
  - `${reference == "enumConstant"}`
- Неявные объекты
  - `${param["paramName"]}`
- Вычисление выражений
  - `${customer.age + 20}`

### **Вызов методов**

- Без параметров
  - `${object.method}`
  - `${object["method"]}`
- С параметрами
  - `${object.method(param1, ..., paramN)}`
  - `${object["method"](param1, ..., paramN)}`

### **Литералы в выражениях**

- Логические `true`, `false`
- Целочисленные `136`
- С плавающей точкой `3.14`, `1.0e10`
- Строковые `"string"`
- Ссылочные `null`

### **Операторы в выражениях**

- Доступ к элементам `[]`
- Операторные скобки `()`
- Унарные - `not` ! `Empty` (`Empty A` возвращает `true`, если `A` – ноль)
- Арифметические `*` `/` `div` `%` `mod` `+` `-`
- Сравнение `<` `>` `<=` `>=` `lt` `gt` `le` `ge` `==` `!=` `eq` `ne`
- Логические `&&` `and` `||` `or` `?` `:`

### **Неявные объекты**

- Объекты доступа к другим объектам
  - `pageScope`
  - `requestScope`
  - `sessionScope`
  - `applicationScope`
- Часто используемые объекты
  - `pageContext`
  - `session`

- request
- response
- param
- cookie
- ...

### Примеры выражений EL

Таблица 8 – Примеры выражений EL

Выражение	Результат
<code>\${(10*10) ne 100}</code>	false
<code>\${'a' &lt; 'b'}</code>	true
<code>\${1.2E4 + 1.4}</code>	12001.4
<code>\${!empty param.Add}</code>	true, если параметр Add не null и не пустая строка
<code>\${pageContext.request.contextPath}</code>	Путь контекста
<code>\${param['mycom.productId']}</code>	Значение параметра mycom.productId
<code>\${departments[deptName]}</code>	Значение элемента deptName в departments
<code>\${header["host"]}</code>	Имя хоста

### JSP-страница с применением EL

#### Пример 80. JSP-страница с применением EL

JSP-страница

```
${20 + 335}

```

```
${!empty param.Add}

```

Фрагмент примерного кода сервлета

```
out.write((java.lang.String)
```

```
org.apache.jasper.runtime.PageContextImpl.evaluateExpression(
 "${20 + 335}", java.lang.String.class,
```

```

 (PageContext)_jspx_page_context, null));
out.write("
\n");
out.write((java.lang.String)
 org.apache.jasper.runtime.PageContextImpl.evaluateExpression(
 "${!empty param.Add}", java.lang.String.class,
 (PageContext)_jspx_page_context, null));
out.write("
\n");

```

### **Custom tags**

- Определяются пользователем и описывают часто используемые операции.
- Описываются и распространяются в виде библиотек тегов, определяющих семейства тегов и содержащих объекты, реализующие теги.
- Для использования тегов необходимо:
  - Объявить библиотеку тегов
  - Сделать ее доступной для web-приложения

```

<prefix:tag attr1="value" ... attrN="value" />
<prefix:tag attr1="value" ... attrN="value" >
 body
</prefix:tag>

```

### **Библиотеки тегов**

- Библиотеки тегов описываются в tld-файлах
- Библиотеки могут быть
  - публичными и храниться в сети
  - локальными и храниться в каталоге WEB-INF
- Для подключения библиотеки необходимо использовать директиву **taglib**

```

<%@ taglib prefix="tlt" uri="/WEB-INF/iterator.tld"%>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core"%>

```

## Обработка custom tags

- В процессе компиляции jsp-страницы теги будут преобразовываться в java-код в соответствии с описанием в библиотеке тегов.
- Если соответствующая библиотека не была подключена, относящиеся к ней теги будут трактоваться как статическая часть jsp-страницы и просто выводиться в отклик.

### Пример 81. JSP-страница с применением custom tags

JSP-страница

```
<c:if test="${!empty param.name}" var="nameExists" scope="request">
 Hello, ${param.name}!

</c:if>
```

Фрагмент примерного кода сервлета

```
org.apache.taglibs.standard.tag.rt.core.IfTag _jspx_th_c_if_0 =
 (org.apache.taglibs.standard.tag.rt.core.IfTag)
 _jspx_tagPool_c_if_var_test_scope.get(
 org.apache.taglibs.standard.tag.rt.core.IfTag.class);
_jspx_th_c_if_0.setPageContext(_jspx_page_context);
_jspx_th_c_if_0.setParent(null);
_jspx_th_c_if_0.setTest(((java.lang.Boolean)
 org.apache.jasper.runtime.PageContextImpl.evaluateExpression(
 "${!empty param.name}", java.lang.Boolean.class,
 (PageContext)_jspx_page_context, null)).booleanValue());
_jspx_th_c_if_0.setVar("nameExists");
_jspx_th_c_if_0.setScope("request");
int _jspx_eval_c_if_0 = _jspx_th_c_if_0.doStartTag();
if (_jspx_eval_c_if_0 != javax.servlet.jsp.tagext.Tag.SKIP_BODY) {
 do {
 out.write("Hello, ");
 out.write((java.lang.String)
 org.apache.jasper.runtime.PageContextImpl.evaluateExpression(
 "${param.name}", java.lang.String.class,
 (PageContext)_jspx_page_context, null)).toString());
 out.write("!");
 } while (_jspx_th_c_if_0.doEndTag() == javax.servlet.jsp.tagext.Tag.SKIP_BODY);
}
```

```

"${param.name}", java.lang.String.class,
(PageContext)_jspx_page_context, null));
out.write("!
\n");
int evalDoAfterBody = _jspx_th_c_if_0.doAfterBody();
if (evalDoAfterBody !=
javax.servlet.jsp.tagext.BodyTag.EVAL_BODY_AGAIN)
break;
} while (true);
}

```

### JSP Standard Tag Library

- Библиотека включает в себя набор тегов, функциональность которых часто используется в web-приложениях.
- Эти теги должны обрабатываться единообразно в рамках всех контейнеров.
- Это позволяет:
  - избежать использования различных библиотек.
  - единообразно организовывать JSP-страницы .

### Библиотеки JSTL

- JSTL поставляется в виде набора библиотек:
  - Различные направления функциональности
  - Различные пространства имен
- Библиотеки и соответствующие пути
  - Core <http://java.sun.com/jsp/jstl/core>
  - XML <http://java.sun.com/jsp/jstl/xml>
  - Internationalization <http://java.sun.com/jsp/jstl/fmt>
  - Database <http://java.sun.com/jsp/jstl/sql>
  - Functions <http://java.sun.com/jsp/jstl/functions>

**Таблица 9 – Библиотеки JSTL**

Библиотека	Функциональность	Префикс
------------	------------------	---------

Core	Работа с переменными	с
	Управление ходом выполнения	
	Работа с URL	
	Разное	
XML	Базовые	х
	Управление выполнением	
	Преобразования	
I18n	Локализация	fmt
	Форматирование сообщений	
	Форматирование чисел и дат	
Database	SQL	sql
Functions	Возможности коллекций	fn
	Работа со строками	

**Таблица 10 – Библиотека Core**

<b>Функциональность</b>	<b>Тэги</b>
Работа с переменными	<b>remove</b> <b>set</b>
Управление ходом выполнения	<b>choose (when,</b> <b>otherwise)</b> <b>forEach</b> <b>forTokens</b> <b>if</b>

Работа с URL	<b>import (param)</b> <b>redirect (param)</b> <b>url (param)</b>
Разное	<b>catch</b> <b>out</b>

**Таблица 11 – Библиотека XML**

<b>Функциональность</b>	<b>Тэги</b>
Базовая	<b>out</b> <b>parse</b> <b>set</b>
Управление ходом выполнения	<b>choose (when, otherwise)</b> <b>forEach</b> <b>if</b>
Преобразования	<b>transform (param)</b>

**Пример 82. Пример использования**

```

<c:if test="${!empty param.Add}">
 <c:set var="bid" value="${param.Add}"/>
 <jsp:useBean id="bid" type="java.lang.String" />
 <sql:query var="books"
 dataSource="${applicationScope.bookDS}">
 select * from PUBLIC.books where id = ?
 <sql:param value="${bid}" />
</sql:query>
<c:forEach var="bookRow" begin="0" items="${books.rows}">
 <jsp:useBean id="bookRow" type="java.util.Map" />

```

```
<jsp:useBean id="addedBook" class="database.Book"
 scope="page" />
...
<% cart.add(bid, addedBook); %>
...
</c:if>
```

## Язык JavaScript

JavaScript – это язык программирования

- Интерпретируемый
- Объектно-ориентированный
- Со слабой типизацией
- С динамической типизацией
- С автоматическим управлением памятью
- Вместо наследования классов используются прототипы объектов
- Функции являются объектами

### Области применения

- Скриптовые фрагменты серверных приложений.
- Виджеты (примитив графического интерфейса пользователя, имеющий стандартный внешний вид и выполняющий стандартные действия).
- Прикладное программное обеспечение.
- Букмарклеты (небольшие приложения, размещаемые в закладках браузера).
- Пользовательские скрипты в браузерах.
- Программы на стороне клиента в web-приложениях.
  - Код встраивается в HTML-страницу.
  - Код выполняется браузером.
  - Возможно изменение структуры страницы, её элементов и их параметров.
  - Изменения могут происходить без перезагрузки страницы.
  - Могут выполняться дополнительные действия.

- Взаимодействие с браузером
- Взаимодействие с сервером
- Бизнес-логика в целом

### Пример 83. Здравствуй, мир!!!

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01
Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd">
<html>
 <head>
 <title>Проверка JS</title>
 </head>
 <body>
 <h1>
 <script>
 document.write("Hello, world!!!");
 </script>
 </h1>
 </body>
</html>
```

### Размещение в HTML-документе

- Тег <script>

```
<html>
 <body>
 <p id="demo">This is a paragraph.</p>
 <script type="text/javascript">
 document.getElementById("demo").innerHTML=Date();
 </script>
 </body>
</html>
```

- Атрибуты тегов (обработка событий)

`<a href="delete.jsp" onclick="return confirm('Вы верены?');">Удалить</a>`

- Гипертекстовые ссылки

`<a href="javascript:document.mainform.submit();">Отправить</a>`

## Тег `<script>`

- Указание языка
  - По умолчанию – именно JavaScript
  - В явном виде – с помощью атрибута
    - Устаревший вариант  
**`language="JavaScript"`**
    - Современный вариант  
**`type="text/javascript"`**
- Две формы кода
  - Код размещается внутри тега
  - Код размещается в отдельном файле (обычно с расширением **js**), файл указывается с помощью атрибута **src**
  - Второй способ «перекрывает» первый
- Типичное использование
  - В заголовке документа – для описания функций и предварительных действий
  - В теле документа – для условной генерации документа

## Обработка браузером

- Обычно код интерпретируется
  - Возможны Just-In-Time решения
- Код выполняется по мере обнаружения
  - Обращаться к элементам документа, описанным в документе позднее – не лучшая мысль
  - Однако обращаться к позднее описанным сущностям из функций можно, если сущности уже будут существовать на момент выполнения функции

- Возможно возникновение ошибок в ходе выполнения
  - Блок скрипта не завершит работу
  - В консоли браузера появится сообщение об ошибке

### Замечания

- JavaScript чувствителен к регистру! (HTML – нечувствителен к регистру, хотя есть рекомендации).
- Соседство атрибутов HTML и событий/методов JavaScript может приводить к неожиданным результатам (например, onClick и onclick).
- Функции выполняются при явном вызове.
- Просто код в теге <script> выполняется по мере обнаружения.
- Код, указанный в атрибутах тегов как обработка событий, выполняется при возникновении событий.
- Код, указанный в ссылках, выполняется при срабатывании ссылки.

### Комментарии

#### Пример 84. Комментарии

```
<html>
<body>
 <script>
 document.write("Это будет выведено.");
 // document.write("А это не будет выведено...");
 /*
 document.write("И это не будет выведено!");
 document.write("И это тоже не будет, надо же...");
 */
 <!-- document.write("И это не будет?!!");
 </script>
</body>
</html>
```

- Однострочные

- От символов // до конца строки
- От символов <!-- до конца строки

```
<script type="text/javascript">
```

```
<!--
```

```
document.write("Вы не увидите никаких следов кода,");
```

```
document.write("если ваш браузер не поддерживает JavaScript!!!");
```

```
//-->
```

```
</script>
```

- Многострочные
  - Начало /\*
  - Конец \*/
  - Не могут быть вложенными

## Литералы и переменные

- Литералы
  - Числовые
 

10 010 0x10 1.1 .1 1. 1e1
  - Строковые
 

"Строка" 'Строка' "I'm a string" \" \"\""
- Переменные
  - Могут объявляться с помощью ключевого слова **var**
    - `var a = 5, b = "лет";`
    - Если объявление вне функции – переменная глобальная, иначе – локальная
  - Могут объявляться и без ключевого слова **var**
    - `c = "И так тоже можно"`
    - Переменная всегда глобальная...
  - Объявление переменных «на лету» + их глобальность + чувствительность к регистру = источник ошибок
  - Тип переменной определяется её содержимым

● Приведение типов автоматическое

**Таблица 12 – Операторы**

Приоритет	Оператор	Запись
1	Доступ к элементу	. []
	Создание объекта	new
2	Вызов функции	()
3	Инкремент (две формы)	++
	Декремент (две формы)	--
4	Логическое отрицание	!
	Побитовое отрицание	~
	Унарный плюс	+
	Унарный минус	-
	Тип объекта	typeof
	Вычисление выражения без возврата значения	void
	Удаление свойства объекта	delete
5	Умножение	*
	Деление	/
	Остаток от деления	%
6	Сложение, конкатенация	+
	Вычитание	-

7	Побитовые сдвиги	<< >> >>>
8	Сравнение	< <= > >=
	Проверка наличия свойства в объекте	in
	Проверка «типа» объекта	instanceof
9	Равенство и строгое (с типом) равенство	== != === !==
10	Побитовое И	&
11	Побитовое исключающее ИЛИ	^
12	Побитовое ИЛИ	
13	Логическое И	&&
14	Логическое ИЛИ	
15	Тернарный условный оператор	?:
16	Присваивания	= += -= *= /= %= <<= >>= >>>= &= ^=  =
17	Разделитель последовательности выражений	,

### Управление порядком выполнения

- Операторные скобки
  - {...}
- Ветвление

- if (условие) инструкция;
- if (условие) инструкция; else инструкция;
- Нелогические значения 0, "", null, undefined и NaN считаются ложью, всё остальное – истиной
- Цикл с предусловием
  - while(условие) инструкция;
- Цикл с постусловием
  - do { ... } while(условие);
- Итерационный цикл
  - for(инициализация; условие; действие) инструкция;
- Цикл по элементам
  - for(переменная in объект) инструкция;
- Прерывание блока
  - **break;**
  - **break имяМетки;**
- Переход на следующую итерацию
  - **continue;**
  - **continue имяМетки;**
- Возврат из метода или обработчика
  - **return;**
  - **return значение;**
- Блок переключателей
  - **switch** (выражение) {
    - case** значение: инструкции;
    - ...
    - default:** инструкции}
  - Для сравнения используется строгое равенство
  - Можно работать со строками
  - Предложений **case** может быть много
  - Для прерывания используется **break**

- Есть предложение **default**

## Функции

- Объявление функции (function declaration)  
**function имя(парамп1, ... параметрN) {**  
**тело** **функции**  
**}**

- Формальные и фактические параметры различаются
- Внутри функции объявление переменных с `var` даёт локальные переменные
- Функции создаются предварительно, до выполнения кода

```
writeLine("Итого:");
writeLine(745);
function writeLine(line) {
 document.write(line);
 document.write("
");
}
```

- Функции – это объекты, у которых есть имя и значение, и которые можно передавать в функции

```
writeLine(writeLine);
/* Будет выведено следующее:
function writeLine(line) { document.write(line); document.write("
"); }
*/
```

- Функция-выражение (function expression)  
**function(параметр1, ..., параметрN) {**  
**тело** **функции**  
**}**

- Полученную функцию можно присвоить в переменную.
- Функции создаются не заранее, а когда до них доходит выполнение.

- Условное объявление функций (переменная объявляется заранее, реализация – потом).
- Лучше такой формой не злоупотреблять.
- Можно вызывать функции, полученные в качестве объектов, и передавать им параметры

### **Пример 85.**

```
function showResult(a, b, calculator) {
 document.write("Операнд 1: " + a + "
");
 document.write("Операнд 2: " + b + "
");
 document.write("Результат: " + calculator(a, b) + "
");
}

function sum(a, b) {
 return a + b;
}

showResult(5, 7, sum); //
showResult(8, 3, function (a, b) { return a - b; });
```

### **Объекты**

- Являются по сути ассоциированными массивами (картами, map)
  - ключи – имена свойств (только строки)
  - значения – значения свойств (любые типы, включая функции)
- Доступ к свойствам
  - ссылка.имяСвойства
  - ссылка["имяСвойства"]
- Свойства добавляются и исключаются динамически.
- Переменные хранят ссылки на объекты.
- Свойства объектов являются переменными.

### **Пример 86. Операции со свойствами**

```
var employee = {}; // Создание пустого объекта
employee.name = "King"; // Добавление свойств
```

```
employee.salary = 5000;
document.write(employee.name + ": " + employee.salary); // Чтение свойств
delete employee.salary // Удаление свойства
for (p in employee) { // Ещё одно чтение свойств
 document.write(typeof(p) + " " + p + " " + employee[p]);
}
/* Будет выведено
King: 5000
string name King
/*
// Литеральное создание объекта
var rectangle = {
 width: 200,
 height: 300,
 getArea: function() { return this.width * this.height; }
}; // this в методах при обращении к свойствам обязателен!!!
// Изменение свойств
rectangle.width = rectangle.height = 5;
rectangle.getPerimeter = function() {
 return 2 * (this.width + this.height);
};
// Проверка работы
document.write(rectangle.getArea() + "
");
document.write(rectangle.getPerimeter() + "
");
```

## **Функции-конструкторы**

- Конструкторами объектов являются функции
  - Их принято называть с большой буквы
  - Вызываются с помощью new

- Могут иметь параметры
- Формируют объект, используя слово `this`
- Возвращают ссылку на созданный объект

### Пример 87.

```
function Employee(name, salary) {
 this.name = name;
 this.salary = salary;
 this.fired = false;
}

var king = new Employee("King", "5000");
```

### Встроенные объекты

- Конструкторы
  - Math
  - Date
  - RegExp
  - Function
  - Array
  - ...
- «Обёртки»
  - String
  - Number
  - Boolean

### Наследование

- Базовым является не наследование классов, а наследование объектов.
- Родительский объект называется прототипом.
- Механизмы наследования заметно отличаются от классического ООП с классами.

- Можно эмулировать почти что классические классы со всеми их возможностями и проблемами.

## Обработка исключений

- Обработка исключений

```
try {
 // код, потенциально выбрасывающий исключения
}
catch (e) {
 // код обработки ошибки
}
finally {
 // код, всегда выполняющийся в конце
}
```

- Блока **catch** или **finally** может не быть.
- Для непользовательских методов ошибка обычно «имеет тип» **Error**, свойства которого могут отличаться в зависимости от браузера.

## Выбрасывание исключений

- Метод `throw` вправо выбросить своё исключение  
`throw ссылкаНаОбъектИсключения;`
- Можно `throw` выбросить вообще любой объект  
`throw 12345;`
- Но лучше выбрасывать что-нибудь вразумительное  
`throw {name: "OhShit!", message: "Ohhhh"};`  
`throw new Error("Oooops!");`

### Пример 88. Смена контекста

#### Смена текущего объекта

```
var employee = {name: "King", salary: 5000};
with (employee) {
 document.write(name + ": " + salary);
}
```

#### Смена контекста **this** у функции/метода

```
function raiseSalary(addition, factor) {
 this.salary = addition + this.salary * factor;
}

var petrov = {position: "Salesman", salary: 1000};
var president = {name: "King", salary: 5000};
raiseSalary.call(petrov, 100, 1.1);
raiseSalary.apply(president, [0, 1.2]);
```

### Пользовательские массивы

- Одномерные
  - Но можно создать массив массивов, в т.ч. непрямоугольный.
  - Обращение по индексу с помощью оператора `[]`.
  - Нумерация элементов с 0.
  - Есть поле **length**, хранящее количество элементов.
- Способы создания
  - Пустой массив  
**var a1 = new Array();**
  - Массив с заданным количеством элементов  
**var a2 = new Array(10);**
  - Массив с заданными элементами  
**var a3 = new Array(10, "и это не длина", 5.5, '!');**
  - Литеральная форма с заданными элементами  
**var a4 = [10, "и это не длина", 5.5, '!];**
- Динамические
  - Изменение значения **length**
  - Добавление новых элементов
  - Явно не указанные элементы получают значение **undefined**

### Пример 89.

```
<script>
```

```

var a = [1, 2];
a[5] = 5;
document.write(a[4] + "
");
document.write(a[5] + "
");
a.length = 2;
a.length = 5;
document.write(a[4] + "
");
document.write(a[5] + "
");
</script>

```

### Сортировка пользовательских массивов

- Выполняется с помощью метода **sort()** массива.
- По умолчанию – это сортировка в лексикографическом порядке.
- Если требуется другой порядок, то следует задать свой критерий сравнения в виде функции.

### Пример 90

```

<script>
var a = [1, 2, 15, 23];
a.sort();
document.write(a + "
"); // 1,15,2,23
function compareNumeric(a, b) {
 return a - b;
}
a.sort(compareNumeric);
document.write(a + "
"); // 1,2,15,23
</script>

```

### Дополнительные методы пользовательских массивов

Инвертирование порядка элементов	<b>reverse()</b>
Добавление элемента в конец	<b>push(element)</b>

Удаление элемента из конца	<b>pop()</b>
Добавление элемента в начало	<b>unshift(element)</b>
Удаление элемента из начала	<b>shift()</b>
Объединение в строку с указанием разделителя	<b>join(separator)</b>
Удаление и вставка элементов	<b>splice(start, [deleteCount, add1, ..., addN])</b>
Копирование части массива	<b>slice(begin, end)</b>

### Пример 91

```

<html>
 <body>
 <script>
 var a = [1, 2, 15, 23];
 a.reverse();
 document.write(a + "
"); // 23,15,2,1
 a.push("добавка 1");
 document.write(a + "
"); // 23,15,2,1,добавка 1
 a.pop();
 document.write(a + "
"); // 23,15,2,1
 a.unshift("добавка 2");
 document.write(a + "
"); // добавка 2,23,15,2,1
 a.shift();
 document.write(a + "
"); // 23,15,2,1
 //...
 // ...

 var str = a.join("; ");
 document.write(str + "
"); // 23; 15; 2; 1
 var b = str.split("; ");

```

```
document.write(b + "
"); // 23,15,2,1
a.splice(2, 1, "Замена");
document.write(a + "
"); // 23,15,Замена,1
var c = a.slice(1, a.length);
c[0] = "И что будет?";
document.write(a + "
"); // 23,15,Замена,1
document.write(c + "
"); // И что будет?,Замена,1
</script>
</body>
</html>
```

### **Виды объектов**

- Встроенные
  - По сути – библиотеки и базовые объекты
- Пользовательские
  - Всё, что создаёт пользователь-программист
- Серверные
  - Определяют и предоставляют взаимодействие с сервером
- Клиентские
  - Определяют и предоставляют взаимодействие с браузером и документом

### **BOM и DOM**

- Browser Object Model
  - Объектная модель для взаимодействия с браузером
  - В настоящий момент не стандартизована
  - Базовый объект – **window**
- Document Object Model
  - Объектная модель для взаимодействия с документом
  - В целом стандартизирована

- Базовый объект – **document**

### **Объект window**

- Глобальный объект.
- Все объявляемые переменные и объекты становятся его свойствами.
- Содержит либо напрямую информацию о документе, либо ассоциированный массив (карт, map) фреймов (frames).
- Имеет свои свойства, методы и события.

### **Пример 92. Взаимодействие с пользователем**

*// Окно с сообщением*

*alert("Случилось страшное!!!");*

*// Окно с запросом на подтверждение*

*// Возвращает true или false*

*confirm("Хотите поговорить об этом?..");*

*// Окно со вводом строки*

*// Возвращает введенную строку*

*prompt("Вам слово!", "говорите сюда");*

### **Создание и закрытие новых окон**

- Создание
  - Параметры задаются строкой
  - Поведение зависит от браузера
  - Возвращается ссылка на объект окна

```
var ncwin = window.open('http://www.netcracker.com',
'Сайт компании', 'directories=no,height=300,location=no,' +
'menubar=no,resizable=yes,scrollbars=yes,' +
'status=no,toolbar=no,width=300');
```

- Закрытие

```
ncwin.close();
```

### **Запуск новых потоков**

- Запуск нового потока
  - *setTimeout("код", времяВМиллисекундах)*
  - Возвращает ссылку на поток
- Остановка запущенного разового потока
  - *clearTimeout(ссылкаНаПоток)*
- Периодический запуск нового потока
  - *setInterval("код", времяВМиллисекундах)*
  - Возвращает ссылку на поток
- Остановка запущенного периодического потока
  - *clearInterval(ссылкаНаПоток)*

### Свойства window

- **location** – объект «типа» URL, текущий адрес
  - **href** – собственно URL
  - Набор свойств, соответствующих фрагментам URL
  - Методы
  - **replace()**
  - **reload()**
- **history** – объект, хранящий историю переходов по страницам
  - **forward()**
  - **back()**
  - **go(количествоСтраниц)**
- **navigator** – объект, хранящий данные о браузере
- **opener** – ссылка на родительское окно

### Объект document

- В нём и живёт DOM.
- Это тоже ассоциированный массив специфического вида.
- Все теги получают в соответствие объект.

- Имя объекта определяется значением атрибута **name** тега.
- Атрибуты тегов становятся свойствами соответствующих объектов.
- Не все свойства объектов соответствуют атрибутам тегов.
- Объекты формируются по ходу чтения документа браузером.
- Некоторые виды объектов объединяются в дополнительные массивы (формы, ссылки и т.д.).
- У объектов могут быть события.
- У объектов могут быть дополнительные методы.

### Прямая запись в документ

- Методы
  - **document.write(значение)**
  - **document.writeln(значение)**
- Используются для генерирования (в т.ч. условного) содержимого документа.
- Можно использовать только если документ ещё «открыт».
  - Для загружаемых документов – документ ещё не до конца прочитан браузером.
  - Для документов в динамически открытых окнах – после вызова **document.open()** и до вызова **document.close()**.
- Если документ уже закрыт, то его можно изменять только через объекты и управление ими.

### События объектов

- Обычно именуются **ончто-нибудь**.
- Обработчик можно указать прямо в HTML-коде.
- Обработчик можно задать программно.
- Возврат из некоторых обработчиков **null** означает прекращение обработки.

### Пример 93.

```

<html>

<body>

 <p
onMouseOver="document.getElementById('addition').innerHTML = 'А он
есть...'"
onMouseOut="document.getElementById('addition').innerHTML = "">
 Видишь суслика?..</p>
 <p id="addition"></p>

 <script>
 document.getElementById("picture").onclick = function() {
 alert("Вот же он, суслик!");
 };
 </script>
</body>
</html>

```

#### **Пример 94. Динамическое изменение DOM**

```

<html>
<body>
 <table border="1px" id="table">
 <thead id="head"> <tr> <th>Фамилия</th> <th>Имя</th> </thead>
 <tbody> <tr> <td>Иванов</td> <td>Иван</td> </tr>
 </table>
 <form name="mainform">
 Фамилия:
 <input type="text" name="sname"/>

 Имя:
 <input type="text" name="fname"/>

 <input type="submit" name="addition" value="Добавить"

```

```
onClick="appendToTable(document.mainform.fname.value,
document.mainform.sname.value); return false;"/>
```

```
</form>
```

```
<script>
```

```
function appendToTable(fname, sname) {
 var table = document.getElementById("table");
 var row = document.createElement("tr");
 var surname = document.createElement("td");
 var firstname = document.createElement("td");
 surname.innerHTML = sname;
 firstname.innerHTML = fname;
 row.appendChild(surname);
 row.appendChild(firstname);
 table.appendChild(row);
}
```

```
</script>
```

```
</body>
```

```
</html>
```

## Работа с формами

- Доступ к форме:
  - По имени как к свойству документа
  - В массиве **document.forms**
- Доступ к элементам:
  - По имени как к свойству формы
  - В массиве **elements** формы
- Атрибуты тегов элементов формы и самой формы — свойства соответствующих объектов.
- События формы:
  - **onsubmit**

- **onreset**
- Методы формы
  - **submit()**
  - **reset()**

### Программирование гиперссылок

- Доступ к гиперссылкам:
  - По имени/идентификатору
  - В массиве **document.links[]**
  - Объект имеет «тип» URL
- Объект ссылки имеет свойства и события
- Ссылка может иметь программный вид
  - **javascript:код**
  - результат работы кода может быть показан в браузере
  - Результатом считается результат последнего выражения
  - Приём **void(0);**
  - Так можно не только в ссылке, но и в **action** у форм

### Пример 95. Использование ссылок

```
<html>
<body>
 <table border="1px" id="table">
 <thead id="head"> <td>Фамилия</td> <td>Имя</td> </thead>
 <tr> <td>Иванов</td> <td>Иван</td> </tr>
 </table>
 <form name="mainform">
 Фамилия: <input type="text" name="sname"/>

 Имя: <input type="text" name="fname"/>

 </form>
```

```
<a href="javascript:appendToTable(document.mainform.fname.value,
 document.mainform.sname.value);
 void(0);">Добавить
```

```

 Отправить данные
```

### **Некоторые замечания**

- Часто JavaScript воспринимается как набор приёмов по программированию клиентской части.
- Это не так.
- JavaScript – интересный и богатый язык с широкими возможностями.
- И детальное его изучение требует значительного времени...

## HTML (самостоятельно)

**HTML (HyperText Markup Language)** – язык разметки гипертекста. Его предназначение - указывать браузеру, как разместить элементы на странице при ее отображении на экране монитора.

Например,

`<b>Этот текст будет отображаться жирным</b>`

`<i>А этот - курсивом</i>`

Бывают также теги одиночные, т.е. не требующие закрытия. Такие теги просто совершают определенное действие, например, перевод строки или вставку рисунка.

Теги могут иметь параметры (или атрибуты), например,

`<body bgcolor="red">Здесь содержимое</body>`

### Структура HTML

#### Пример 30.

```
<html>
<head>
 <title>Тег body в html</title>
 <base href="http://www.my_site.ru/">
 <link rel="stylesheet" type="text/css" href="style.css">
 <meta http-equiv="Content-Type" content="text/html; charset=windows-
1251">
 <meta http-equiv="KEYWORDS" content="тег, html, заголовок
страницы">
 <meta http-equiv="DESCRIPTION" content="Описание элементов
заголовка">
 <script type="text/javascript" src="function.js"> </script>
</head>
<body bgcolor="yellow" leftmargin="100" topmargin="50"
rightmargin="50" bottommargin="50" text="gray" alink="red" vlink="green">
 Просто текст
```

<br/>

<a href="index.html">Ссылка на страницу index1.html</a>

<br/>

<a href="map.html">Ссылка на страницу index2.html</a>

</body>

</html>

Теги <html> </html> являются контейнером для всех остальных, т.е. в них помещаются все остальные. Таким образом, ваш документ должен начинаться с тега <html>, а заканчиваться тегом </html>.

Сам документ условно разделен на две части - заголовок документа (теги <head> </head>) и тело документа (теги <body> </body>). Заголовок документа содержит служебную информацию и не влияет на внешний вид документа. Его задачей является предоставление браузеру пользователя или серверу информации о том, как отобразить ваш документ.

Единственным обязательным элементом заголовка документа являются теги <title> </title>. Они необходимы, чтобы дать документу название, оно отражается в заголовке окна браузера.

Одиночный тег <base> служит для указания полного URL-адреса документа. Зачем это нужно? Представьте, что вы сохранили какую-нибудь html-страницу себе на компьютер, с тем, чтобы просмотреть ее позже. Все картинки на этой страницы превратятся в красные крестики. Но если вы не отключены от сети, а на странице присутствует тег <base>, то браузер будет знать, где искать необходимый файл, найдет его и загрузит картинки.

Одиночный тег <link> необходим для подключения внешних файлов. Например, если вы будете использовать каскадную таблицу стилей, то ее удобнее хранить в отдельном файле и подключать этот файл ко всем страницам сайта.

У тега <link> несколько атрибутов:

*href* - указывает URL-адрес подключаемого файла.

*rel* - указывает на тип отношения данного документа к внешнему (например: *rel="stylesheet"* указывает, что внешний файл определяет стиль текущего документа).

*type* - указывает тип и параметры присоединенной таблицы стилей.

Информация в этом теге не имеет никакого отношения к HTML, однако ее использование очень удобно для решения ряда задач:

- указание кодировки страницы, например, для русского текста в кодировке Windows;
- указание ключевых слов страницы (используется при оптимизации страниц);
- указание краткого описания страницы.

Возможны и другие варианты. Рассмотрим параметры тега *<meta>*:

*http-equiv* - определяет свойство тега (тип контента, ключевые слова, описание и т.д.).

*name* - используется для дополнительного описания тега, если отсутствует, то считается эквивалентным параметру *http-equiv*.

*content* - значение параметра *http-equiv*.

Теги *<script></script>* используются для подключения внешних файлов скриптов. Это позволяет оптимизировать код страниц. Например, если вы используете функции *java script* для большинства своих страниц, то поместив эти функции на отдельную страницу - *function.js*, с помощью тега *<script>* можно указать путь к этой странице.

Это повышает читабельность кода и ускоряет загрузку страниц. У этого тега несколько параметров:

*language* - указывает язык написания скрипта, в спецификации HTML 4.0 данный параметр не рекомендуется к употреблению. Вместо него следует указывать параметр *type*.

*type* - указывает тип MIME для языка.

*src* - указывает путь к внешнему файлу со скриптами.

Все, что отображается на web-странице, находится в тегах `<body></body>`. Это текст, картинки и исполняющиеся скрипты, а также теги для оформления всего этого.

Обязательных параметров у тега `<body>` нет, да и применение необязательных параметров тоже не приветствуется. Тем не менее, большинство параметров до сих пор поддерживается разными браузерами. Рассмотрим те, которые пока поддерживаются всеми браузерами:

*alink* - устанавливает цвет активной ссылки. Текущий цвет ссылки меняется на активный при нажатии на нее.

*vlink* - устанавливает цвет посещенной ссылки, т.е. той, по которой уже щелкали.

*background* - указывает на изображение, которое будет использоваться в качестве фонового рисунка. Этот рисунок заполняет собой все видимое пространство окна. Если рисунок меньше окна браузера, то он повторяется, образуя мозаику из одинаковых картинок. На стыке этих картинок возникают видимые переходы. Поэтому к подбору фоновых рисунков следует подходить с большим вниманием.

*bgcolor* - указывает фоновый цвет документа.

*leftmargin* - определяет отступ от левого края окна браузера до контента страницы.

*rightmargin* - определяет отступ от правого края окна браузера до контента страницы.

*topmargin* - определяет отступ от верхнего края окна браузера до контента страницы.

*bottommargin* - определяет отступ от нижнего края окна браузера до контента страницы.

*text* - устанавливает цвет текста для всего документа.

**Теги, делающие текст заголовками**

`<h1></h1>`

`<h2></h2>`

<h3> </h3>

<h4> </h4>

<h5> </h5>

<h6> </h6>

Эти теги выделяют текст в виде заголовков. Т.е. каждый заголовок начинается с новой строки, выделен полужирным шрифтом и имеет свой размер (заголовок первого уровня самый большой, шестого - самый маленький).

Эти теги могут использоваться с параметром горизонтального выравнивания *align*. Возможные значения этого параметра:

*left* - слева,

*right* - справа,

*center* - по центру,

*justify* - по ширине.

### **Абзацы и перенос строки**

Тег <br> - тег принудительного перевода строки. Текст, после этого тега начинается с новой строки.

Теги <p></p> разделяют текст на абзацы. Перед началом каждого абзаца следует поместить тег <p>, закрывающий тег не обязателен. В отличие от тега <br> абзацы отделяются друг от друга пустой строкой.

У тега <p> есть параметр *align*, который задает способ выравнивания текста внутри параграфа. Возможные значения этого параметра:

*left* - слева,

*right* - справа,

*center* - по центру,

*justify* - по ширине.

### **Курсив, полужирный, подчеркивание**

<cite> </cite>

<dfn> </dfn>

<em> </em>

`<i></i>`

Эти теги выделяют текст курсивом, но делают они это по разным причинам. Теги `<cite></cite>` используются для логического выделения названий книг, статей и цитат. Теги `<dfn></dfn>` используются для выделения определений. Тегами `<em></em>` и `<i></i>` выделяют важные фрагменты текста. Последний не рекомендуется к употреблению.

`<strong></strong>`

`<b></b>`

Оба используются для выделения важных фрагментов текста, но предпочтительнее использовать первый.

`<ins></ins>`

`<u></u>`

Оба используются для подчеркивания важных фрагментов текста, но предпочтительнее использовать первый.

### **Тег font и его параметры**

Теги `<font></font>` указывают параметры шрифта текста:

*face* - название шрифта. Названий шрифтов можно привести несколько, через запятую. В этом случае, если первый указанный шрифт не будет найден (вы же не знаете, какие шрифты установлены на компьютере пользователя), браузер станет использовать следующий по списку.

*size* - размер шрифта в условных единицах от 1 до 7. По умолчанию размер шрифта равен 3.

*color* - цвет текста (по умолчанию - черный). Существуют два способа задания цвета: по имени и указанием шестнадцатеричного кода цвета.

### **Форматирование текста**

Теги `<abbr></abbr>` указывают на то, что текст является аббревиатурой. По умолчанию текст подчеркивается пунктиром, а при наведении на него курсора мыши появляется всплывающая подсказка с расшифровкой аббревиатуры. Это достигается добавлением в тег параметра

*title*, в качестве значения которого и выступает расшифровка. Теги `<acronym></acronym>` ведут себя аналогично тегам `<abbr></abbr>` , но поддерживаются всеми браузерами.

Теги `<address></address>` применяются для указания адреса. Отображается курсивом с новой строки.

Теги `<code></code>` используются при необходимости вывести на экран фрагмент программного кода.

Теги `<del></del>` делают текст перечеркнутым. Аналогично ведут себя теги `<strike></strike>` и теги `<s></s>` , но они не рекомендуются к использованию спецификацией HTML 4.0.

Теги `<q></q>` выделяют короткие цитаты.

Теги `<big></big>` выводят текст большего размера, чем остальная часть текста. Теги `<small></small>` выводят текст меньшего размера, чем остальная часть текста.

Тег `<hr>` рисует горизонтальную линию, отделенную сверху и снизу пустыми строками.

Теги `<pre></pre>` используются тогда, когда необходимо отобразить текст так, как он набран.

### **Специальные символы**

`&lt;` - знак "меньше"

`&gt;` - знак "больше"

`&nbsp;` - непрерывный пробел

`&copy;` - знак "copyright"

`&amp;` - знак "амперсанд"

`&laquo;` - левая открывающаяся кавычка

`&raquo;` - правая закрывающаяся кавычка

`&uarr;` - стрелка вверх

`&rarr;` - стрелка вправо

`&darr;` - стрелка вниз

`&larr;` - стрелка влево

**&harr;** - двунаправленная стрелка влево-вправо

## **Ссылки**

Ссылка - одно из важнейших понятий для html-документов. За переходы отвечает тег-контейнер `<a></a>`, а за то, куда направить пользователя по щелчку - его атрибут *href*.

При абсолютной адресации используется полный путь, т.е. с указанием протокола, сервера, каталога и страницы. В нашем примере абсолютный адрес страницы `index.html` следующий:

```

```

При относительной адресации путь указывается от того каталога, в котором хранится исходный файл. Например, если мы хотим разместить ссылку со страницы `index.html` (исходный файл) на страницу `articles.html`, то, учитывая, что обе страницы лежат в одном каталоге (т.е. имеют один уровень), она будет выглядеть следующим образом:

```

```

Если же мы захотим сделать ссылку со страницы `index.html` на страницу `price.html`, то, учитывая, что страницы лежат в разных каталогах, она будет выглядеть следующим образом:

```

```

По умолчанию ссылки загружаются в то же окно, но если указать параметр *target="\_blank"*, то ссылка загрузится в новое окно.

Иногда, когда документ слишком длинный, полезно сделать ссылки на его разделы. Для таких переходов следует создать пустую ссылку с параметром *name* там, куда следует перейти. И ссылку с указанием этого имени (после знака `#`) в качестве адреса параметра *href* там, откуда следует перейти.

У ссылок есть параметр *title*, в качестве значения которого можно указать текст, который будет выводиться в качестве подсказки, если подвести курсор мыши к ссылке.

## **Нумерованные списки**

Такой список представляет собой перечень "пронумерованный" каким-либо образом. Это могут быть арабские цифры, цифры римского алфавита или буквы. Чтобы указать браузеру, что список будет нумерованным, используются теги `<ol>` `</ol>`. Весь список располагается между этими тегами и каждый его пункт задается тегами `<li>` `</li>`.

```

 кошки
 собаки
 лошади

```

По умолчанию нумерация ведется арабскими цифрами. Это можно изменить с помощью параметра *type*:

- *type*="A" - нумерация заглавными латинскими буквами (A, B, C).
- *type*="a" - нумерация прописными латинскими буквами (a, b, c).
- *type*="I" - нумерация большими римскими цифрами (I, II, III).
- *type*="i" - нумерация маленькими римскими цифрами (i, ii, iii).

Если нам нужно начать нумерацию не с единицы, то следует использовать параметр *start*, в качестве значения которого указывается число, с которого будет начинаться список (если нумерация задается с помощью букв, то это число будет указывать на позицию в алфавите).

### **Маркированные списки**

В маркированном списке вместо букв и цифр используются маркеры. Чтобы указать браузеру, что список будет маркированным, используются теги `<ul>` `</ul>`. Весь список располагается между этими тегами и каждый его пункт задается тегами `<li>` `</li>`.

```

 кошки
 собаки
 лошади

```

По умолчанию маркер отображается в виде черного кружочка. Это можно изменить с помощью параметра *type*:

- *type="disc"* - закрашенный кружочек.
- *type="circle"* - пустой кружочек.
- *type="square"* - закрашенный квадратик.

Если нужно начать нумерацию не с единицы, то следует использовать параметр *start*, в качестве значения которого указывается число, с которого будет начинаться список (если нумерация задается с помощью букв, то это число будет указывать на позицию в алфавите).

### **Работа с изображениями**

Фоновое изображение заполняет собой все пространство элемента для которого оно задано. Так, указав в теге *<body>* атрибут *background="fon.gif"*, рисунком *fon.gif* будет залито все окно браузера.

Для размещения на странице изображений используется тег *<img>*, имеющий единственный обязательный параметр *src*, определяющий URL-адрес изображения.

Для выравнивания изображений используется параметр *align*. Он имеет следующие значения: *"left"*, *"right"*, *"top"*, *"texttop"*, *"middle"*, *"absmiddle"*, *"bottom"*, *"absbottom"*.

Любое изображение имеет размер, заданный в пикселах. Для этого у тега *<img>* существуют параметры *width* - ширина и *height* - высота. Они задаются в пикселах или в процентах.

Вы, наверно, обратили внимание, что текст очень близко прилипает к картинкам. Это не всегда красиво. Для решения проблемы прилипания текста к картинкам имеются параметры *hspace* - горизонтальный отступ и *vspace* - вертикальный отступ. Отступы задаются в пикселах.

Если у пользователя отключен режим загрузки изображений, то вместо картинки он увидит красный крестик (в Internet Explorer) или другую иконку (в других браузерах). Чтобы дать ему подсказку, что изображено на рисунке, используется параметр *alt*.

Чтобы изображение сделать ссылкой, достаточно поместить его внутрь тега-контейнера `<a>`. В примере наша картинка будет ссылаться на страницу `html_example1.html`, которая будет открываться в отдельном окне.

```



```

### Работа с таблицами

Таблица - один из основных инструментов для создания web-страниц. Без использования CSS, только с помощью таблиц можно создавать страницы со сложным дизайном.

```
<table>
 <caption>Название таблицы</caption>
 <tr><th>1</th><th>2</th><th>3</th></tr>
 <tr><td>11</td><td>12</td><td>13</td></tr>
 <tr><td>21</td><td>22</td><td>23</td></tr>
 <tr><td>31</td><td>32</td><td>33</td></tr>
</table>
```

Для оформления у тега `<table>` существует ряд параметров:

*width* - задает ширину таблицы (в пикселах или % от ширины экрана),

*bgcolor* - задает цвет фона ячеек таблицы,

*background* - заливает фон таблицы рисунком,

*border* - задает рамку указанной ширины (в пикселах) вокруг всей таблицы,

*align* - задает выравнивание таблицы: слева (right), справа (left), по центру (center),

*cellspacing* - задает расстояние между ячейками таблицы (в пикселах),

*cellpadding* - задает расстояние между текстом и внутренней границей ячейки таблицы (в пикселах).

За границы отвечают два параметра:

*frame* - задает вид рамки вокруг таблицы и может принимать следующие

значения: *void, above, below, hside, vside, lhs, rhs, box, rules* - задает вид внутренних границ таблицы и может принимать следующие значения: *none, groups, rows, cols, all*.

У ячеек можно указать размер: *width* - задает ширину столбца (в пикселах или в процентном соотношении, где за 100% принимается ширина таблицы), *height* - задает высоту ячейки, причем все ячейки в этой же строке станут этой высоты. Параметр *colspan* указывает, сколько столбцов занимает ячейка. Аналогично *rowspan* – для строк.

### **HTML формы**

Формы дают возможность пользователям вводить информацию. В html форма задается тегами `<form></form>`. Все остальные элементы формы располагаются между этими тегами.

У тега `<form>` есть несколько параметров:

*name* - имя формы. Необходимо, если на странице несколько форм,

*action* - определяет URL-адрес, по которому будет отправлена информация введенная пользователем,

*method* - определяет способ отправки информации,

*target* - указывает имя окна, в котором будут отображаться результаты обработки отправленной формы.

```
<form name="forma"></form>
```

### **Текстовое поле**

Простое однострочное текстовое поле, в которое можно вводить и редактировать текст. Задается тегом `<input>`

```
<form name="forma">
```

```
<input type="text" name="text1" size="20" maxlength="50" value="Введите текст">
```

```
</form>
```

Параметры:

*name* - имя элемента,

*type* - тип элемента (в данном случае - text),

*size* - размер текстового поля в символах, которые одновременно будут видны, при вводе большего количества символов, они будут прокручиваться,

*maxlength* - максимальное количество символов, которое можно ввести в поле, если опустить этот параметр, то число символов будет неограниченным,

*value* - текст, который будет отображаться (его можно стереть), при отсутствии этого параметра поле будет пустым,

*disabled* - блокирует поле от любых изменений,

*readonly* - делает поле доступным только для чтения.

Для ввода паролей используется текстовое поле с параметром *type="password"*.

### **Чекбокс**

```
<form name="forma">
```

*Какими языками вы владеете:*<br>

```
<input type="checkbox" name="lan1" value="english" checked>английский
```

```
<input type="checkbox" name="lan2" value="german"> немецкий
```

```
<input type="checkbox" name="lan3" value="spanish"> испанский
```

```
<input type="checkbox" name="lan4" value="french"> французский
```

```
</form>
```

Рассмотрим параметры:

*type* - тип элемента (в данном случае - checkbox),

*name* - имя элемента, указывает программе-обработчику формы, какой пункт выбрал пользователь,

*value* - значение элемента, указывает программе-обработчику формы значение пункта, который выбрал пользователь. В нашем примере выбран пункт английский, следовательно, программа-обработчик получит: *lan1="english"*,

*checked* - им обычно помечают наиболее вероятные для выбора пункты, пользователь щелчком мыши может выбрать другие пункты.

## Радиокнопки

В отличие от чекбоксов, можно выбрать только один пункт. В связи с этим значения параметра *name* должны быть одинаковы для всех элементов группы. Параметр *type="radio"*, все остальные аналогичны.

```
<form name="forma">
```

Укажите ваш пол:<br>

```
<input type="radio" name="sex" value="man" checked>мужской
```

```
<input type="radio" name="sex" value="woman"> женский
```

```
</form>
```

## Кнопки

Существует четыре вида кнопок:

*submit* - кнопка отправки содержимого формы web-серверу. Ее параметры: *type="submit"* - тип кнопки, *name* - имя кнопки, *value* - надпись на кнопке.

*image* - графическая кнопка отправки содержимого формы web-серверу. Для ее использования необходимо подготовить картинку кнопки, а потом использовать ее в виде кнопки. Ее параметры: *type="image"* - тип графической кнопки, *name* - имя кнопки, *src* - адрес картинки для кнопки.

*reset* - кнопка, позволяющая восстановить все значения по умолчанию в форме. Ее параметры: *type="reset"* - тип кнопки очищения, *name* - имя кнопки, *value* - надпись на кнопке.

*button* - произвольная кнопка, ее действия назначаются вами, т.е. сама она делать ничего не умеет. Ее параметры: *type="button"* - тип произвольной кнопки, *name* - имя кнопки, *value* - надпись на кнопке, *onclick* - указывает, что делать при щелчке по кнопке. Также у этого типа кнопок есть и другие события (например, двойной щелчок).

```
<form name="forma">
```

```
<input type="submit" name="submit" value="Отправить">
```

```
<input type="image" name="but_img" src="but.gif">
```

```
<input type="reset" name="reset" value="Очистить">
```

```
<input type="button" name="button" value="Отправить">
</form>
```

### **Файлы и многострочный ввод**

Поле для ввода имени файла, сопровождаемое кнопкой Browse (Обзор), при щелчке по которой открывается окно просмотра дерева папок компьютера, где можно выбрать нужный файл. Выбранный файл присоединяется к содержимому формы при отправке на сервер.

```
<form name="forma">
 <input type="file" name="load" size="50">
</form>
```

Для объемных текстов, например, для почтовых сообщений, удобно использовать именно этот элемент. Он создается тегами `<textarea>` `</textarea>` и имеет следующие параметры:

*name* - имя поля,

*cols* - ширина поля в символах,

*rows* - количество строк текста, видимых на экране,

*wrap* - способ переноса слов: *off* - переноса не происходит, *virtual* - перенос отображается, но на сервер поступает неделимая строка, *physical* - перенос и на экране и при поступлении на сервер,

*disabled* - неактивное поле,

*readonly* - разрешено только чтение.

```
<form name="forma">
 <textarea cols="20" rows="3" wrap="off">
</form>
```

### **Раскрывающиеся списки**

Списки бывают с возможностью выбора одного элемента и с множественным выбором. Задаются и те, и другие с помощью тегов `<select>` `</select>`, внутри которых располагаются элементы значений, заданных тегом `<option>`. Рассмотрим параметры этих тегов:

`<select>`:

*name* - имя списка. Каждый выбранный элемент списка при передаче на сервер будет иметь вид: *name.value*, где значение (*value*) берется из тега *option*,

*size* - определяет количество видимых элементов в списке: 1 - простой раскрывающийся список, больше 1 - список с полосой прокрутки,

*multiple* - разрешает выбор нескольких элементов списка.

*<option>*:

*selected* - им помечают наиболее вероятный для выбора элемент списка, если список со множественным выбором, то можно пометить несколько пунктов,

*value* - значение, которое будет отправлено серверу, если пункт выбран.

```
<form name="forma">
```

*Какой язык вы хотите изучать:*

```
<select name="language" size="1">
```

```
<option selected value="html">html
```

```
<option value="php">php
```

```
<option value="java">java
```

```
</select>
```

```
</form>
```

## **Навигационные карты**

Многие html-страницы используют для организации ссылок так называемые карты-изображения. При таком подходе берется изображение и к различным его областям привязываются ссылки.

Для описания таких областей используются теги *<map>* *</map>* с единственным параметром *name*, который задает имя карты ссылок и используется потом для ссылки на эту карту. Карту надо подключить к картинке, для этого в тег *<img>* надо добавить параметр *usemap*, в качестве значения которого выступает имя карты после значка #:

```
<body>
```

```

```

```
<map name="map">
```

```
<area shape="rect" coords="550,135,590,192" href="#" alt="area1">
<area shape="rect" coords="591,90,660,192" href="#" alt="area2">
<area shape="rect" coords="661,0,730,192" href="#" alt="area3">
</map>
</body>
```

Для описания конкретных областей внутри тегов `<map>` `</map>` используются теги `<area>`. Этот тег имеет следующие параметры:

*shape* - определяет форму области, может принимать следующие значения: *rect* - область в виде прямоугольника, *circle* - область в виде круга, *poly* - область в виде многоугольника, *default* - вся область.

*coords* - задает координаты отдельной области: для *rect* задаются координаты верхнего левого и правого нижнего углов прямоугольника, для *circle* задаются координаты центра круга и радиус, для *poly* задаются координаты вершин многоугольника в нужном порядке.

*href* - определяет адрес ссылки.

*target* - применяется при использовании фреймов и указывает фрейм в который нужно загрузить страницу.

*alt* - задает альтернативный текст для области.

## CSS

HTML - это язык разметки, но его создатели так хотели облегчить жизнь разработчиков сайтов, что добавляли в него элементы и параметры, отвечающие за внешний вид страницы: теги `<font>`, `<b>`, `<i>`, `<bgcolor>` и так далее. На какой-то момент код страниц стал слишком громоздким и нечитабельным. Тогда было принято решение разделить разметку страницы (HTML) и ее визуальное оформление (CSS).

CSS (Cascading Style Sheets) - каскадные таблицы стилей. *Стиль* - набор параметров, задающий внешнее представление объекта.

Каждый элемент на странице может иметь свой стиль (параграфы, заголовки, линии, текст). Набор стилей всех элементов называют *таблицей стилей*. Если для одного элемента задано несколько стилей (как в примере с

заголовками), то применяется *каскадирование*, которое определяет приоритет того или иного стиля.

### **Преимущества CSS**

- CSS позволяет значительно сократить размер кода и сделать его читабельным.
- CSS позволяет задавать такие параметры, которые нельзя задать только языком HTML. Например, убрать подчеркивание у ссылок.
- CSS позволяет легко изменять внешний вид страниц. Представьте, вы сделали сайт из 50 страниц, на которых все заголовки синего цвета. Через какое-то время, вы захотели поменять синий цвет на зеленый. Вам придется пройтись по всем 50 страницам и поменять цвет в соответствующем атрибуте. С CSS вам придется сделать это лишь один раз, в таблице стилей.

С CSS связана так называемая блочная верстка сайта.

### **Подключение CSS к HTML**

#### *Внешние таблицы стилей*

Нам надо подключить страницу style.css к html-странице. Для этого в html существует тег `<link>`, который и отвечает за подключение внешних файлов.

```
<head>
```

```
<title>Подключение CSS к HTML</title>
```

```
<link rel="stylesheet" type="text/css" href="style.css">
```

```
</head>
```

#### *Внутренние таблицы стилей*

Так называют таблицу стилей, заданную внутри элемента HTML, при помощи атрибута *style*.

```
<h1 style="color:red">Это заголовок красного цвета</h1>
```

Недостаток этого способа очевиден: параметр *style* придется задавать каждому заголовку. Таким образом, теряется преимущество использования CSS.

## *Встроенные таблицы стилей*

При этом способе таблица стилей встраивается в заголовок html-страницы. Для этого в HTML существуют теги `<style></style>`, с параметром `type`, который указывает, что подключается именно таблица стилей CSS (существуют и другие).

```
<head>
 <title>Подключение CSS к HTML</title>
 <style type="text/css"> h1 { color:red } </style>
</head>
<body>
 <h1>Этот заголовок будет красного цвета</h1>
 <h1 style="color:blue">Этот заголовок будет синего цвета</h1>
</body>
```

## **Правила и селекторы CSS**

CSS, как и любой язык, имеет свой синтаксис. В нем нет ни элементов, ни параметров, ни тегов. В нем есть правила. Правило состоит из селектора и блока объявления стилей, заключенного в фигурные скобки. Сам блок объявления стилей состоит из свойств и их значений, разделенных точкой с запятой.

```
body {
 background: blue;
 color: white;
}
h1 {
 color: red;
}
```

В HTML идентификатор элемента задается при помощи параметра `id`, в качестве значения которого указывается уникальное имя.

```
p#pink { color:pink; }
<p id="pink">Текст параграфа с идентификатором (id).</p>
```

Если нужно, чтобы розовый цвет текста был у двух или трех параграфов, то для этого в HTML существует параметр *class*, в качестве значения которого указывается имя стиля.

```
p.pink { color:pink; }
```

Контекстный селектор

```
p i { color:green; }
```

```
p > i { color:green; }
```

Группировка селекторов

```
h1, h2, h3 { color:green; }
```

### **Псевдоэлементы и псевдоклассы**

К псевдоэлементам относят:

*first-letter* - задает стиль первой букве слова.

*first-line* - задает стиль первой строке абзаца.

В CSS существуют четыре псевдокласса, они позволяют работать со ссылками. У ссылок есть четыре состояния: простая, активная, посещенная и та, на которую наведен курсор. Состояние ссылок зависит от действия пользователя, и браузер, в зависимости от этих действий может применять разные стили. Для описания этих стилей и существуют псевдоклассы.

*a:link* - задает стиль обычной ссылки.

*a:active* - задает стиль активной ссылки.

*a:visited* - задает стиль посещенной ссылки.

*a:hover* - задает стиль ссылки, на которую наведен курсор.

### **color и background**

Задать цвет текста для элемента можно тремя способами:

```
body { color:green; } h1 { color:#FF0000; } h2 { color:rgb(255,0,0); }
```

*background* – на самом деле это группа свойств, так или иначе связанная с фоном. При помощи CSS фон можно задать не только странице, но и заголовку, и абзацу, и любому другому элементу. Рассмотрим группу свойств *background*:

*background-color* - задает цвет фона. По умолчанию не наследуется, но его можно сделать наследуемым, если в качестве значения указать значение *inherit*.

*background-image* - задает фоновое изображение. Значением свойства является URL графического файла. Формат задания следующий: сначала идет обозначение функции *url*, а затем в круглых скобках указывается путь к файлу. Путь к файлу указывается относительно таблицы стилей.

*background-repeat* - задает возможность повторения фонового изображения. В качестве фонового изображения может выступать как цельное изображение (например, шапка сайта), так и маленькое изображение, которое должно замостить собой все пространство элемента. Данное свойство как раз и указывает, повторять ли изображение и, если да, то как именно повторять. Возможны 4 варианта: *repeat* - повторять изображение по горизонтали и вертикали, *repeat-x* - повторять изображение только по горизонтали, *repeat-y* - повторять изображение только по вертикали, *no-repeat* - не повторять изображение.

*background-attachment* - указывает, должно ли фоновое изображение прокручиваться вместе с текстом элемента или оно должно оставаться неподвижным. Может принимать два значения: *scroll* - фон прокручивается вместе с текстом. Это значение используется по умолчанию, *fixed* - фоновое изображение фиксируется относительно окна браузера, и во время прокрутки текста оно остается неподвижным.

*background-position* - задает расположение элемента относительно окна браузера. Значения можно задавать в процентах, в единицах длины и при помощи ключевых слов.

```
body { background-image: url(picture.gif); background-repeat: no-repeat;
background-color:#243CED; background-position:50px 50px; color: yellow; }
```

```
body { background: url(picture.gif) no-repeat #33CCFF center top; color:
yellow; }
```

## Шрифты

Для задания параметров шрифтов в CSS используется свойство *font*.

*font-family* задает гарнитуру шрифта. Все шрифты можно условно разделить на несколько групп: *Serif* - шрифты с засечками, например, Times New Roman, *Sans-serif* - шрифты рубленые, без засечек, например, Arial, *Monospace* - моноширинные шрифты, например, Courier New, *Cursive* - курсивные шрифты, например, Calisto MT, *Fantasy* - декоративные шрифты, например, Torhok.

*font-variant* задает вариант написания букв из двух возможных: *normal* (обычный) и *small-caps* (малые прописные буквы).

*font-weight* задает толщину букв шрифта. В качестве значений выступают числа: 100, 200, 300, 400, 500, 600, 700, 800 и 900. А также ключевые слова: *normal* (нормальный), *bold* (полужирный), *bolder* (более жирный по отношению к базовому, унаследованному от предка) и *lighter* (менее жирный по отношению к базовому, унаследованному от предка).

*font-size* задает размер шрифта. Задавать размер в css можно тремя способами: с помощью ключевых слов (xx-small, x-small, small, medium, large, x-large, xx-large, smaller, large), с помощью относительных единиц (% и em), с помощью единиц измерения длины (пиксели, пункты, сантиметры и миллиметры).

```
p { font-family: Verdana, sans-serif; font-size: 12px; font-weight: bold; font-variant: small-caps; font-style: italic; }
```

```
p { font: italic 12px Verdana, sans-serif; }
```

### Текст

*text-decoration* отвечает за оформление текста. Может принимать следующие значения: *none* - у текста нет оформления, *underline* - текст подчеркивается, *overline* - текст надчеркивается линией, расположенной над текстом, *line-through* - текст отображается зачеркнутым, *blink* - текст становится мигающим (не работает в IE).

*text-align* отвечает за горизонтальное выравнивание текста. Может принимать следующие значения: *left* - выравнивание по левому краю, *center* -

выравнивание по центру, *right* - выравнивание по правому краю, *justify* - выравнивание по ширине.

*text-indent* отвечает за отступы в абзацах. Отступы задаются в единицах измерения и %, могут быть как положительными (красная строка), так и отрицательными (висячая строка).

*text-transform* отвечает за видоизменение текста, точнее за смену регистра. Используются следующие значения: *capitalize* - меняет первую букву каждого слова на заглавную, *uppercase* - меняет все буквы на заглавные, *lowercase* - меняет все буквы на строчные, *none* - изменений не происходит.

В CSS есть несколько свойств, регулирующих расстояния между словами, буквами в словах и строками: *word-spacing* - задает интервал между словами, *letter-spacing* - задает интервал между буквами, *line-height* - задает интервал между строками.

```
body { background: #0033CC; color: #FFFFFF; font: 12px Verdana, sans-serif; }
```

```
h1 { font-size: 1.6em; color: #FFCC00; letter-spacing: 5px; text-align: center; }
```

```
h2 { font-size: 1.3em; color: #FFCC00; text-decoration: underline; text-align: center; }
```

```
p:first-letter { color: #FFCC00; }
```

```
p { text-indent: 1.5em; word-spacing: 10px; font-style: italic; }
```

## **Блоки**

В HTML все элементы можно разделить на два типа: блочные и строчные.

Блочные элементы визуально создают самостоятельную структурную единицу - блок. К ним можно отнести, например, элементы H1-H6, P, DIV. Такие элементы отделяются от других абзацными отступами.

Строчные элементы выводятся линейной строкой. К ним можно отнести, например, элементы I, B, U, S и другие. В CSS модель документа представляется блоком. Каждый элемент в дереве элементов документа

представляет собой самостоятельный блок. Причем, есть блоки, структурно отделенные от остальных, а есть строчные блоки, которые могут находиться внутри структурных блоков.

У блока есть *содержимое*, например, для элемента *p* - это текст. Вокруг содержимого есть *отступы (padding)*, они служат для того, чтобы текст не примыкал вплотную к границе блока. Фон отступов такой же, как и у содержимого. Затем идет *граница блока (border)*, которая может быть как видимой, так и невидимой. Также блок имеет *поля (margin)*, которые задают дополнительное свободное пространство вокруг блока. Фон полей прозрачный, т.е. сквозь него просвечивает фон родительского элемента. Размер блока, т.е. его *ширина (width)* и *высота (height)*, определяются содержимым. И это надо запомнить: поля и отступы не учитываются в размере блока.

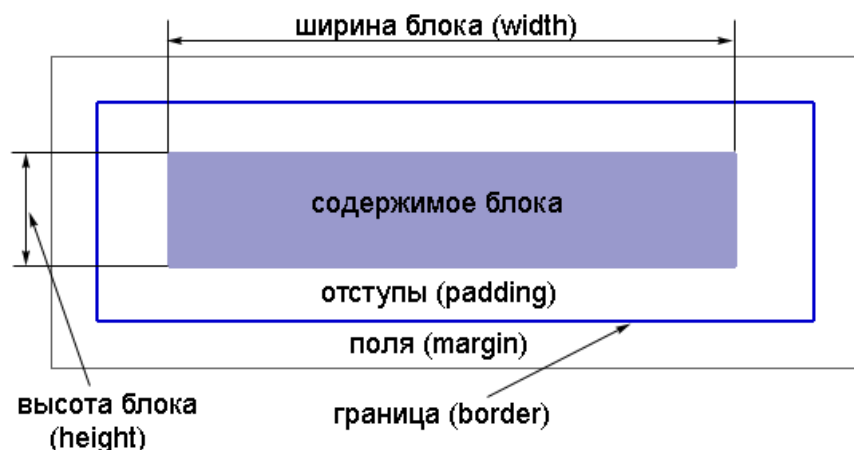


Рисунок 9 –Блок.

### Свойства – border

*div* является контейнером для остальных. Элемент *div* отделяется от остальных элементов абзацными отступами, содержимое тегов *div* располагается с абзацным отступом, т.е. одно под другим. Элемент *span*, в отличие от *div*, создает строчный блок.

Границы в CSS можно задавать отдельно для каждой стороны: *border-top* - верхняя граница, *border-right* - правая граница, *border-bottom* - нижняя граница, *border-left* - левая граница.

Каждый сегмент границы может иметь свои характеристики: цвет, толщину и тип линии. Для этого к свойству границы через дефис необходимо дописать ключевые слова: *color* (для цвета), *width* (для толщины) и *style* (для типа линии). Например, *border-top-color* определяет цвет верхней границы, а *border-left-style* - тип линии для левой границы. Если все четыре границы будут иметь одинаковые значения, то следует воспользоваться сокращенной записью: *border-color* - цвет всех границ, *border-width* - толщина всех границ, *border-style* - стиль всех границ.

Значениями свойства *color* могут быть именные цвета (*red*, *blue* и т.д.), шестнадцатеричные коды цветов (типа *#FFCCFF*) и десятичные коды в модели RGB (типа *rgb(255, 0, 0)*).

Значениями свойства *width* могут быть ключевые слова: *thin* (тонкая граница), *medium* (средняя граница) и *thick* (толстая граница). А также любая единица измерения.

Значениями свойства *style* могут быть следующие ключевые слова: *none* - граница отсутствует, *dotted* - граница состоит из точек, *dashed* - граница в виде пунктирной линии, *solid* - граница отображается сплошной линией, *double* - граница отображается двойной сплошной линией, *groove* - граница отображается вдавленной объемной линией, *ridge* - граница отображается выпуклой объемной линией, *inset* - граница отображается так, что весь блок выглядит вдавленным, *outset* - граница отображается так, что весь блок выглядит выпуклым.

```
#first { border-bottom-color: red; border-top-style: solid; border-left-width: 2px; }
```

```
#second { border: 1px solid red; }
```

### **Свойства – *margin*, *padding***

Поля задают свободное пространство вокруг элемента. Как и границы, поля в *css* можно определять отдельно для верхней, правой, нижней и левой сторон. Для этого используются свойства: *margin-top* - ширина верхнего поля,

*margin-right* - ширина правого поля, *margin-bottom* - ширина нижнего поля, *margin-left* - ширина левого поля.

Чаще используется сокращенная запись *margin*, где через пробел указываются ширина верхнего, правого, нижнего и левого полей. Причем, именно в таком порядке.

Отступы позволяют отделить содержимое блока от границы. Параметры отступов в CSS можно задать для каждой стороны отдельно, используя следующие свойства: *padding-top* - ширина верхнего отступа, *padding-right* - ширина правого отступа, *padding-bottom* - ширина нижнего отступа, *padding-left* - ширина левого отступа.

Значения свойств могут задаваться в различных единицах длины или в процентах. Проценты вычисляются относительно ширины блока. В качестве значения может выступать только положительная величина.

```
#third { border: 1px solid red; margin: 10px; padding: 10px 5px; }
```

### **Позиционирование блоков**

В HTML для позиционирования элементов на странице используются таблицы. У таблиц есть как преимущества (легкость использования, одинаковое отображение браузерами), так и недостатки (объемный, нечитабельный код, нелогичность верстки и т.д.).

В CSS для позиционирования элементов используются блоки (div-ы). Код при этом становится компактным, логичным и легко изменяемым. К недостаткам блочной верстки можно отнести неодинаковую поддержку браузерами, поэтому приходится писать кроссбраузерный код (т.е. код, который отображается разными браузерами почти одинаково).

*Позиционированием в нормальном потоке* означает, что все элементы отображаются в окне браузера сверху вниз, по вертикали, в том порядке, в каком они следуют друг за другом в html-коде. По своей сути нормальный поток ничем не отличается от позиционирования элементов в HTML. И для верстки такой страницы без CSS, нам пришлось бы использовать таблицу, за неимением других вариантов.

В CSS же предоставляются и другие схемы позиционирования: абсолютное позиционирование, относительное позиционирование, плавающая блоковая модель.

Для определения схемы позиционирования используется свойство *position*, оно может принимать четыре значения, соответствующие выбранной схеме позиционирования:

*static* - блок позиционируется в нормальном потоке. Это значение по умолчанию.

*relative* - относительное позиционирование (относительно нормального потока).

*absolute* - абсолютное позиционирование.

*fixed* - фиксированное позиционирование (фиксируется относительно области просмотра).

### **Абсолютное позиционирование**

При этой схеме позиционирования расположение блока на странице не зависит от того, в каком месте html-кода расположен этот блок. Расположение каждого блока задается указанием, в каком месте экрана отобразить данный блок. Для этого существуют четыре свойства:

*left* – указывает, на сколько надо сместить блок относительно левого края окна.

*right* – указывает, на сколько надо сместить блок относительно правого края окна.

*top* – указывает, на сколько надо сместить блок относительно верхнего края окна.

*bottom* – указывает, на сколько надо сместить блок относительно нижнего края окна.

Для блока *content* укажем свойство *position: absolute* и зададим смещение: от левого края окна на 200 пикселей, а от верхнего края окна на 100 пикселей.

```
#content {
```

```
background: oldlace;
width: 500px;
height: 300px;
position: absolute;
left: 200px;
top: 100px;
}
```

### **Относительное позиционирование**

При относительном позиционировании блока надо задать свойство *position: relative* и свойства смещения. Смещение в этом случае будет происходить не относительно "родительского" элемента (как при абсолютном позиционировании), а относительно самого блока в нормальном потоке.

*Плавающие блоки* нельзя позиционировать с точностью до пиксела, как в предыдущих схемах, но именно эта схема позиционирования очень распространена. Без плавающих блоков обходится редкий сайт, а уж сделать "резиновую" верстку сайта без них и вовсе невозможно. Такие блоки могут свободно перемещаться по странице, подобным образом ведут себя картинки в HTML, выровненные с помощью параметра *align*. Плавающие блоки определяются свойством *float*, которое определяет, будет ли блок плавающим и в какую сторону он будет перемещаться. Возможны три варианта: *left* - блок прижимается к левому краю, остальные элементы обтекают его с правой стороны, *right* - блок прижимается к правому краю, остальные элементы обтекают его с левой стороны, *none* - блок не перемещается и позиционируется согласно свойству *position*.

Иногда нужно, чтобы блоки были прижаты к правому краю, но располагались один под другим. Для этого существует свойство *clear*, которое определяет, какие стороны плавающего блока не могут соседствовать с другими плавающими блоками. У этого свойства может быть задано одно из четырех значений: *left* - блок должен располагаться ниже всех левосторонних блоков, *right* - блок должен располагаться ниже всех правосторонних блоков,

*both* - блок должен располагаться ниже всех плавающих блоков, *none* - никаких ограничений нет, это значение по умолчанию.

```
#blok1 { border:1px solid red; width: 150px; height: 50px; float: right; }
#blok2 { border:1px solid red; width: 150px; height: 50px; float: right; clear:
right; }
```

### **Виды и свойства блоков**

Элементы могут быть блочными и строчными. По умолчанию для каждого элемента его вид определен, так элементы *div* и *p* являются блочными, а *span* и *a* - строчными. Но иногда это необходимо изменить, для этого используется свойство *display*. Это свойство может принимать одно из четырех значений: *display:block* делает элемент блочным, *display:inline* делает элемент строчным, *display:list-item* делает любой элемент частью списка, *display:none* убирает элемент со страницы.

В CSS есть еще одно свойство на первый взгляд похожее на *display:none*. Это свойство, отвечающее за видимость блока - *visibility*. Оно может принимать два значения: *visible* (отображать) и *hidden* (сделать невидимым). Различия здесь следующее: *display:none* скрывает элемент, как будто его и не было, а *visibility:hidden* делает элемент прозрачным.

Свойство *overflow* может принимать четыре значения: *visible* - если содержимое превышает размеры блока, оно все равно останется на экране, *hidden* - браузер отрежет содержимое, которое превышает размер блока, *scroll* - блок будет снабжен полосами прокрутки, причем как горизонтальной, так и вертикальной, *auto* - блок будет снабжен только теми полосами прокрутки, которые необходимы.

### **Слои**

В CSS можно создать несколько слоев, на каждом разместить необходимые элементы и пронумеровать слои с помощью свойства *z-index*. Чем больше номер, тем выше слой находится в стопке слоев. Например, если сделать 6 слоев, то пользователь сначала увидит именно слой *z-index:6*.

Слои часто используются при создании раскрывающихся меню.

## Списки

Как всегда CSS предоставляет большие возможности при создании списков, чем только HTML. Собственно свойств для списков существует всего три: *list-style-type* - определяет внешний вид маркера или нумератора, *list-style-image* - определяет пользовательское изображение маркера, *list-style-position* - определяет положение маркеров относительно блока.

Для маркированных списков *list-style-type* не содержит ничего нового, все те же значения, что и в HTML: *disk* - закрашенный кружок, *circle* - незакрашенный кружок, *square* - закрашенный квадрат. Для нумерованных списков перечень более широк, чем в HTML, но не все значения поддерживаются всеми браузерами. Поэтому пока рекомендуется использовать только следующие: *decimal* - десятичные числа, *lower-roman* - строчные римские цифры, *upper-roman* - прописные римские цифры, *lower-alpha* - строчные латинские буквы.

Свойство *list-style-image* позволяет задать свой вид маркера. Для этого надо создать картинку с маркером и указать ее адрес в стиле.

Свойство *list-style-position* определяет положение маркера: внутри блока - *inside* или снаружи – *outside*.

```
#spisok1 { list-style: circle url(marker.gif) inside; }
```

## Ajax

Ajax – это аббревиатура, означающая "Асинхронный JavaScript и XML" (Asynchronous JavaScript and XML). Основное назначение Ajax состоит в предоставлении веб-приложению возможности эффективной обработки взаимодействия между пользователем и веб-страницей, при этом значительно снижаются требования к обновлению или полной перезагрузке страницы при каждом взаимодействии с пользователем. Такой подход предоставляет широкие возможности при использовании браузера (аналогичные возможностям настольного приложения или веб-приложения на основе подключаемого модуля). Обработка взаимодействия Ajax осуществляется асинхронно в фоновом режиме. Благодаря этому пользователь может продолжать работу со страницей. Взаимодействие Ajax инициируется посредством кода JavaScript. После выполнения взаимодействия Ajax код JavaScript обновляет исходный текст HTML для страницы. Изменения вносятся немедленно без необходимости обновления страницы.

Взаимодействия Ajax могут использоваться для выполнения таких задач, как проверка правильности формата вводимых записей на основе серверной логики (непосредственно во время их ввода пользователем), извлечение подробных данных из сервера, динамическое обновление данных на странице и передача элементов форм страницы.

Для того, чтобы осуществлять обмен данными, на странице должен быть создан объект XMLHttpRequest, который является своеобразным посредником между Браузером пользователя и сервером (рис. 30). С помощью XMLHttpRequest можно отправить запрос на сервер, а также получить ответ в виде различного рода данных.

Обмениваться данными с сервером можно двумя способами. Первый способ — это GET-запрос. В этом запросе вы обращаетесь к документу на сервере, передавая ему аргументы через сам URL. Не рекомендуется делать GET-запросы к серверу с большими объемами данных. Для этого существует POST-запрос.

Клиентская часть, написанная на Javascript, должна обеспечивать необходимую функциональность для безопасного обмена с сервером и предоставлять методы для обмена данными любым из вышеперечисленных способов. Серверная часть должна обрабатывать входные данные, и на основе их генерировать новую информацию (например, работая с базой данных), и отдавать ее обратно клиенту. Например, для запроса информации с сервера можно использовать обычный GET-запрос с передачей нескольких и небольших по размеру параметров, а для обновления информации, или добавления новой информации потребуется использовать уже POST-запрос, так как он позволяет передавать большие объемы данных.

Ответ от сервера может быть не только XML, как следует из названия технологии. Помимо XML, можно получить ответ в виде обычного текста, или же JSON (Javascript Object Notation). Если ответ был получен простым текстом, то его можно сразу вывести в контейнер на странице. При получении ответа в виде XML, обычно происходит обработка полученного XML документа на стороне клиента и преобразование данных к (X)HTML. При получении ответа в формате JSON клиент должен лишь выполнить полученный код для получения полноценного объекта Javascript.

### **Методы объекта XMLHttpRequest**

*abort()* — отмена текущего запроса к серверу.

*getAllResponseHeaders()* — получить все заголовки ответа от сервера.

*getResponseHeader(«имя\_заголовка»)* — получить указанный заголовок.

*open*(«тип\_запроса», «URL», «асинхронный», «имя\_пользователя», «пароль») — инициализация запроса к серверу, указание метода запроса. Тип запроса и URL — обязательные параметры. Третий аргумент — булево значение. Обычно всегда указывается true или не указывается вообще (по умолчанию — true). Четвертый и пятый аргументы используются для аутентификации (это очень небезопасно, хранить данные об аутентификации в скрипте, так как скрипт может посмотреть любой пользователь).

*send*(«содержимое») — послать HTTP запрос на сервер и получить ответ.

*setRequestHeader*(«имя\_заголовка», «значение») — установить значения заголовка запроса.

### Создание объекта XMLHttpRequest

Как уже говорилось выше, создание данного объекта для каждого типа браузера — уникальный процесс.

Например, для создания объекта в Gecko-совместимых браузерах, Konqueror`е и Safari, нужно использовать следующее выражение:

```
var Request = new XMLHttpRequest();
```

А для Internet Explorer`а используется следующее:

```
var Request = new ActiveXObject("Microsoft.XMLHTTP");
```

Либо

```
var Request = new ActiveXObject("Msxml2.XMLHTTP");
```

Теперь, чтобы добиться кроссбраузерности, необходимо сложить все функции в одну:

### Пример 96

```
function CreateRequest()
{
 var Request = false;

 if (window.XMLHttpRequest)
 {
 //Gecko-совместимые браузеры, Safari, Konqueror
 Request = new XMLHttpRequest();
 }
 else if (window.ActiveXObject)
 {
 //Internet explorer
 try
 {
 Request = new ActiveXObject("Microsoft.XMLHTTP");
 }
 catch (CatchException)
 {
 }
 }
}
```

```
 Request = new ActiveXObject("Msxml2.XMLHTTP");
 }
}

if (!Request)
{
 alert("Невозможно создать XMLHttpRequest");
}

return Request;
}
```

После всего этого можно создавать данный объект и не беспокоиться за работоспособность на популярных браузерах. Но создать объект можно в разных местах. Если создать его глобально, то в определенный момент времени возможен будет только один запрос к серверу. Можно создавать объект всякий раз, как происходит запрос к серверу (это почти полностью решит проблему).

### **Запрос к серверу**

Алгоритм запроса к серверу выглядит так:

1. Проверка существования XMLHttpRequest.
2. Инициализация соединения с сервером.
3. Посылка запроса серверу.
4. Обработка полученных данных.

Предположим, что существует веб-страница, на которой пользователь может выполнять поиск информации о композиторах. Эта страница содержит поле, в которое вводится имя композитора. В примере приложения для поля ввода может использоваться функция автозавершения. Другими словами, пользователь может ввести часть имени композитора, после чего веб-приложение предложит варианты полного имени, перечислив всех композиторов, чьи имена или фамилии начинаются с введенных символов. Таким образом, пользователь может не помнить полное имя композитора, однако функция автозавершения обеспечит быстрый интуитивный доступ к требуемой информации.

Реализация функции автозавершения в поле поиска может служить примером возможностей, предоставляемых Ajax. Ajax использует объект XMLHttpRequest для асинхронной передачи запросов и ответов между клиентом и сервером. На рисунке 30 представлена блок-схема операций связи, происходящих между клиентом и сервером.

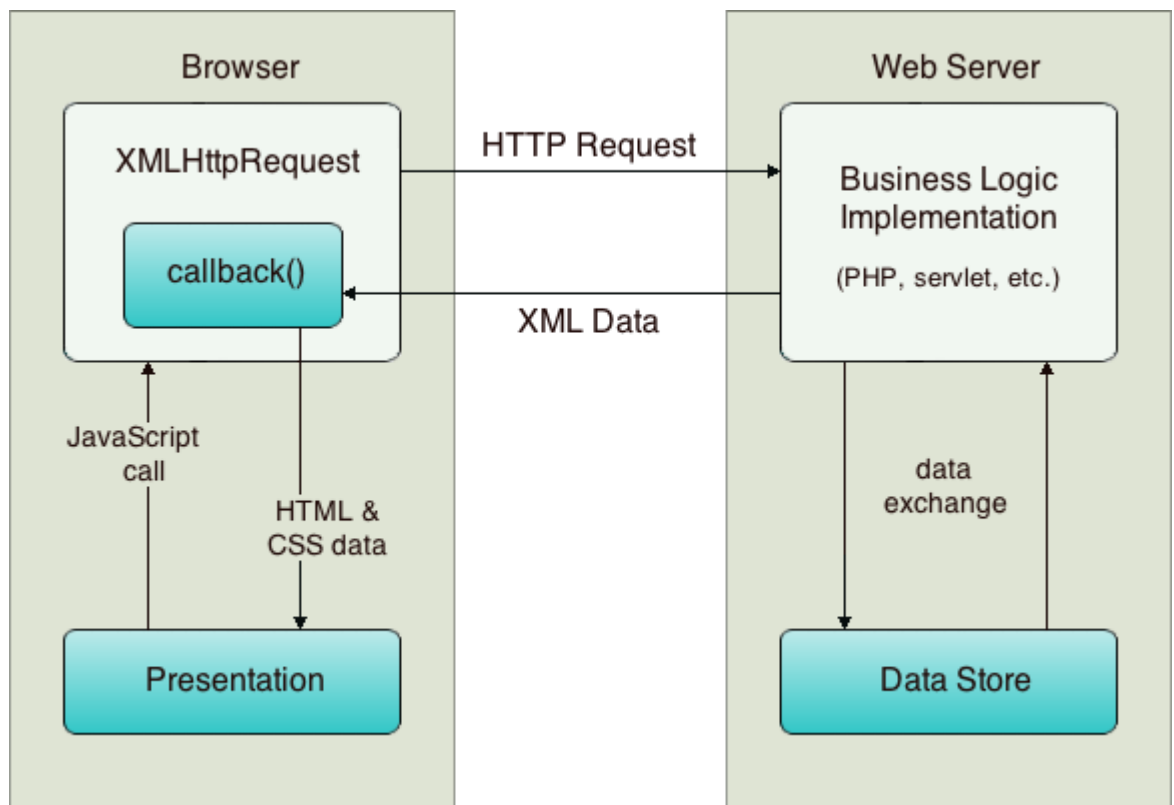


Рисунок 30 - Диаграмма потоков процессов Ajax

Для описания потока операций на блок-схеме можно использовать следующие действия.

1. Пользователь инициирует событие, например, отпускает клавишу при наборе имени. Это приводит к вызову функции JavaScript, которая инициализирует объект XMLHttpRequest.
2. Объект XMLHttpRequest настроен с учетом параметра запроса, который включает идентификатор элемента управления, инициировавшего событие, а также произвольное значение, введенное пользователем. Затем объект XMLHttpRequest выполняет асинхронный запрос к веб-серверу.
3. На веб-сервере осуществляется обработка этого запроса с использованием соответствующего объекта, например сервлета или прослушивающего процесса. Из хранилища данных извлекаются необходимые данные и подготавливается ответ, содержащий данные в форме документа XML.
4. Наконец, объект XMLHttpRequest получает данные XML с использованием функции ответного вызова, выполняет их обработку и обновляет модель DOM (Document Object Model, объектная модель документов) HTML для отображения страницы, содержащей новые данные.

Способ создания варианта автозаполнения путем выполнения потока операций процесса, показанного на рисунке выше, состоит в следующем: сначала на стороне клиента создаются файлы для страницы представления данных и функции, необходимые для генерации объекта XMLHttpRequest. Затем выполняется настройка на стороне сервера, заключающаяся в создании хранилища данных и бизнес-логики с использованием технологий на основе Java. Наконец, на стороне клиента реализуется функция обратного вызова callback() и другие функциональные возможности JavaScript, необходимые для обновления DOM HTML.

## Литература

1. Хорстманн Кей С., Корнелл Гари. Java`2. Библиотека НТМпрофессионала, том II. Тонкости программирования. 7-е издание.: Пер. с англ. – М.: Издательский дом «Вильямс», 2008. – 1168 с.: ил. ISBN 978-5-8459-1033-2 (рус.).
2. [http://docs.oracle.com/cd/B10501\\_01/appdev.920/a96621/adx04paj.htm#1003830](http://docs.oracle.com/cd/B10501_01/appdev.920/a96621/adx04paj.htm#1003830)
3. <http://www.quizful.net/post/sax-parser-java>
4. [http://en.wikipedia.org/wiki/Object-relational\\_mapping](http://en.wikipedia.org/wiki/Object-relational_mapping)
5. [http://www.hibernate.org/hib\\_docs/reference/ru/html\\_single/](http://www.hibernate.org/hib_docs/reference/ru/html_single/)
6. <http://java-course.ru/student/book2/hibernate-annotation/>
7. Дейтел, Х.М. Технологии программирования на Java 2. Книга 3. Корпоративные системы, сервлеты, JSP, Web-сервисы [Текст] / Х.М. Дейтел, П.Дж. Дейтел, С.И. Сантари. – М. : Бином-пресс, 2003. – 672 с.
8. Перри, Б.У. Java сервлеты и JSP. Сборник рецептов [Текст] / Брюс У. Перри. – М. : Кудиц-пресс, 2009. – 768 с.
9. Соломон, М.К. Oracle. Программирование на языке Java [Текст] / Мартин К. Соломон, Нирва Мориссо-Леруа, Джули Басу. – М. : Лори, 2010. – 512 с.
10. Курवानян, Б. Программирование web-приложений на языке Java [Текст] / Буди Курवानян. – М. : Лори, 2009. – 880 с.
11. JavaEEDocumentation [Электронный ресурс]. – Режим доступа: <http://www.oracle.com/technetwork/java/javaee/documentation/index.html>, дата доступа: 13.12.2013.
12. JavaEETutorials [Электронный ресурс]. – Режим доступа: <http://www.oracle.com/technetwork/java/javaee/documentation/tutorials-137605.html>, дата доступа: 13.12.2013.
13. <http://orlova.rsue.ru/content/sst/j2ee1.htm>
14. [https://ru.wikipedia.org/wiki/HTTP\\_cookie](https://ru.wikipedia.org/wiki/HTTP_cookie)

15.<http://learn.javascript.ru/#book-toc-137>

16.[https://netbeans.apache.org/kb/docs/web/ajax-quickstart\\_ru.html](https://netbeans.apache.org/kb/docs/web/ajax-quickstart_ru.html)

17.<https://habr.com/ru/post/14246/>